Lesson 1: From Data to Information

Most of our interaction with the world relies on our ability to form an understanding of it; we need to know about it. As we have previously learned, our interactions with the world around us have three basic phases: sensing, processing, and responding. The sensing phase provides us with very basic signals from our environment: light, sound, touch, smell, and others. These are merely signals that we are capable of acquiring and converting into electrical signals our brain can use. It is only after we process these signals that they become meaningful; they become information. Prior to that, they were merely data.

Example 1.1

When we see something, it is initially light signals perceived as various colors and shapes. But once we process them, these colors and shapes become our best friend's face. The colors and shapes in the light signals now have meaning; the data was transformed into information.

> Data is any form of raw input that can be processed into meaningful information.

In this definition of 'data', the term 'meaningful' is added to emphasize that processing imparts meaning to the acquired data, thereby converting it into information. It is meaning that differentiates data from information. The same applies to computers. Regardless of whether a human or a computer is processing data, the act of processing is what converts data into information.

Example 1.2

A sensor inputs the following stream of numbers into a computer: ' 29, 28, 33, 37, 38, 39, 38, 31, 29, 28'. As a list of numbers, this data is meaningless. Let's begin to process it.

1. Identify the sensor that captured the data: Thermometer.
2. Label the data according to the sensor that captured it: Temperature (degrees Celsius).

So far, we have determined that this list of numbers is actually a list of temperatures. One would then want to know, what are these temperatures of?

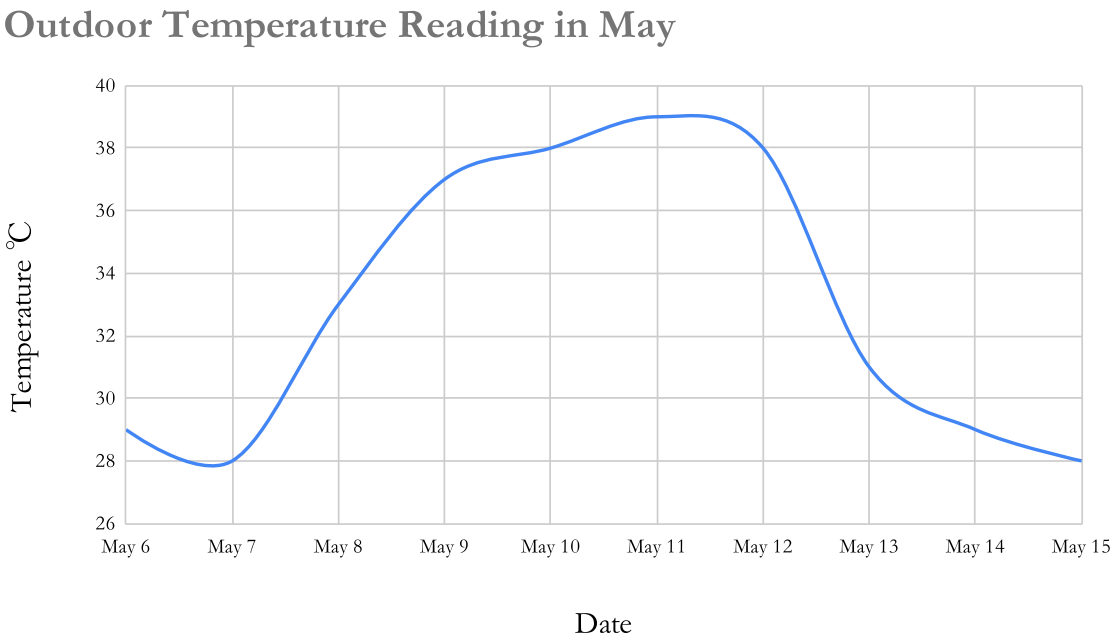
3. Location of sensor: Outdoor.

Now that we have gained more information, we know that these numbers are the outdoor temperatures. But when were these readings obtained? Typically, sensors would keep a date and time stamp with each reading. As part of our processing of this data, let us transform this list of temperature data into a table that contains the date and time each reading was taken.

4. Compile date, time and temperature data into a table.

Date	May 6	May 7	May 8	May 9	May 10	May 11	May 12	May 13	May 14	May 15
Time	12:00	12:00	12:00	12:00	12:00	12:00	12:00	12:00	12:00	12:00
Temp °C	29	28	33	37	38	39	38	31	29	28

Now we have realized that processing data by compiling it with other related data, in this case the date and time of each temperature reading, adds more information. We also learned that organizing data and presenting it in tabular form allows us to better visualize and understand the data. So, the presentation of data, which requires processing, is an added layer of information. Another method of presenting or visualizing data is charting.



So we have gone from pure raw data of numbers to information; we have learned that between May 8 and May 12, there was a sudden rise and fall in temperature in the area from which the data was collected. This leads us to one last step, interpreting the data. A sudden rise in temperature could be interpreted as a heat wave, and therefore the information learned from our data is that: 'between May 8 and May 12, a heat wave occurred in the area from which the data was collected.' This information was derived through the processing of data, which included identifying, organizing, visualizing, and interpreting the data.

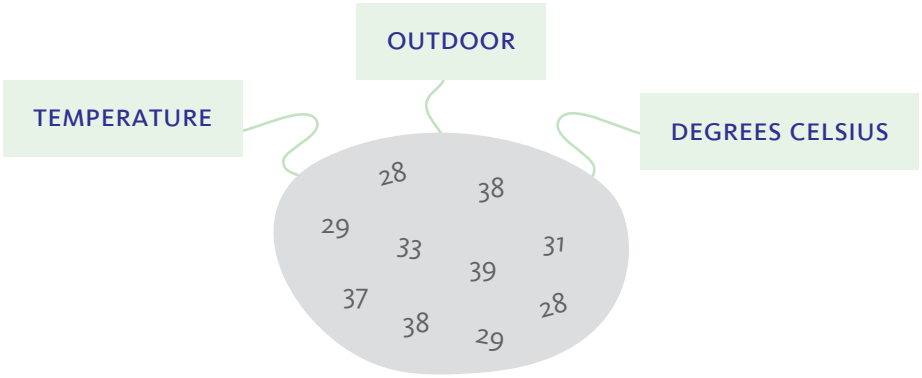
CONVERTING DATA TO INFORMATION INCLUDES IDENTIFYING, ORGANIZING, VISUALIZING, AND INTERPRETING THE DATA.

IDENTIFYING DATA

One of the initial steps in processing data is identifying what it represents. The information extracted from data is significantly influenced by how it is identified. In the previous example, the list of numbers: `29, 28, 33, 37, 38, 39, 38, 31, 29, 28`, could represent the number of students going on a field trip, or the number of views someone received on their YouTube posting. Therefore, identifying the data by determining its source and what it represents is critical, as it sets the foundation for all subsequent steps.

LABELING DATA

Labeling data is an essential part of identifying it. This helps prevent mistakes and ensures that everyone examining the data understands its context and meaning. So in the case of Example 1.2, the labels that would be required are: 'temperature', 'outdoor', and 'degrees Celsius'.



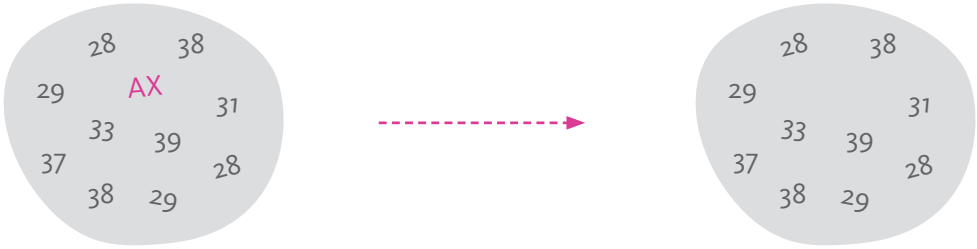
ORGANIZING DATA

Another important step in the process of extracting information is organizing the available data. There are several ways in which data can be organized: cleaning, structuring, and sorting.

CLEANING DATA

Cleaning data is the process of removing erroneous data. Errors can enter a dataset for several reasons: errors during input, such as a faulty sensor in an automated system or a mistake by a person entering the data manually. Other errors can arise while data is being stored, waiting to be processed. While there are other reasons errors can occur, the important point is that data should be screened and errors removed.

For example, what if our list of numbers read: `29, 28, 33, AX, 37, 38, 39, 38, 31, 29, 28` or `29, 28, 33, 300, 37, 38, 39, 38, 31, 29, 28`? In either case, the data items 'AX' and '300' do not fit for obvious reasons: an outdoor temperature must be a number, not letters, and 300°C is certaitnly too high a temperature to be recorded outdoors on Earth, unless maybe you are near a volcano or on a different planet, but that is not the situation.



Raw set of data

Cleaned set of data

STRUCTURING DATA

Data is generally stored in computers in a certain manner that determines how each item of data relates to the others. These are called data structures. We will explore them in greater detail later in this chapter, but for now, we will simply identify two data structures: lists and tables.

1. **List (Array)**: An arrangement of data in a linear fashion; all data items are positioned next to each other in a line.

2. **Table**: Well-suited to organize multiple sets of data. In Example 1.2, the initial organization of the temperature data was linear, with all the numbers presented in an array. But when other sets of data were included, such as the date and time, a table was used to organize the data.

Data structured as an array with one item following the other.

29, 28, 33, 37, 38, 39, 38, 31, 29, 28

Data structured as a table with rows and columns.

Date	May 6	May 7	May 8	May 9	May 10	May 11	May 12	May 13	May 14	May 15
Temp °C	29	28	33	37	38	39	38	31	29	28

SORTING DATA

Sorting plays an important role in data organization and processing. It is the process through which data is rearranged based on certain criteria, such as alphabetical order for alphanumeric data, increasing or decreasing value for numeric data, or by category for data that can be categorized. In the previous example, the data was already sorted. The sorting criterion was date, meaning that the temperature data was in chronological order based on the date it was acquired.

Data sorted chronologically by increasing dates.

Date	May 6	May 7	May 8	May 9	May 10	May 11	May 12	May 13	May 14	May 15
Temp °C	29	28	33	37	38	39	38	31	29	28

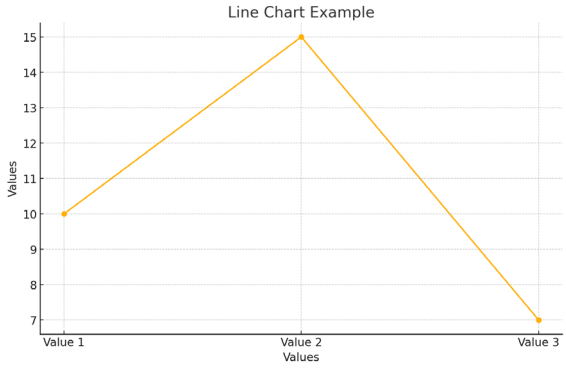
Data sorted by increasing temperature readings.

Date	May 7	May 15	May 6	May 14	May 13	May 8	May 9	May 10	May 12	May 11
Temp °C	28	28	29	29	31	33	37	38	38	39

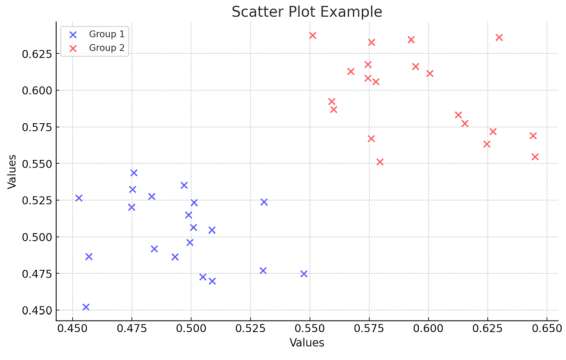
Each method of sorting allows us to better identify certain aspects of the data. Data sorted chronologically by increasing dates allows us to more easily identify trends over time, while data sorted by increasing temperature readings allows us to more easily identify the lowest and highest temperature readings collected in the dataset.

VISUALIZING DATA

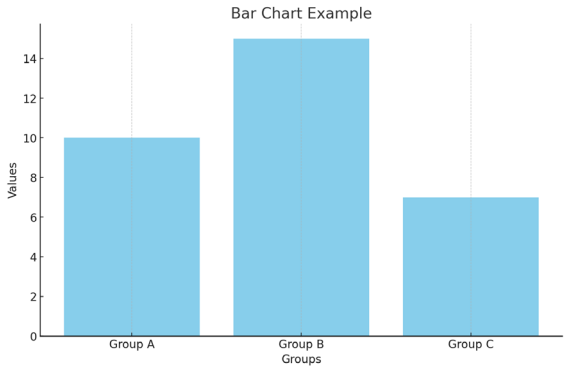
To visualize data means to present it in a way that makes it easy to visually identify trends and patterns. A common and effective method to visually present data is through charts. There are different types of charts, each presenting the data in a slightly different way. Four common types of charts are: line chart, scatter plot, bar chart, and pie chart.



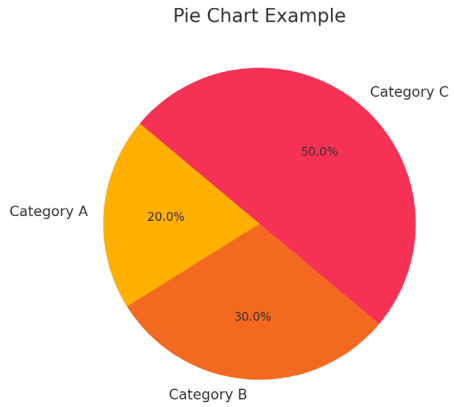
A **line chart** is a chart in which the value of each data point is represented by a point on the chart, and the points are then connected by a line. Line charts are useful for showing trends over time or continuous data.



A **scatter plot** is similar to a line chart in that the value of each data point is represented by a point on the chart. However, in a scatter plot, the points are not connected by a line. Instead, scatter plots are used to display the relationship between two continuous variables, making it easier to identify patterns, trends, and correlations.



A **bar chart** is a chart in which data is organized into groups or categories, with the value of each category represented by a bar. Bar charts can be either vertical or horizontal and are useful for comparing quantities across different categories.



A **pie chart** is a chart in which data is organized into groups or categories that are part of a whole. Each category is represented by a slice of the pie, which indicates the proportion of the whole that this category represents. Pie charts are useful for showing relative proportions and percentages of a whole.

The choice of the type of chart used to visualize data depends on the type of data itself. Generally, data can be broadly classified into two main types: quantitative and qualitative. Such classification influences the handling and presentation of the data.

Notes

Understanding the transformation of data into information is a crucial skill that extends beyond computer science, impacting fields like healthcare, business, finance, education, and environmental studies. This fosters critical thinking and analytical skills, enhancing our ability to interpret and act on complex data.

Quantitative Data

Quantitative data is numerical data that can be measured or counted, which means that it can either be continuous or discrete. Continuous data is essentially measurable data that can take any value within a range, such as temperature, height, and time. This type of data is best visualized using line charts and scatter plots. For example, in Example 1.2, a line chart was used to visually present the data because line charts are best suited for showing trends over time or continuous progression, such as the change in temperature over time.

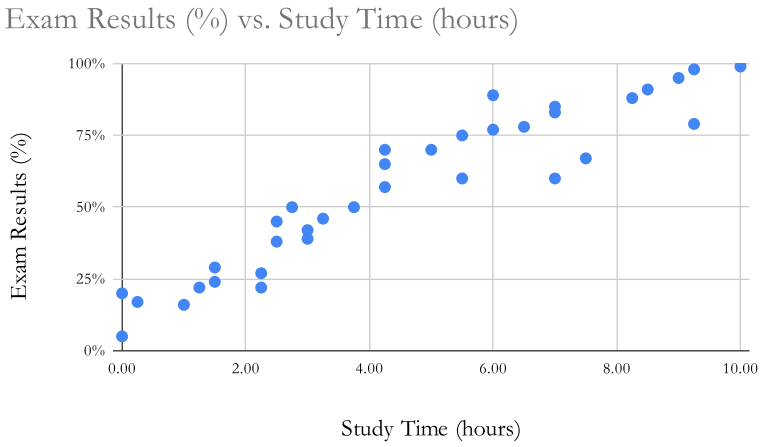
QUANTITATIVE DATA IS NUMERICAL DATA THAT CAN BE MEASURED OR COUNTED.

EXAMPLE 1.3

Data was collected from 35 students. Two sets of data were obtained: the hours of study for an exam and the results achieved on that exam. This is the raw data unprocessed in the form of a list, or more specifically, a paired list:

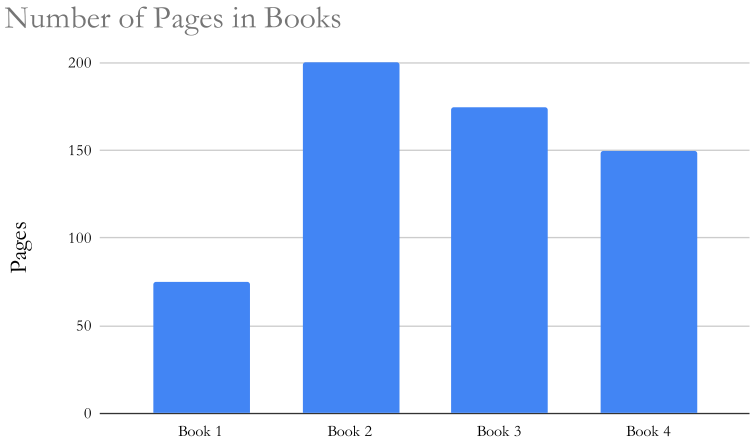
(3.00, 42%), (5.00, 70%), (7.00, 60%), (1.25, 22%), (2.50, 38%), (10.00, 99%), (6.00, 77%), (0.00, 20%), (8.50, 91%), (4.25, 70%), (0.00, 5%), (1.50, 24%), (6.50, 78%), (0.25, 17%), (2.75, 50%), (9.00, 95%), (9.25, 98%), (1.00, 16%), (2.25, 27%), (7.50, 67%), (9.25, 79%), (1.50, 29%), (2.50, 45%), (2.25, 22%), (7.00, 83%), (3.25, 46%), (6.00, 89%), (4.25, 65%), (3.00, 39%), (5.50, 75%), (3.75, 50%), (8.25, 88%), (4.25, 57%), (5.50, 60%), (7.00, 85%)

Looking at this data, it is unclear what information it holds. However, if it is processed and charted in the form of a scatter plot, the information it holds becomes better visualized.



The scatter plot of exam results versus study time reveals a pattern: as the hours of studying increase, so do the exam results. While it is not a perfect straight line, it does reveal a trend that is not noticeable in the raw data.

Discrete data, on the other hand, is typically countable data that can only take distinct values, such as the number of pages in a book or the number of students in a class. It is not continuous, meaning it cannot be in the form of fractions or decimals. For example, you cannot have 10 ½ pages in a book or 20.25 students in a class. The values of discrete data can only be whole numbers. In general, bar charts are a good choice for visually representing discrete data.



The bar chart allows for comparison of the number of pages in each book.

Qualitative Data

Qualitative data is descriptive data that can be observed but not measured. It is also referred to as categorical data because it can be categorized into groups and labeled, such as types of fruits, names of cities, colors of cars, and yes/no responses in a survey. This type of data is best visualized using bar charts and pie charts, which allow for a clear representation of categories and their frequencies. However, before it can be represented in charts, this type of data must be converted into quantitative data. The most common method of making this conversion is by counting the data items that fall within each category.

QUALITATIVE DATA IS DESCRIPTIVE DATA THAT CAN BE OBSERVED BUT NOT MEASURED.

EXAMPLE 1.4

The school's information management system is backed by a database that contains data about students attending the school, including their names, ages, grades, gender, and other pertinent information. The school has 618 students enrolled in Grades 1 through 12, which is too many to show in this book. Here are the first 6 entries in the database table:

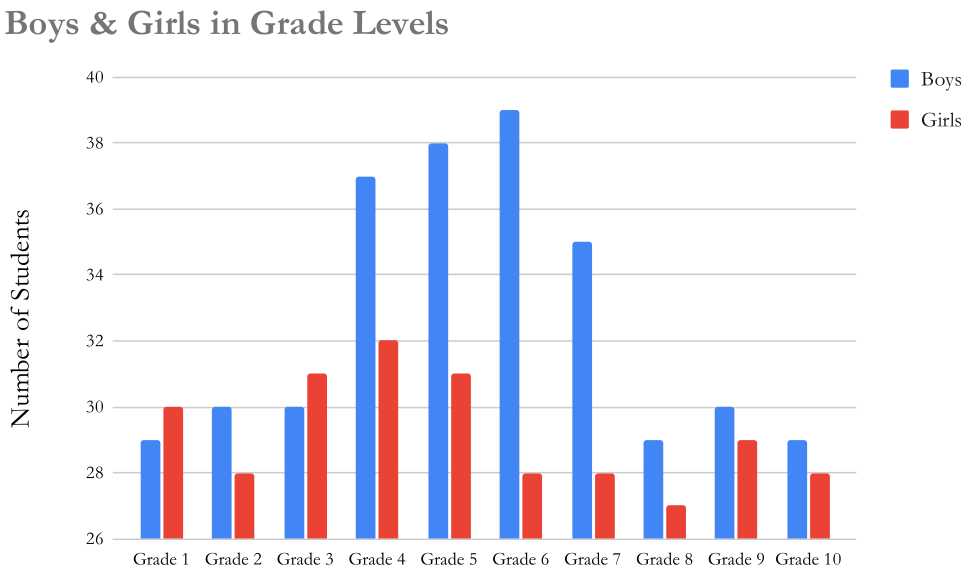
Name	Family	Age	Grade	Gender
Dana	Lankart	6	1	Girl
Ahmed	Dandash	6	1	Boy
Ruba	Najem	8	3	Girl
Tarek	Najem	10	5	Boy
Thomas	Johnson	9	4	Boy
Ghassan	Mekki	9	4	Boy

Based on this table, grade levels and gender are considered categorical qualitative data. The data items of grade levels are the names of each grade, such as 'Grade 1', 'Grade 2', and so on up to 'Grade 12'. The data items under gender are categorized as 'Boy' and 'Girl'. None of these data items are measurable or numerical in nature.

Assuming we want to learn the number of boys and girls in each grade level, the first step would be to query the database to count the number of occurrences of the data items 'Boy' and 'Girl' for each grade. This query results in the conversion of qualitative data into discrete quantitative data, from which we can extract the following information:

	Grade 1	Grade 2	Grade 3	Grade 4	Grade 5	Grade 6	Grade 7	Grade 8	Grade 9	Grade 10
Boys	29	30	30	37	38	39	35	29	30	29
Girls	30	28	31	32	31	28	28	27	29	28

Since the data can be grouped by grade levels (Grade 1 - Grade 10) and gender (boy, girl) it is better visualized using a bar chart. The resulting bar chart would look like this:

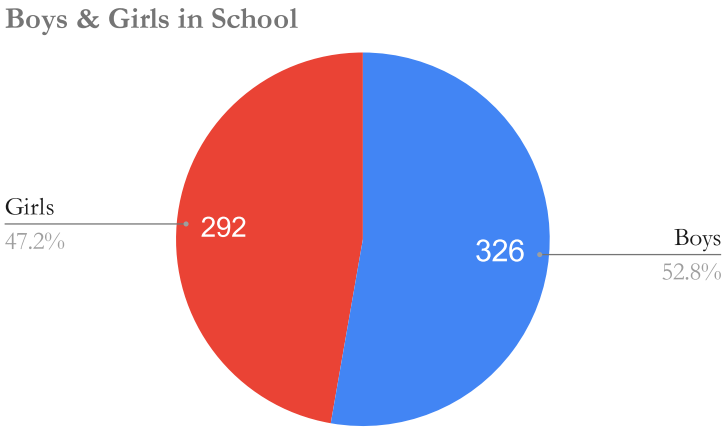


The bar chart reveals that the number of boys exceeds the number of girls, particularly in grades 4, 5, 6, and 7.

EXAMPLE 1.5

If, for the same data, we wanted to visualize the proportion of boys and girls in the whole school, we could use a pie chart. Since we are now interested in the entire school rather than the details of each grade level, a pie chart is appropriate. To obtain this information, we would add the number of boys across all grades and the number of girls across all grades. The two categories, 'Boys' and 'Girls,' can then be represented in a pie chart, showing that the number of boys and girls is nearly the same.

	Whole School
Boys	326
Girls	292
Total	618



INTERPRETING DATA

The final step in the process of converting raw data to actionable information is the interpretation of the findings. This step involves explaining what the data is presenting to us, whether it is a spike in outdoor temperature, a relationship between two sets of data such as study time and exam results, or a discrepancy between the proportion of boys and girls in a school. Interpreting these findings poses two crucial questions: "Why?" and "What should we do?"

The first question, "Why?" forces us to identify the factors underlying our findings. This step is essential for uncovering the reasons that have led to the

information we have discovered. It adds an extra layer of understanding that is necessary for a deeper comprehension of the world around us.

The second question, “What should we do?” prompts us to consider our next steps based on our findings. These next steps may involve further investigating the underlying causes or, if we are satisfied with our findings, learning from them and incorporating them into our decision-making process. This kind of information is referred to as ‘actionable information’, which is information that we can use to make informed decisions.

The process of converting raw data into information is fundamental to understanding and interpreting the world around us. By carefully following the basic stages of identifying, organizing, visualizing, and interpreting data, we can transform unprocessed data into actionable information regardless of the initial nature of the data, whether it is quantitative or qualitative. This process should guide our learning and decision-making, providing us with the analytical tools necessary to make informed decisions based on accurate and meaningful information.

Career Tips

Understanding the process of converting raw data into actionable information can open doors to a variety of rewarding careers, such as data analysts, data scientists, business intelligence developers, financial analysts, and market research analysts. All of these roles are in high demand in today’s data-driven world.

SUMMARY

The essential steps and processes for converting raw data into information are categorized into four main stages:

1. Identifying Data: This includes collecting, identifying, and labeling data.

2. Organizing Data: This includes cleaning, structuring, and sorting data.

3. Visualizing Data: Creating visual representations (charts, graphs) to highlight trends and patterns.

4. Interpreting Data: Analyzing visualized data to extract meaningful insights that lead to informed decision-making.

Data can be categorized as quantitative and qualitative.

Quantitative data is numerical data that can be measured (continuous) or counted (discrete).

Qualitative data is descriptive data that can be observed but not measured and is referred to as categorical data because it can be categorized into groups and labeled.

Four common charts used to visualize data are line charts, scatter plots, bar charts, and pie charts. Line charts and scatter plots are best for continuous quantitative data, while bar charts and pie charts are best for discrete and categorical data.

Activity 1.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
F	The process of converting raw data into information only includes organizing the data.
T	Labeling data helps prevent mistakes.
F	Continuous data can only take discrete values.
T	A line chart is best for showing trends over time.
F	Categorical data represents measurements that can take any value within a range.
T	A scatter plot is best suited for displaying continuous data.
T	Cleaning data involves removing erroneous data.
F	A pie chart is suitable for visualizing continuous data.
T	Sorting data arranges it for easy access and analysis.
T	Interpreting data involves explaining what the data is presenting.

Activity 1.2

Answer the questions below.

1. What is the first step in converting raw data into information?
   a) Visualizing Data
   b) Organizing Data
   c) Identifying Data
   d) Interpreting Data

2. Which of the following charts is best used for showing trends over time?
   a) Bar Chart
   b) Line Chart
   c) Pie Chart
   d) Scatter Plot

3. What is the primary purpose of cleaning data?
   a) To identify data
   b) To create charts
   c) To remove erroneous data
   d) To label data

4. Which type of data can take any value within a range?
   a) Categorical Data
   b) Discrete Data
   c) Continuous Data
   d) Nominal Data

5. What does a scatter plot best represent?
   a) Proportions within a whole
   b) Trends over time
   c) Relationships between two continuous variables
   d) Comparisons across categories

6. What type of chart would you use to compare quantities across different categories?
   a) Bar Chart
   b) Line Chart
   c) Pie Chart
   d) Scatter Plot

7. What type of chart is best for showing proportions of a whole?
   a) Bar Chart
   b) Line Chart
   c) Pie Chart
   d) Scatter Plot

8. Which of the following is a key aspect of data visualization?
   a) Removing erroneous data
   b) Arranging data in a list
   c) Presenting data in a visual format
   d) Labeling data points

9. Why is it important to interpret data after visualizing it?
   a) To ensure data is labeled correctly
   b) To derive meaningful insights and make decisions
   c) To remove duplicate entries
   d) To organize data in a table

Activity 1.3

Read the provided data examples. Classify each example as either quantitative or qualitative.

DATA EXAMPLES

1. *Types of Pets: Different kinds of pets owned by students.*
2. *Temperature: Daily temperatures recorded over a month.*
3. *Weight: The weight of different fruits*
4. *Brands of Cars: Different car brands owned by people.*
5. *Time: The time taken by athletes to complete a race.*
6. *Blood Types: Blood types of students.*

Quantitative : 2. Temperature, 3. Weight, 5. Time

Qualitative : 1. Types of Pets, 4. Brands of Cars, 6. Blood types

Activity 1.4

Identify 2 examples of quantitative data and 2 examples of qualitative data from your daily life.

Quantitative : Measure the time you spend on activities such as studying, exercising, and watching TV.

Qualitative : Record the different modes of transportation you use throughout the week such as bus, car, bike, and walking.

Activity 1.5

Examine the dataset provided in each question and determine the choice that best describes it.

1. List A = {12.5, 12.3, 11.7, 14.6, 15.1}
   a) Qualitative Categorical Data
   b) Quantitative Continuous Data
   c) Quantitative Discrete Data

2. List B = {red, blue, red, red, yellow, blue, blue, red}
   a) Qualitative Categorical Data
   b) Quantitative Continuous Data
   c) Quantitative Discrete Data

3. List C = {yes, yes, no, no, maybe, maybe, yes, yes, yes}
   a) Qualitative Categorical Data
   b) Quantitative Continuous Data
   c) Quantitative Discrete Data

4. List D = {2, 3, 7, 2, 4, 1, 1, 1, 4, 5}
   a) Qualitative Categorical Data
   b) Quantitative Continuous Data
   c) Quantitative Discrete Data

5. List E = {$\frac{1}{2}$, 8 $\frac{1}{2}$, 7, 8 $\frac{1}{4}$, 5 $\frac{3}{4}$}
   a) Qualitative Categorical Data
   b) Quantitative Continuous Data
   c) Quantitative Discrete Data

Activity 1.6

Examine the datasets provided and read the supporting narrative. Based on the information given, use a spreadsheet application to generate a chart that visually represents the data. Ensure that the data is appropriately organized, and that both your tabulated data and chart are clearly labeled. Identify patterns and draw conclusions from the data.
Refer to the spreadsheet 'IT Frontiers A Part 2 - Lesson 1 - Activity 1.6'.

1. Age_Height_List = { (11, 143), (12, 145), (13, 153), (14, 162), (15, 170), (16, 175), (17, 178), (18, 180), (19, 181), (20, 181) }

   Format: (age, height)

   Omar's pediatrician has been keeping track of his height from the age of 11 to 20 years. Every year, she records his height in centimeters.

2. Rainfall_Weight_List = { (12, 150), (10, 120), (20, 180), (11, 140), (15, 160), (13, 137), (19, 195), (10, 135), (18, 180), (11, 115), (17, 180), (14, 142), (13, 155), (13, 130), (17, 165), (11, 134), (14, 158), (12, 141), (16, 180), (16, 162) }

   Format: (rainfall, weight)

   A farmer records the weekly rainfall in millimeters and the weight of the fruit picked from the tree in grams. The weekly rainfall ranged between 10 and 20 mm.

3. Item_Count_List = { (Laptops, 200), (Smartphones, 300), (Tablets, 170), (Desktops, 150) }

   Format: (item, count)

   An electronics store manager keeps a record of the items in his store. He wants to present this data to his administration. He queries the database for a count of certain items and receives this response.

4. Coats_Beachwear_List = { (January, 220, 50), (February, 200, 60), (March, 170, 100), (April, 130, 150), (May, 100, 200), (June, 50, 250), (July, 40, 230), (August, 30, 210) }

   Format: (month, winter coats, beachwear)

   A fashion retailer wants to compare the sales of winter coats and beachwear over the last 8 months.

Notes

A list can contain items in the form of doubles, triplets, or more, which are referred to as tuples. Tuples are similar to lists, but the items in a tuple have a fixed order and cannot be changed.

Activity 1.7

Examine the provided datasets and supporting narrative. Using the information given, use a spreadsheet application to generate a chart that visually represents the data. Organize the data appropriately, ensuring that both your tabulated data and chart are clearly labeled. Identify and clean any erroneous data in the dataset. Finally, identify patterns in the data and draw conclusions. Refer to the spreadsheet 'IT Frontiers A Part 2 - Lesson 1 - Activity 1.7'.

1. Climate_Data_List = { (20th, 310, 14.5), (19th, 290, 14.0), (18th, 280, 13.8), (17th, 275, 13.6), (16th, 270, 13.5), (15th, 265, 13.3), (14th, 260, 13.2), (13th, 255, 13.1), (12th, 250, 13.0), (11th, 245, 12.9), (10th, 240, 12.8), (9th, 235, 12.7), (8th, 230, 12.6), (7th, 225, 12.5), (6th, 220, 12.4), (5th, 215, 12.3), (4th, 210, 12.2), (3rd, 205, 12.1), (2nd, 200, 12.0), (1st, 195, 11.9) }

   Format: (century, CO_2 level, temperature)

   Sarah is a climatologist. She is studying global warming and is collecting ice core samples from Antarctica to measure the carbon dioxide levels trapped in the ice for each century. She is also measuring the ratios of gases trapped in the ice samples to estimate global temperatures at the time the ice was formed.

2. Coral_Reef_List = { (25.0, 8.2), (24.8, 8.3), (25.2, 8.1), (24.5, 8.5), (26.0, 7.8), (24.3, 8.7), (24.4, 8.6), (25.0, 8.0), (24.2, 8.8), (25.5, 8.0), (24.5, 8.4), (26.2, 7.5), (25.8, 7.6), (24.1, 8.9), (24.5, 19.1), (25.2, 7.9), (24.9, 8.1), (26.5, 7.4), (24.7, 8.5), (24.3, 8.6), (26.0, 7.7), (24.8, 8.3), (24.5, 8.5), (25.1, 8.0), (26.3, 7.5), (24.0, 9.0) }

   Format: (temperature, O_2 level)

   Jordan is a marine biologist studying the health of coral reefs. He uses underwater sensors to monitor the temperature of the water and the amount of dissolved oxygen at various depths.

3. Movie_Opinion_List = { Great, Good, Okay, Great, Good, Bad, Awful, Great, Good, Good, Great, Good, Okay, Great, Good, Bad, 1734, Good, Great, Good, Good, Okay, Bad, Good, Great, Okay, Good, Awful, Great, Good, Great, Good, Great, Good, Okay, Great, Good, Bad, Okay, Good, Great, Okay, Good, Great, Good, Good, Great, Good, Okay, Good, Great, Good, Great, Bad, Good, Great, Good, Good, Great, Okay, Great, Bad, !!!!, Good, Great, Okay, Good, Great, Awful, Good, Great, Good, Great, Okay, Good, Great, Good, Good, Great, Bad, Good, Great, Good, Okay, Great, Good, Good, Great, Good, Bad, Okay, Great }

   A movie production company wants to gather audience feedback on their latest film. They categorize the feedback into five levels: Great, Good, Okay, Bad, and Awful. Note that this data is categorical and needs to be converted to discrete quantitative data before it is further processed.

LESSON 2: DATA STRUCTURES

In our current digital age, data is everywhere. From a simple image file on a smartphone to a complex database on a Google server, data surrounds us and is essential for the functioning of our apps and the generation of the information we rely on daily. However, data does not exist randomly in our computers; all stored data is organized in specific structures that allow us to organize, retrieve, modify, process, and store it efficiently. In this lesson, we will explore these data structures to gain a better understanding of their roles and importance.

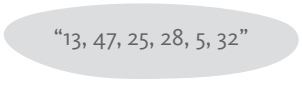
DATA STRUCTURES ARE SPECIALIZED FORMATS FOR ORGANIZING, RETRIEVING, MODIFYING, PROCESSING, AND STORING DATA.

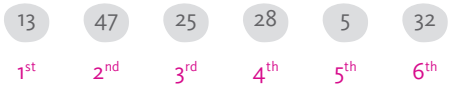
CLASSIFYING DATA STRUCTURES

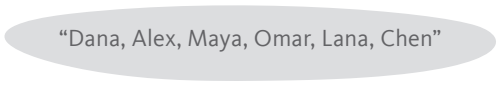
Data structures are broadly classified based on the arrangement of data items within the structure. When data items are arranged sequentially (e.g., 1st, 2nd, 3rd, and so on) like books on a shelf, such structures are referred to as linear, such as lists or arrays.

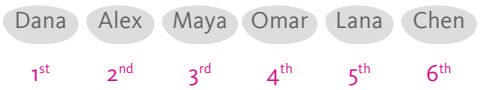
EXAMPLE 2.1

Data in a linear structure are arranged sequentially.





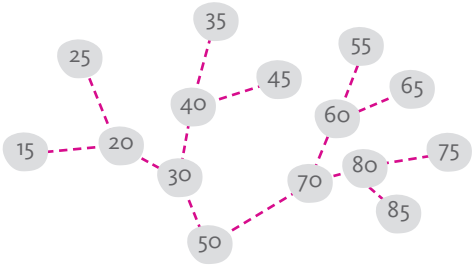




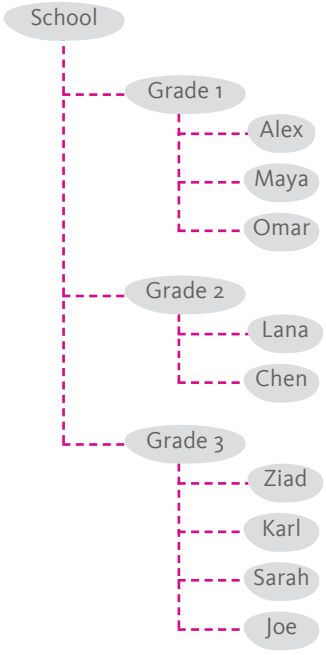
DATA STRUCTURES ARE CLASSIFIED AS LINEAR AND NON-LINEAR.

However, data items can be arranged in other ways. One such way is as a hierarchy, forming a structure similar to a tree, and therefore referred to as a tree data structure. In a tree, data is arranged along branches. Each branching point is referred to as a node, starting from the 'root node'. Each branching results in a new level in the hierarchy, with the root node existing at the first level.

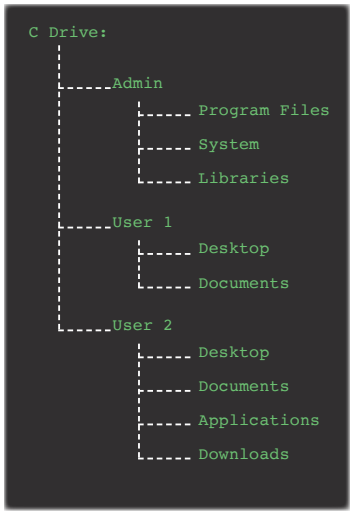
EXAMPLE 2.2



A binary search tree is used for efficiently searching databases and directories. This one has four levels. Can you count them?



This organization chart for a school is a tree data structure with three hierarchical levels.



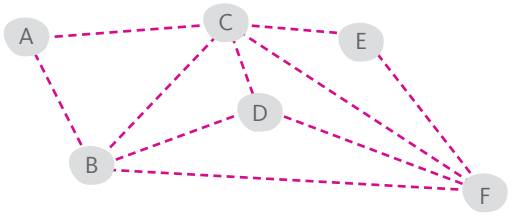
This computer directory is a tree data structure with three hierarchical levels.

In the tree data structure, data items are connected along branches, and items in one branch cannot connect to items in another branch. However, sometimes it is necessary for data items to freely interconnect so that any data item can connect to any other data item, forming a structure known as a graph data structure.

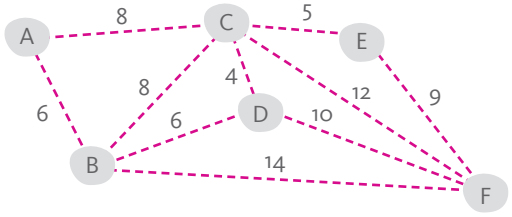
In a graph, each data point acts as a node and can point to one or more other nodes in the structure. Information about the link between two nodes, referred to as an edge, is also stored as data information at each node. This information about the edge could represent the distance between the nodes, the time it takes to travel between the two nodes, or other relevant metrics.

EXAMPLE 2.3

A map of flight routes between cities.



A graph data structure used to digitally represent the flight routes, with each city defined as a node.



The graph identifies the weight of each edge, with the number representing the hours of flight between two cities.

Adjacency List:

A: [(B, 6), (C, 8)]
B: [(A, 6), (C, 8), (D, 6), (F, 14)]
C: [(A, 8), (B, 8), (D, 4), (E, 5), (F, 12)]
D: [(B, 6), (C, 4), (F, 10)]
E: [(C, 5), (F, 9)]
F: [(B, 14), (C, 12), (D, 10), (E, 9)]

This is an adjacency list to represent the graph. For each node, the nodes connected to it are listed along with their edge weights.

ALGORITHMS & DATA STRUCTURES

Everything inside a computer is determined by instructions, or algorithms—sets of instructions specific for particular tasks. One such group of tasks involves controlling data structures. Algorithms control data structures. These algorithms define the rules for size, data type, and methods for adding, removing, and accessing items in various data structures, whether linear (like lists), hierarchical (like trees), or interconnected (like graphs).

ALGORITHMS CONTROL DATA STRUCTURES.

LINEAR STRUCTURES

There are five common linear data structures: the array, list, tuple, queue, and stack. Being linear structures, all five maintain their items in a sequential manner. However, they vary depending on the rules that determine their size, the type of data they can accept, and how items are inserted or removed.

THE ARRAY

The array is unique because it is defined by a fixed size and a homogeneous set of data. Once an array is created, the number of positions it contains remains constant. While the data items within an array can be changed or swapped, no new positions can be added or removed, maintaining the array's fixed size. Additionally, an array can only hold one type of data, meaning all items must be of the same type, either numeric or alphanumeric.

	Array	List	Stack	Queue
Size	Fixed	Dynamic	Dynamic	Dynamic
Data type accepted	one type	mixed	mixed	mixed
Insert data	not allowed	any position	end	end
Remove data	not allowed	any position	end	beginning

In contrast, the list, stack, and queue are dynamic data structures. This means that their sizes can change as data is inserted or removed. They are also more flexible regarding the types of data they can contain, allowing for a mixture of data types.

EXAMPLE 2.4

Maya, Sarah, and Leen are three sisters. Maya is 12 years old, Sarah is 9, and Leen is 14. If we wanted to use an array as a data structure to store their names and ages, we would need to use two separate arrays: one for the alphanumeric (string) data, which would store their names, and another for the numeric (number) data, which would store their ages. This is because an array cannot hold more than one type of data simultaneously.

The three sisters: Maya, Sarah, and Leen.

```
name_array = ["Maya", "Sarah", "Leen"]
age_array = [12, 7, 14]
```

Instructions for creating two arrays one for each type of data.

```
CREATE New Dataset
        Structure = Linear
        Size = Fixed
        Data Type = Numeric
        Elements = [12, 7, 14]
        Name = "age_array"
        Insert = Not Allowed
        Remove = Not Allowed
```

A hypothetical algorithm in pseudocode that would operate behind the scenes to create the 'age_array'.

Let's assume there was an error with Sarah's age—she is actually 9 years old, not 7. Although we cannot insert or remove data in an array, as that would change the array's size, we can swap out or modify the existing data items. In this case, we would simply update the value at the position corresponding to Sarah's age, correcting it from 7 to 9.

```
age_array = [12, 7, 14]
        UPDATE index (1) = 9
        PRINT age_array
END
```

```
OUTPUT:
12, 9, 14
```

The pseudocode instructions and the output for changing the value of the data held at position (1) in the array.

Note: Positions in a linear data structure are indexed, meaning each position is assigned a sequential number. Indexing typically starts at 0, so the first position in the structure has an index of 0.

The List

A list is another linear data structure that, unlike an array, has a dynamic size and can accept a mixture of data types—numeric, alphanumeric, or both. Being dynamic in size means that it can change by inserting or removing data items. In a list, this insertion and removal of data items can occur at any position.

Example 2.5

Jamal is Maya, Sarah, and Leen's 11-year-old brother. To add his name to the 'name_array' or his age to the 'age_array' is not possible, as that would change the size of the arrays. However, if these arrays were lists, then adding Jamal and his age would be possible.

Jamal is 11 years old.

```
name_list = [“Maya”, “Sarah”, “Leen”]
        INSERT index (2) = “Jamal”
        PRINT name_list
END
```

OUTPUT:
Maya, Sarah, Jamal, Leen

```
age_list = [12, 9, 14]
        INSERT index (2) = 11
        PRINT age_list
END
```

OUTPUT:
12, 9, 11, 14

Two sets of pseudocode instructions, each creating a list, inserting a new item at the third position (2), and then printing the list. Inserting a new item shifts the current items up by one position.

Note that each list contains only one data type, even though a list can simultaneously accept multiple data types.

If we want to sort the children in ascending order from youngest to oldest, we would need to shift the data items “Maya” and her age, 12, from their current positions (0) to the third position (2), currently occupied by “Jamal” and his age, 11.

```
age_list = [12, 9, 11, 14]
        REMOVE index (0) = 12
        INSERT index (2) = 12
        PRINT age_list
END
```

OUTPUT:
9, 11, 12, 14

```
name_list = [“Maya”, “Sarah”, “Jamal”,
“Leen”]
        REMOVE index (0) = “Maya”
        INSERT index (2) = “Maya”
        PRINT name_list
END
```

OUTPUT:
Sarah, Jamal, Maya, Leen

The two sets of pseudocode instructions show that we first sort the ‘age_list’ by changing the position of data item 12 from index (0) to (2). Then, the second set of instructions does the same with the name “Maya.” Now, both lists are sorted.

The last example, while it demonstrates how easily a list lends itself to the insertion and removal of data items, also reveals a limitation: when dealing with two related datasets, a list does not allow for associating between related data items, such as the name and age of each sibling. Even if we add both names and ages to one list, the related data items cannot be inherently associated and manipulated together.

THE TUPLE

The tuple is yet another linear data structure. It is similar to an array in that it has a fixed size, but it can accept different data types. The more interesting feature of a tuple is that it allows each position to be associated with specific data, effectively allowing related data items to be associated together.

EXAMPLE 2.6

Tuples can be used to combine the name and age of each sibling into one data unit with the format (name, age). Using such tuples would make the list more organized and make data manipulation easier. We have used brackets '[ ]' to define arrays and lists, whereas a tuple will be defined using parentheses '( )'.

```
sibling_list = [("Maya", 12), ("Sarah", 9), ("Jamal", 11), ("Leen", 14)]
       PRINT sibling_list
END
```

```
OUTPUT:
Maya, 12, Sarah, 9, Jamal, 11, Leen, 14
```

Using tuples to organize related datasets in a list ensures that the related data items, such as the name and age of each sibling, follow each other. This method helps maintain the association between data points and makes it easier to manage and manipulate the data within the list.

Now, if we want to sort the children in ascending order from youngest to oldest, we would still need to shift the data items "Maya" and her age, 12, from their current positions (0) to the third position (2), currently occupied by "Jamal" and his age, 11. However, since the data is now organized by tuples, we simply need to move the entire tuple containing Maya's data from its current position (0) to occupy the third position (2), which currently holds the tuple with Jamal's data.

This approach makes the sorting process more straightforward, as related data points remain together within each tuple, reducing the complexity of shifting multiple individual data items.

```
sibling_list = [("Maya", 12), ("Sarah", 9), ("Jamal", 11), ("Leen", 14)]
       REMOVE index (0) = ("Maya", 12)
       INSERT index (2) = ("Maya", 12)
       PRINT sibling_list
END
```

```
OUTPUT:
Sarah, 9, Jamal, 11, Maya, 12, Leen, 14
```

These instructions create a list of tuples, each formatted as (name, age). The data items in each tuple are fixed together in the order in which they were placed, with the name at index (0) and the age at index (1) of the tuple. The tuples are then placed in a list, with each tuple occupying one position in the list as if it were a single data item. So when Maya's data needs to be moved, it is moved together as a tuple from its position at index (0) to its new position at index (2).

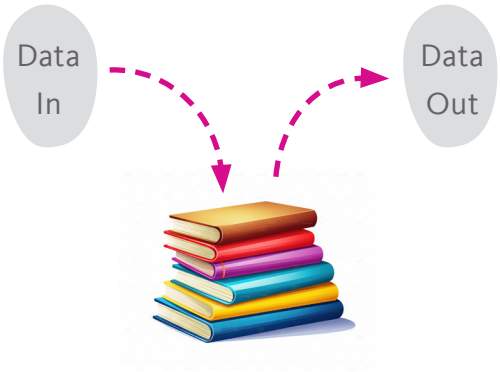
Notes

Creating a list of tuples in the same format is akin to creating a table, with each tuple acting as an entry on a row and the data items in each tuple placed according to their corresponding field or column.

THE STACK

The stack is similar to a list in that it has a dynamic size, meaning we can add to and remove items from it as needed, and it can accept data of different types, including both strings and numbers. The main difference is that in a stack, inserting and removing data items can only happen from the end of the stack. As a result, the last item added to the stack will be the first item to be removed; this is referred to as the Last In, First Out (LIFO) principle.

THE LIFO PRINCIPLE STATES THAT IN AN ORDERED SET OF ITEMS, THE LAST ITEM ADDED WILL BE THE FIRST ITEM REMOVED.



In a stack of books, the first book that can be removed is the last book that was added to the stack.

```
CREATE New Dataset
        Structure = Linear
        Size = Dynamic
        Data Type = Any
        Elements = ["A", "B", "C"]
        Name = "file_stack"
        Insert = To End
        Remove = From End
```

A hypothetical algorithm in pseudo-code that would operate behind the scenes to create a stack.

EXAMPLE 2.7

Lee works at a warehouse for an electronics store, where he maintains an up-to-date inventory of all products in stock. As products frequently move in and out of the warehouse, Lee must keep an accurate list of the current inventory.

One of the products in the warehouse is television sets. Each inbound TV is assigned a unique SKU code, which is entered into the warehouse computer system before the TV is placed on the shelf with other TVs waiting to be sold. The TVs are typically stacked, with the most recent arrival placed on top. When a client buys a TV, the one at the top of the stack is selected and shipped out to the client.

```
CREATE New Dataset
        Structure = Linear
        Size = Dynamic
        Data Type = Any
        Elements = []
        Name = "tv_stack"
        Insert = To End
        Remove = From End
```

```
OUTPUT:
tv_stack = []
```

The hypothetical pseudocode instructions would create a stack, a linear data structure that can grow to any length and is assigned the name 'tv_stack'.

It is initially empty and does not contain any data elements. Insertion and removal of data is defined to occur only at the end of the stack.

```
IF tv = inbound THEN
        OPEN tv_stack
        READ tv_sku
        INSERT index (END) = tv_sku
        SEND tv TO shelf
END
```

```
OUTPUT:
tv_stack = [
        "TV-55S4K-BLK-342"
]
```

When the first TV arrives at the warehouse, the computer system checks to confirm it is inbound. It reads the SKU barcode, 'tv_sku', and adds it to the end of the stack, 'tv_stack', which in this case is the first position since no other TVs have entered the warehouse. The TV is then placed on a shelf.

SKU: TV-55S4K-BLK-342

```
OUTPUT:
tv_stack = [
        "TV-55S4K-BLK-342"
        "TV-55S4K-BLK-110"
        "TV-55S4K-BLK-729"
        "TV-55S4K-BLK-845"
]
```

As more TVs arrive at the warehouse, the same process is repeated, and each new 'tv_sku' is added to the end of the stack 'tv_stack'.

Now that a client has purchased a TV, the warehouse needs to ship it out. The computer system receives the order, identifies the last TV set brought in, uses that TV's SKU to locate it on the shelf, and sends it for shipping to the client. This means the last TV set to arrive will be the first to go out, following the LIFO (Last In, First Out) principle.

```
OUTPUT:
tv_stack = [
        "TV-55S4K-BLK-342"
        "TV-55S4K-BLK-110"
        "TV-55S4K-BLK-729"
        "TV-55S4K-BLK-845"
]
```

The last 'tv_sku' is "TV-55S4K-BLK-845".

SKU: TV-55S4K-BLK-845

```
IF tv = outbound THEN
        OPEN tv_stack
        REMOVE index (END) = tv_sku
        FIND tv = tv_sku
        SEND tv TO client
END
```

```
OUTPUT:
tv_stack = [
        "TV-55S4K-BLK-342"
        "TV-55S4K-BLK-110"
        "TV-55S4K-BLK-729"
]
```

The computer system will open the `tv_stack`, check the last entry, remove that SKU, locate the TV with that SKU, and send it for delivery to the client.

Now, if another TV is sold, the next TV to leave the shelf will be the one with the SKU "TV-55S4K-BLK-729."

A data structure should be selected to match the nature of the data and the specific objectives for its use. When a real-world scenario operates under the Last In, First Out (LIFO) principle, a LIFO-based data structure, like a stack, is the most appropriate choice to effectively manage its data. In the electronics store warehouse, where the last product in is typically the first product out, the best data structure to manage the warehouse data is a stack.

The Queue

The queue is similar to the stack and the list in that it has a dynamic size and can accept data of different types, including both strings and numbers. However, the difference lies in the rules that govern the insertion and removal of data. While in a list, these operations can occur at any position, and in a stack, they occur only at the end, the queue follows the First In, First Out (FIFO) principle. This means that the first item added to the queue will be the first item to be removed.

THE FIFO PRINCIPLE STATES THAT IN AN ORDERED SET OF ITEMS, THE FIRST ITEM ADDED WILL BE THE FIRST ITEM REMOVED.

In a queue of people at the grocery store, the first person to stand in line is the first person served at the counter. Similarly, in a queue of data, this implies that data can only be removed from the beginning and inserted at the end.

Example 2.8

Jenna works at a warehouse for a grocery store. Unlike Lee, who works for an electronics store, many of the products Jenna handles are perishable and have a limited shelf life, such as apples, and should be shipped out as soon as possible before they go bad. Therefore, the first products that come in should be the first to go out.

A crate of apples

```
CREATE New Dataset
        Structure = Linear
        Size = Dynamic
        Data Type = Any
        Elements = []
        Name = "apple_queue"
        Insert = To End
        Remove = From Beginning
```

```
OUTPUT:
apple_queue = []
```

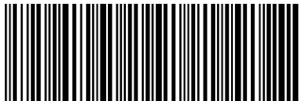
The hypothetical pseudocode instructions would create a queue, a linear data structure that can grow to any length and is assigned the name apple_queue.

It is initially empty and does not contain any data elements. Insertion is defined to occur at the end, while removal occurs from the beginning of the queue.

```
IF apple_crate = inbound THEN
        OPEN apple_queue
        READ crate_sku
        INSERT index (END) = crate_sku
        SEND apple_crate TO shelf
END
```

```
OUTPUT:
apple_queue = [
        "APL-1025-20M-012"
]
```

When the first crate of apples arrives at the storage facility, the computer system checks to confirm it is inbound. It reads the SKU barcode on the crate, 'crate_sku', and adds it to the end of the queue, 'apple_queue', which in this case is the first position since this is the first inbound apple crate. The apple crate is then placed in a storage.



SKU: APL-1025-20M-012

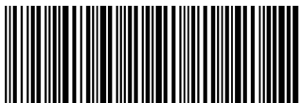
```
OUTPUT:
apple_queue = [
        "APL-1025-20M-012"
        "APL-1025-20M-327"
        "APL-1025-20M-445"
        "APL-1025-20M-129"
]
```

As more apple crates arrive, the same process is repeated, and each new 'crate_sku' is added to the end of the queue 'apple_queue'.

Now that a client has purchased two crates of apples, they need to be shipped out. The computer system receives the order, identifies the first two apple crates that arrived, uses those crate SKUs to locate them in the storage area, and sends the crates for shipping to the client. This means the first two apple crates to arrive will be the first ones to go out, following the FIFO (First In, First Out) principle.

```
OUTPUT:
apple_queue = [
        "APL-1025-20M-012"
        "APL-1025-20M-327"
        "APL-1025-20M-445"
        "APL-1025-20M-129""
]
```

The first two 'crate_sku' codes are: "APL-1025-20M-012" and "APL-1025-20M-327".



SKU: APL-1025-20M-012

SKU: "APL-1025-20M-327"

```
IF apple_crate = outbound THEN
        OPEN apple_queue
        REMOVE index (END) = crate_sku
        FIND apple_crate = crate_sku
        SEND apple_crate TO client
END
```

```
OUTPUT:
apple_queue = [
        "APL-1025-20M-445"
        "APL-1025-20M-129""
]
```

The computer system will open the `apple_queue`, check the first two entries, remove that SKU codes, locate the apple crates with those SKU codes, and send them for delivery to the client.

Now, if another apple crate is sold, the next apple crate to leave the storage will be the one with the SKU "APL-1025-20M-445".

At the electronics store, it was more efficient to remove the TV set on top of the stack to save time, rather than shifting all the TV sets to reach the bottom or the first one that came in. In this scenario, using a FIFO approach would waste time. However, with the apples, it was crucial to get the oldest apples out of storage first, as the longer they remain, the more likely they are to spoil; a LIFO approach would not have worked.

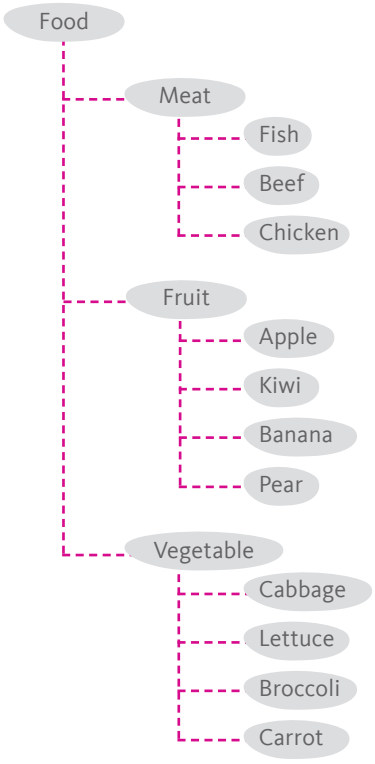
This illustrates that data structures are not just digital constructs but are essential tools designed to meet specific real-world needs. Choosing the right data structure depends on the context and the objectives, whether it's maximizing efficiency or ensuring the freshness of products.

TREE STRUCTURES

When data is structured using linear constructs, items are arranged in a sequence, with each item following the next. These structures are well-suited for representing real-world scenarios that require lists, stacks, and queues. However, the real world often presents more complex situations where data items need to be connected and organized in ways beyond a simple linear sequence.

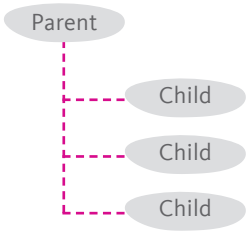
EXAMPLE 2.9

While the warehouse may rely on a stack or queue to efficiently manage inbound and outbound products, maintaining a full inventory of its stock requires a more complex data structure—one that can accommodate various categories and subcategories of products. The tree data structure is well-suited for this purpose, as it allows data to be organized hierarchically in levels that effectively support the categorization and subcategorization of items along branches.

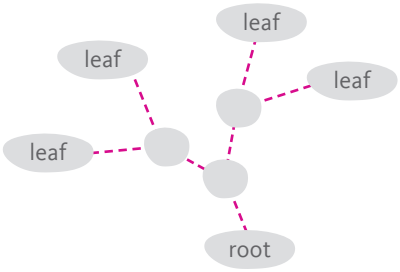


This is the organization chart for the food inventory at the warehouse where Jenna works. It is a tree structure with three hierarchical levels, each containing a category or subcategory of a food product.

In a tree structure, data items are referred to as nodes. A node functions as a data point that, in its most basic form, includes the data itself and additional information that points to other nodes to which this node is connected. Nodes that precede it are referred to as parent nodes, while nodes that follow are known as child nodes. The original parent node from which all other nodes originate is called the root node. Nodes at the very end of the tree structure, which do not have any child nodes, are referred to as leaf nodes.



A parent node with three child nodes.



A tree data structure with the root node and leaf nodes labeled.

A NODE IS A DATA CONSTRUCT THAT HOLDS BOTH DATA AND REFERENCES TO OTHER NODES.

Let's construct Jenna's warehouse food inventory as a tree data structure. The first step would be to create the file that would hold the dataset.

```
CREATE New Dataset
       Structure = Tree
       Size = Dynamic
       Data Type = Any
       Elements = []
       Node = (parent_index, data)
       Name = "food_inventory"
       Insert = any position
       Remove = any position
```

```
OUTPUT:
food_inventory = []
```

These instructions create a dynamic file named food_inventory with no initial elements and allow data of any type to be inserted or removed from any position. The instructions also define the format of each node to contain the data and identify its parent nodes.

The instructions present a simplified method for defining a node. A tuple is used to contain the data, along with information about the parent nodes of the node being defined. It also employs the indexing method used in linear data structures to reference the position of each node within its hierarchical level and along its branch.

```
food_inventory = []
        INSERT node (R, “Food”)
        PRINT food_inventory
END
```

```
OUTPUT:
Food
```

These instructions insert the root node, identified by the letter ‘R’, into an empty food_inventory tree and establish “Food” as the data associated with that node.

```
food_inventory = [(R,”Food”)]
        INSERT node (R_0, “Meat”)
        INSERT node (R_1, “Fruit”)
        INSERT node (R_2, “Vegetable”)
        PRINT food_inventory
END
```

```
OUTPUT:
Food
    |_____ Meat
    |_____ Fruit
    |_____ Vegetable
```

Three additional nodes are inserted as child nodes of the root, creating a new level in the hierarchy. Each node is indexed based on its position in this level, starting from 0.

```
food_inventory = [(R,”Food”),
        (R_0, ”Meat”),
        (R_1, “Fruit”),
        (R_2, “Vegetable”)
        ]
        INSERT node (R_1_0, “Apple”)
        PRINT food_inventory
END
```

```
OUTPUT:
Food
    |_____ Meat
    |_____ Fruit
              |_____ Apple
    |_____ Vegetable
```

These instructions further expand the food_inventory tree by adding a new child node under the existing “Fruit” node. The new node, labeled R_1_0 with the data “Apple,” is inserted as a child of R_1 (“Fruit”).

Removing data from a tree is straightforward when the data exists within a leaf node, meaning there are no other nodes dependent on it. However, if the data is within a parent node, removing it will likely result in all its dependent child nodes being deleted as well. This is because the child nodes rely on the parent node to maintain their position and relationship within the tree structure.

```
food_inventory = [(R,"Food"),
       (R_0, "Meat"),
       (R_1, "Fruit"),
       (R_2, "Vegetable")
       (R_1_0, "Apple"
       ]
       REMOVE node (R_1)
       PRINT food_inventory
END
```

```
OUTPUT:
Food
   |_____ Meat
   |_____ Vegetable
```

The food_inventory tree initially contains nodes representing categories like "Meat," "Fruit," and "Vegetable." When the instruction 'REMOVE node (R_1)' is executed, the "Fruit" node (R_1) is removed from the tree. As a result, all child nodes under "Fruit," including "Apple" (R_1_0), are also deleted.

Nodes can change position in a tree as long as their child nodes are moved with them.

```
food_inventory = [(R,"Food"),
       (R_0, "Meat"),
       (R_1, "Fruit"),
       (R_2, "Vegetable")
       (R_1_0, "Apple"
       ]
       SWAP node (R_1),(R_2)
       PRINT food_inventory
END
```

```
OUTPUT:
Food
   |_____ Meat
   |_____ Vegetable
   |_____ Fruit
            |_____ Apple
```

The instruction 'SWAP node (R_1), (R_2)' swaps the positions of the "Fruit" (R_1) and "Vegetable" (R_2) nodes. As a result, "Vegetable" will move into the position previously occupied by "Fruit," and vice versa, while all their respective child nodes (such as "Apple" under "Fruit") are moved along with them, maintaining the hierarchical integrity of the tree structure.

GRAPH STRUCTURES

In a tree, data is placed in nodes that are arranged along branches. Nodes can only connect to other nodes along their branch; they cannot directly connect to nodes in different branches. In contrast, in a graph, data is also placed in nodes, but the connections between nodes are not restricted. Nodes can interconnect freely, allowing for more complex relationships between data points.

In a tree data structure, we gave nodes a simplistic format that contains a reference to parent nodes and the data itself (parent_index, data). In a graph data structure, nodes have more complex connections, so a node will need to contain more information about linked nodes. It includes a list of all the indices of linked nodes, formatted as (node_index, data, linked_node1, linked_node2, ...).

EXAMPLE 2.10

Now that Jenna has used a queue to organize inbound and outbound food products, ensuring their freshness by minimizing shelf time, and has utilized a tree to create an inventory for all products in the warehouse, she now needs to manage the delivery process from the warehouse to her clients. For this task, she will need a graph to represent the various client destinations on the map.

Jenna's warehouse is in Seville, in the south of Spain. Her clients are in the cities of Madrid, Barcelona, and Valencia. Each of these four cities can be represented as a node in a graph data structure, and the roads connecting the cities can be represented as links between the nodes of the graph.

Just like with any data structure that uses nodes, nodes in a graph must be indexed. Typically, indexing is done using numbers, but that's not always the case—characters such as letters can also be used, as seen in the previous example where the letter R was used to refer to the root node. In a graph structure, there is no root node, and in fact, any node can act as a root. In this example, we will use letters to index the nodes. The letters A, B, C, and D will refer to the nodes representing the cities of Seville, Madrid, Barcelona, and Valencia, respectively.

One method to approach constructing a graph is by creating a list of each node and every node connected to it, known as an adjacency list. In this example, each node represents a city, and the cities are connected by roads. For the sake of this example, we will assume that the road between Madrid and Valencia is still under construction and cannot be used or included.

Adjacency List using City Names:
Seville is connected to: Madrid, Barcelona, and Valencia
Madrid is connected to: Seville and Barcelona
Barcelona is connected to: Seville, Madrid, and Valencia
Valencia is connected to: Seville and Barcelona

Adjacency List using Indices:
A: [B, C, D]
B: [A, C]
C: [A, B, D]
D: [A, C]

But graphs can carry more information—not just the data at each node, such as the name of the city, but also details about the links between nodes, referred to as 'edges'. In this example, the edges between nodes could represent data such as the distance between cities. This additional information allows the graph to model real-world scenarios more accurately, where the relationships between entities (like cities) are as important as the entities themselves.

IN A GRAPH, AN EDGE IS A LINK BETWEEN TWO NODES THAT CARRIES ADDITIONAL INFORMATION ABOUT THEIR RELATIONSHIP.

Distances between cities:

Seville - Madrid:	538 km
Seville - Barcelona:	1,031 km
Seville - Valencia:	653 km
Madrid - Barcelona:	621 km
Valencia - Barcelona:	351 km

Numerical Value Assigned to Each Edge:

AB, BA = 538
AC, CA = 1,031
AD, DA = 653
BC, CB = 621
CD, DC = 351

This new data about the distances between cities will be included as additional information regarding the edge between each node. Accordingly, the adjacency list will be updated to include the edge data.

Updated Adjacency List:

A: [(B, 538), (C, 1,031), (D, 653)]
B: [(A, 538), (C, 1,031)]
C: [(A, 1,031), (B, 621), (D, 351)]
D: [(A, 653), (C, 351)]

To construct Jenna's delivery map as a graph data structure, the first step is to create the file that will hold the dataset.

```
CREATE New Dataset
        Structure = Graph
        Size = Dynamic
        Data Type = Any
        Elements = []
        Node = (node_index, data, (linked_node, edge))
        Name = "delivery_map"
        Insert = any position
        Remove = any position
```

```
OUTPUT:
delivery_map = []
```

These instructions create a dynamic file named delivery_map with no initial elements and allow data of any type to be inserted or removed from any position. The instructions also define the format of each node to contain the data and identify its connected nodes and their edges.

Once the graph has been created, the first node can be inserted. Since there are no other nodes to link to, the first node would be defined as (A, “Seville”), with A as the index of the node and “Seville” as the data it contains.

```
delivery_map = []
        INSERT node (A,“Seville”)
END
```

```
OUTPUT:
delivery_map = [(A,”Seville”)]
```

These instructions insert the first node, identified by the letter ‘A’, into an empty delivery_map graph and associate the city name “Seville” with that node.

Since there are no other nodes yet, no additional data, such as node indices or edge information, can be included at this stage.

The next step is to insert the remaining three nodes that represent the cities of Madrid, Valencia, and Barcelona, along with the distances between them as values for the edges connecting the nodes.

```
delivery_map = [(A,“Seville”)]
        INSERT node (B,“Madrid”,(A,538))
        INSERT node (C,“Barcelona”,(A,1031))
        INSERT node (D,“Valencia”,(A,653))
END
```

```
OUTPUT:
delivery_map = [
        (A,“Seville”,(B,538),(C,1031),(D,653)),
        (B,“Madrid”,(A,538))
        (C,“Barcelona”,(A,1031))
        (D,“Valencia”,(A,653))
]
```

With the insertion of the three nodes “Madrid” (B), “Barcelona” (C), and “Valencia” (D), the graph now includes all four nodes, with each node identifying the nodes it is linked to and their corresponding edges.

Note that the A node is automatically updated to include these connections, reflecting the distances to each city.

If Jenna wants to consider the distance her delivery truck needs to travel to deliver to her clients in Madrid, Barcelona, and Valencia, she can have the computer system perform the necessary calculations now that the dataset has been defined.

```
delivery_map = [
        (A,"Seville",(B,538),(C,1031),(D,653)),
        (B,"Madrid",(A,538))
        (C,"Barcelona",(A,1031))
        (D,"Valencia",(A,653))
        ]
        Trip_1 = AB + BA
        Trip_2 = AC + CA
        Trip_3 = AD + DA
        Trip_All = Trip_1 + Trip_2 + Trip_3
        PRINT Trip_All
END
```

```
OUTPUT:
4444
```

These instructions calculate the total distance Jenna's delivery truck would travel to visit each client in Madrid, Barcelona, and Valencia, and then return to Seville.

The distances for each round trip (Seville to Madrid and back, Seville to Barcelona and back, and Seville to Valencia and back) are summed, resulting in a total trip distance of 4,444 km, which is then printed as the final output.

Jenna realized that the total distance of 4,444 km is quite long. She decided to consider an alternative delivery route that started in Seville, passed through each of the three delivery destinations—Madrid, Barcelona, and Valencia—before returning to the warehouse in Seville. To calculate this route, she would need to include the distances between Madrid and Barcelona, as well as between Barcelona and Valencia.

```
delivery_map = [
        (A,"Seville",(B,538),(C,1031),(D,653)),
        (B,"Madrid",(A,538))
        (C,"Barcelona",(A,1031))
        (D,"Valencia",(A,653))
        ]
        INSERT edge (B:C,621)
        INSERT edge (C:D,351)
        PRINT delivery_map
END
```

```
OUTPUT:
delivery_map = [
    (A,"Seville",(B,538),(C,1031),(D,653)),
    (B,"Madrid",(A,538),(C,621)),
    (C,"Barcelona",(A,1031),(B,621),(D,351)),
    (D,"Valencia",(A,653),(C,351))
]
```

The instructions update the delivery_map by inserting edges between Madrid (B) and Barcelona (C) with a distance of 621 km, and between Barcelona (C) and Valencia (D) with a distance of 351 km.

Now that the required data has been entered into the graph, Jenna can have the distance for the alternate route calculated.

```
delivery_map = [
    (A,"Seville",(B,538),(C,1031),(D,653)),
    (B,"Madrid",(A,538),(C,621)),
    (C,"Barcelona",(A,1031),(B,621),(D,351)),
    (D,"Valencia",(A,653),(C,351))
    ]
    Trip_All = AB + BC + CD + DA
    PRINT Trip_All
END
```

```
OUTPUT:
2163
```

The instructions calculate the distance of the alternate route by summing the values of the edges between each of the four nodes: AB, BC, CD, and DA. The calculated distance for this route is 2,163 km, significantly less than the initial route, making it a more efficient option for deliveries.

The use of a graph helped organize the data in a way that allowed for the calculations Jenna needed to make an informed decision on which route to take. At the warehouse, Jenna used a linear queue to manage inbound and outbound products, a tree to manage inventory, and a graph to manage delivery routes. These simple examples illustrate how data structures are crucial in the process of converting data into actionable information.

Conclusion

In conclusion, this lesson and the examples provided emphasize that the selection of a data structure is not arbitrary; it must be carefully aligned with the nature of the data and the specific requirements of the task at hand. Linear structures, such as arrays, lists, and stacks, are straightforward and efficient for managing sequential data, while trees and graphs offer the complexity needed to handle more complex relationships, such as hierarchical or interconnected data. Therefore, it is crucial to select the most appropriate structure to accurately represent and manage the underlying data.

SUMMARY

Data structures are specialized formats for organizing, retrieving, modifying, processing, and storing data.

Algorithms control these data structures by defining the rules for size, data type, and methods for inserting, removing, and accessing items.

Data structures are broadly classified into linear (array, list, tuple, stack, and queue) and non-linear (tree and graph) categories.

In linear data structures, data is arranged sequentially.

- Array: Fixed size, accepts one data type, and does not allow the insertion or removal of data after creation.
- List: Dynamic size, accepts mixed data types, and allows insertion or removal of data from any position.
- Tuple: Fixed size and format, accepts mixed data types, but does not allow the insertion or removal of data after creation.
- Stack: Dynamic size, accepts mixed data types, and allows data to be added or removed only from the end, following the Last In, First Out (LIFO) principle.
- Queue: Dynamic size, accepts mixed data types, allows data to be added at the end and removed from the beginning, following the First In, First Out (FIFO) principle.

In non-linear data structures like trees and graphs, data is held in nodes that link data together.

- Tree: Nodes are linked to form a hierarchical structure of parent-child relationships arranged in branches.
- Graph: Nodes can connect freely, allowing more complex relationships between data points.

A node is a data construct that contains both data and references to other nodes, making it essential in organizing and navigating data in non-linear structures.

In a graph, an edge is a link between two nodes that carries additional information about their relationship.

Activity 2.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
F	An array can hold multiple data types within the same structure.
T	A list has a dynamic size, meaning you can add or remove items from it.
F	A stack follows the First In, First Out (FIFO) principle.
T	In a tree structure, the root node is the node from which all other nodes descend.
F	A queue allows data to be added at any position and removed from any position.
T	In a graph, an edge represents a connection between two nodes and can carry additional information.
F	Tuples can have a dynamic size and allow for the addition of new data after creation.
F	Nodes in a tree can connect directly to nodes in different branches.
T	The Last In, First Out (LIFO) principle is applied when using a stack data structure.
F	A graph structure is more suitable than a tree structure for representing hierarchical data.

Activity 2.2

Answer the questions below.

1. Which data structure follows the Last In, First Out (LIFO) principle?
   a) Queue
   b) Stack
   c) List
   d) Array

2. Which is a characteristic of an array?
   a) It can hold multiple data types
   b) It has a dynamic size
   c) It can only hold one data type
   d) It allows insertion and deletion of data at any position

3. In a tree structure, what is the topmost node called?
   a) Leaf node
   b) Child node
   c) Root node
   d) Parent node

4. Which data structure is most suitable for managing hierarchical data?
   a) Tree
   b) Graph
   c) Array
   d) Stack

5. What is the primary feature of a tuple?
   a) It has a dynamic size
   b) It follows the First In, First Out (FIFO) principle
   c) It can be modified after creation
   d) It has a fixed format

Activity 2.3

Answer the questions below.

1. Which data structure follows the First In, First Out (FIFO) principle?
   a) Stack
   b) Queue
   c) Tuple
   d) List

2. What does an edge represent in a graph?
   a) The data stored in a node
   b) The hierarchical level of a node
   c) The connection between two nodes
   d) The index of a node

3. Which of these data structures can freely connect nodes?
   a) Tree
   b) Graph
   c) Stack
   d) Queue

4. Which data structure is best suited for representing a subway network?
   a) Tree
   b) Array
   c) Stack
   d) Graph

5. What happens when a node with child nodes is removed from a tree structure?
   a) The tree collapses.
   b) The child nodes become root nodes.
   c) The tree becomes a graph.
   d) The child nodes are automatically removed.

Activity 2.4

Match each data structure with the statement that best describes it.

Queue	**Graph**	**Stack**
Tree	**Tuple**	**List**

1. A fixed-size, ordered collection of the same data type. List
2. A hierarchical structure where each element is a node. Tree
3. A structure that follows the FIFO principle. Queue
4. It has a fixed size and fixed format. Tuple
5. A structure that follows the LIFO principle. Stack
6. A flexible structure where nodes are connected by edges. Graph

Activity 2.5

Match each data structure with the task for which it is best optimized.

Tree	**Array**	**Tuple**
Queue	**Stack**	**Graph**

1. Handling tasks in the order they arrive, like print jobs. Queue
2. Modeling networks, like social connections. Graph
3. Organizing a file system, like a computer's directory. Tree
4. Storing the coordinates of a point in a 2D space as (x, y). Tuple
5. Storing a city's monthly average temperatures for a year. Array
6. Managing undo operations in a software application. Stack

Activity 2.6

Answer the questions below about the following dataset, which represents the average monthly temperatures (in °C) for a city over a year.

```
monthly_temp = [2.5, 3.0, 7.8, 12.5, 16.4, 20.1, 23.3, 22.8, 18.5, 13.2, 8.3, 4.1]
```

1. How many data items are in the array? 12
2. What type of data is in the array? numeric
3. What data item is in the fourth position? 12.5
4. What was the temperature in September? 18.5
5. What would be the output of this instruction? 8.3

```
PRINT index (10)
```

6. Assume there was an error with the average temperature for February—it is actually 3.2°C rather than 3.0°C. Review the following instructions and determine which ones can be implemented and which cannot. Explain.

```
REMOVE index (1) = 3.0
INSERT index (1) = 3.2
```

```
UPDATE index (1) = 3.2
```

The remove and insert instructions cannot be implemented because they would change the size of the array, which is not allowed. However, updating the value held at a particular position is allowed as it does not affect the size of the array.

Activity 2.7

Answer the questions below about the following dataset, which represents a grocery shopping list.

```
grocery_list = ["Apples", "Bread", "Milk", "Eggs", "Cheese", "Tomatoes"]
```

1. How many data items are in the list? 6

2. What type of data is in the list? alphanumeric

3. What data item is in the third position? Milk

4. What is the index of the data item "Cheese"? 4

5. Fill in the blanks with the data items from the list and write the index beneath each.

Apples	Bread	Milk	Eggs	Cheese	Tomatoes
0	1	2	3	4	5

6. Assume the eggs were removed from the grocery cart. To modify the list, you must:

   a) Remove all data items before and including index (3).

   b) Remove all data items after and including index (3).

   c) Remove the data item at index (3).

   Update the list accordingly.

Apples	Bread	Milk	Cheese	Tomatoes	
0	1	2	3	4	

7. Pears were added to the grocery cart. Write a pseudocode instruction that will add "Pears" to the list to follow "Apples". Then update the list.

INSERT index (1) = "Pears"

Apples	Pears	Bread	Milk	Cheese	Tomatoes
0	1	2	3	4	

Activity 2.8

Answer the questions below about the undo stack of a text editor.

```
undo_stack = [
        "Italicize 'World'",
        "Type 'World'",
        "Bold 'Hello'",
        "Type 'Hello'"
]
```

1. How many actions were taken by the user? 4

2. What was the first action taken by the user? Type 'Hello'

3. What was the last action taken by the user? Italicize 'World'

4. What action is held at index (1)? Type 'World'

5. To undo "Bold 'Hello'", which actions must be undone? Write them in the order in which they are removed from the stack.

   The last action taken will be removed first: "Italicize 'World'".

   The next action to be removed is: "Type 'World'".

   Finally, after removing the previous two actions, you can undo "Bold 'Hello'".

6. How many are left in the stack? 1

7. What is the item left in the stack? Type 'Hello'

8. The user now performs the following actions in this order: italicizes 'Hello', types 'World', italicizes 'World', underlines 'Hello World'. Rewrite the items in the order in which they would appear in the stack. Indicate the index of each.

   Index (0) - Underlines 'Hello World'

   Index (1) - Italicizes 'World'

   Index (2) - Types 'World'

   Index (3) - Italicizes 'Hello'

   Index (4) - Type 'Hello'

Activity 2.9

Answer the questions below about the queue of customers waiting to speak with a customer service representative.

```
customer_queue = ["Maya", "Lee", "Chen", "Sara", "Omar"]
```

1. How many customers are in the queue? 5
2. Who was the first customer to enter the queue? Maya
3. Who was the last customer to enter the queue? Omar
4. Which customer will be the first to be served? Maya
5. From which index will customer names leave the queue? 0
6. Write the updated queue after the next customer is served.

   customer_queue = ["Lee", "Chen", "Sara", "Omar"]
7. Which customer will be served next? Lee
8. Write the updated queue after the Lee is served.

   customer_queue = ["Chen", "Sara", "Omar"]
9. Jenna joins the queue. Write the updated queue.

   customer_queue = ["Chen", "Sara", "Omar", "Jenna"]
10. At which index is "Jenna"? 3
11. Read the following instruction and determine how it would be handled.

    ```
    INSERT index (1) = "Mike"
    ```

    A queue operates based on the FIFO principle, so any new data must be added at the end. This instruction will result in an error and cannot be carried out.

Activity 2.10

Answer the questions about the following list of tuples.

```
student_grades = [
        (“Maya”, 85),
        (“Lee”, 92),
        (“Sara”, 78),
]
```

1. How many tuples are in the list? 3

2. What format does each tuple have? (name, grade)

3. Which tuple is at index (0)? (“Maya”, 85)

4. What is Sara’s grade? 78

5. Read the following instruction and update the list accordingly.

```
INSERT index (1) = (“Mike”, 83)
```

student_grades = [(“Maya”, 85), (“Mike”, 83), (“Lee”, 92), (“Sara”, 78)]

6. Assume there was an error entering Lee’s grade. To modify the data, you must:
   a) Swap the incorrect grade with the correct one inside the tuple at index (2).
   b) Remove the tuple at index (2) and insert a corrected tuple at index (2).
   c) Remove all tuples before index (2) before inserting the correct tuple.

   Lee’s correct grade is 91. Write the pseudocode instructions that will carry out this correction.

   REMOVE index (2) = (“Lee”, 92)

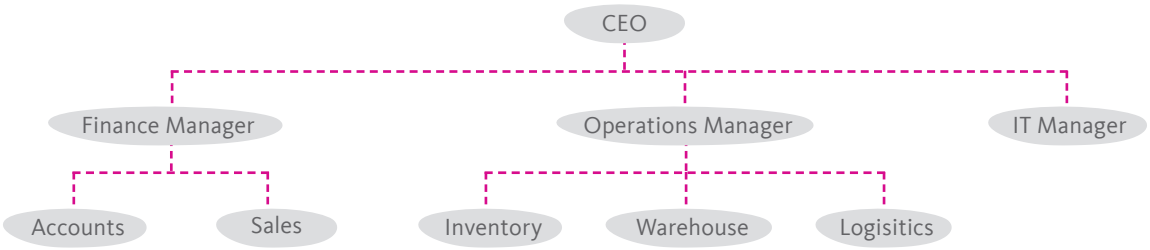
   INSERT index (2) = (“Lee”, 91)

   Write the corrected list.

   student_grades = [(“Maya”, 85), (“Mike”, 83), (“Lee”, 91), (“Sara”, 78)]

Activity 2.11

Answer the questions below about the organization chart of the company where Jenna works.



1. Which data structure is best suited for this chart? Tree
2. How many hierarchical levels are in this chart? 3
3. Which data item is at the root node? CEO
4. How many child nodes does the root node have? 3
5. How many child nodes does the 'Finance Manager' have? 2
6. How many leaf nodes are there in total? 6
7. What are the leaf nodes under 'Finance Manager'? Accounts, Sales
8. Is 'IT Manager' a leaf node or a parent node? Leaf node
9. Using pseudocode, write the tree data structure for the first two levels of the hierarchy. Name it 'org_chart'.

   org_chart = [(R, "CEO"), (R_0, "Finance Manager"), (R_1, "Operations Manager"),

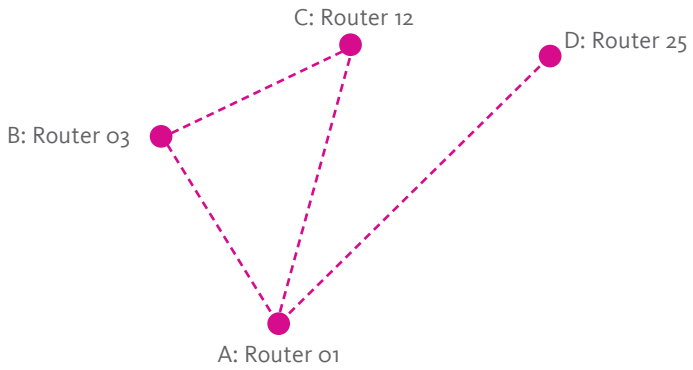
   (R_2, "IT Manager")]

10. Write a pseudocode instruction that will add 'Marketing' to follow 'Sales' under 'Finance Manager'.

    INSERT node (R_0_2, "Marketing")

Activity 2.12

Answer the questions about the following network of routers.



1. What type of data structure best represents this network? Graph
2. How many nodes are in this graph? 4
3. What is the name of the router designated as node B? Router 03
4. What is the node index of "Router 12"? C
5. Using node indices, write an adjacency list for this graph. A: [B, C, D]

   B: [A, C]

   C: [A, B]

   D: [A]

6. If the time it takes for a signal to travel between nodes (latency) is: A to B 12ms, A to C 18ms, A to D 25ms, and B to C 9ms, rewrite the adjacency list to include latency data as edges.

   A: [(B, 12ms), (C, 18ms), (D, 25ms)]

   B: [(A, 12ms), (C, 9ms)]

   C: [(A, 18ms), (B, 9ms)]

   D: [(A, 25ms)]

LESSON 3: SEARCHING LINEAR DATA

One of the fundamental operations on data is searching for a specific item within a dataset. Before we can modify, extract, relocate, or process any data item, we must first locate it accurately. Searching, like all computer operations, is defined and guided by algorithms, each with its own performance characteristics.

In this lesson, we will explore the mechanics and applications of basic search algorithms—linear search and binary search—and compare their performance. Through this comparison, we'll understand how choosing the right algorithm can make a significant difference in efficiency, particularly when dealing with large datasets organized in a linear format.

LINEAR SEARCH

One of the simplest strategies is known as linear search. It is a straightforward approach applied to linear data structures, where the algorithm starts at the first position in the dataset and moves sequentially from one position to the next, checking each data item for a match. If a data item matches the search item, the search ends. Otherwise, it continues until the end of the dataset.

EXAMPLE 3.1

Let's go back to the example of books on a shelf. To search for a particular title using a linear search approach, you would start at the beginning of the shelf and check each book title in order until you find the one you're looking for. However, if you reach the end of the shelf without finding the title, then the book is not on this shelf.

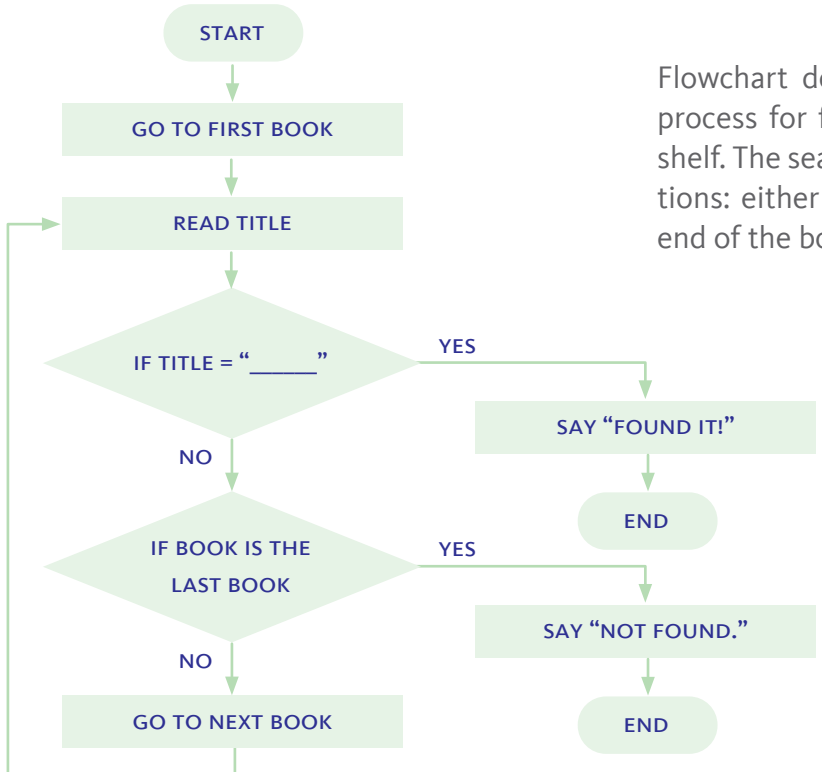
Using a linear search, books on a bookshelf are checked one by one from left to right across the shelf.

A LINEAR SEARCH ALGORITHM SEQUENTIALLY CHECKS EACH POSITION IN A DATASET.

The Linear Search Process

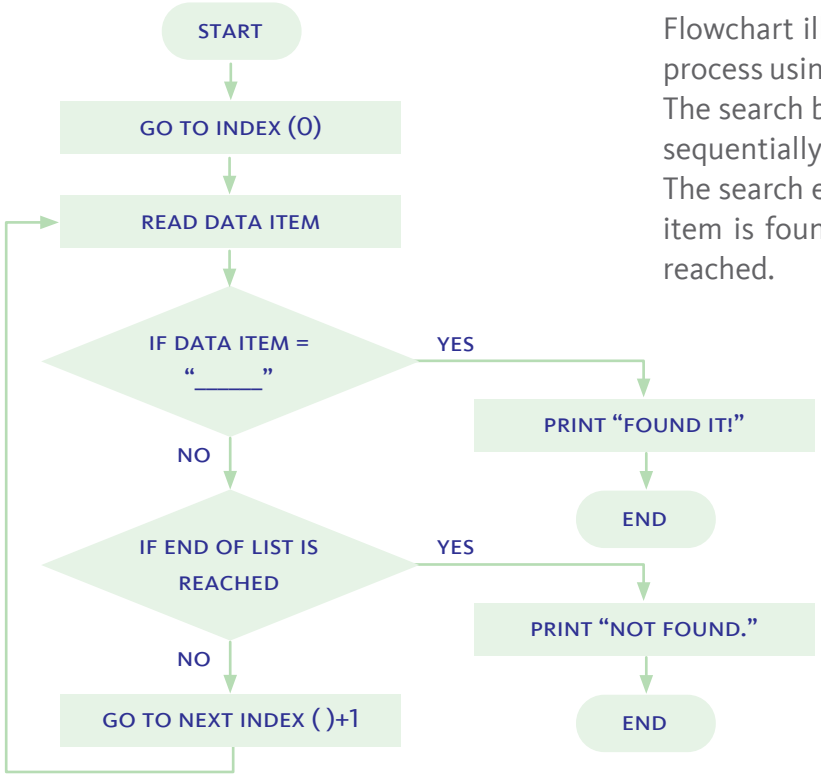
While the process of searching for an item, such as a book on a bookshelf or a toy in a toy basket, may come intuitively, breaking it down into discrete steps—including those required for decision-making—is essential. These steps include deciding to end the search once you've found what you're looking for, or concluding that the item is not on the bookshelf or in the toy basket.

When dealing with any step-by-step process, it is best understood when visually depicted in the form of a flowchart. A well-structured flowchart includes each individual step of the process, a clear flow from one step to the next, and decision points that indicate the path of flow based on each condition that is met.



Flowchart depicting the linear search process for finding a book on a bookshelf. The search ends under two conditions: either the book is found, or the end of the bookshelf is reached.

Now that the basic process of the linear search strategy has been established—even if only for books—we can extrapolate the structure of an algorithm that can be applied to any list of items. Simply described, the algorithm must start at the beginning of the list, sequentially check each item, and end the search if one of two conditions is satisfied: the item is found, or the end of the list is reached.



Flowchart illustrating the linear search process using an index-based approach. The search begins at the first index and sequentially checks each data item. The search ends when either the target item is found, or the end of the list is reached.

Achieving a clear understanding of the process is essential before creating a program that a computer can execute, following the steps of the linear search algorithm. Breaking down the linear search into its fundamental components not only clarifies each action but also prepares us to translate these actions into code.

Next, we will present the linear search algorithm in pseudocode form. This pseudocode provides a step-by-step outline of the process. Each instruction

represents a specific action that the computer will execute, transforming the logical steps of the search process into a precise, executable program.

```
START LinearSearch
OPEN list
SET search_index (0)

REPEAT_UNTIL search_index reaches end of list
    READ data_item AT: search_index

    IF data_item = target THEN
        PRINT "Found it!"
        END LinearSearch
    ELSE
        SET search_index (search_index + 1)

PRINT "Not found"
END LinearSearch
```

SET search_index (0): Initializes the starting position of the search to the first index (0), marking the beginning of the list.

REPEAT_UNTIL: Creates a loop that continues until the end of the list is reached. The condition for ending the search upon reaching the last position in the list is embedded within this instruction.

Within the loop, the algorithm **READ**s each data item and ends the search **IF** the data item matches the target being searched for.

SET search_index (search_index + 1): Ensures that the search progresses by moving to the next position in the list, incrementing search_index by 1 with each iteration.

The strategy of searching sequentially through a list is straightforward and would work for any list. However, as the size of the list grows, challenges arise. When there are only 10 books on a shelf, checking each one is manageable, but what if the search involves an entire library with hundreds or even thousands of books?

PERFORMANCE

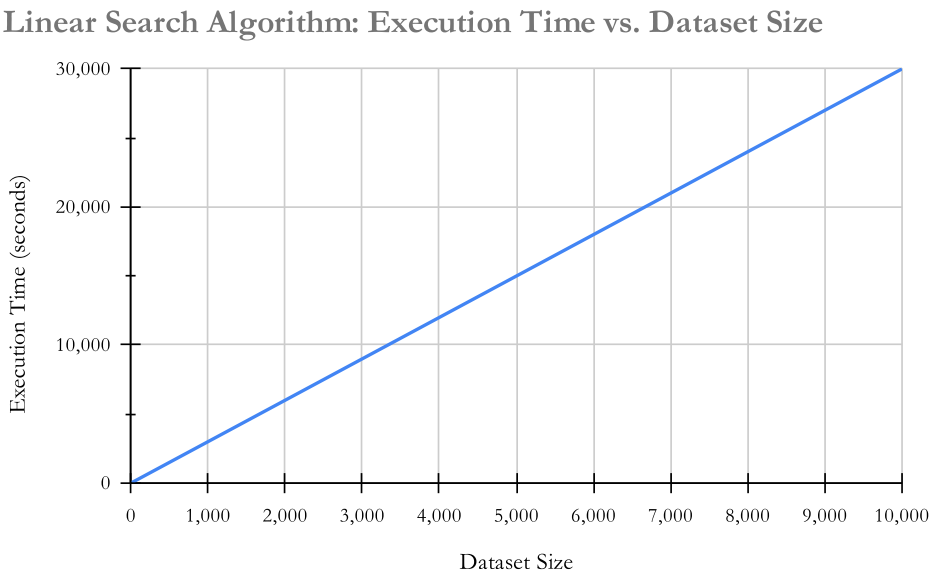
Let's explore the performance of a linear search algorithm. One simple measure of performance is execution time, which refers to the amount of time it takes for the program to complete its run.

EXAMPLE 3.2

Referring back to the books on the bookshelf, let's assume it takes approximately 3 seconds for a person to check each book, decide whether they've found what they're looking for, or move on to the next book on the shelf. We'll also assume that this person can perform this task tirelessly, without breaks or errors. Since it takes 3 seconds to check each book, the time to search *n* books would be:

Execution Time = Time per Attempt x Number of Attempts

Size of Dataset	Execution Time
10 books	*3 x 10 = 30 seconds*
100 books	*3 x 100 = 300 seconds (5 minutes)*
1,000 books	*3 x 1,000 = 3,000 seconds (50 minutes)*
10,000 books	*3 x 10,000 = 30,000 seconds (8 hours 20 minutes)*



The graph illustrates how execution time increases in direct proportion to the size of the dataset when using a linear search algorithm. Each additional book requires a constant amount of time (3 seconds) to check, resulting in a straight-line growth pattern.

WORST-CASE

To best understand the performance of an algorithm, we need to evaluate it in its worst-case scenario. For a search algorithm, this means assuming it has to examine every item in the list before finding the target (or concluding that the target isn't present). This contrasts with a best-case scenario, where the algorithm finds its target on the very first attempt. For a bookshelf, the best case would mean that the target book is the very first one checked.

A WORST-CASE SCENARIO IS THE SITUATION IN WHICH AN ALGORITHM TAKES THE LONGEST POSSIBLE TIME TO ACHIEVE ITS GOAL.

In the previous example, the execution time calculations are based on this worst-case scenario for our linear search, where the algorithm wasn't fortunate enough to find the book among the first few items and had to search all the way to the end of the shelf. This worst-case analysis helps us understand the maximum time our linear search algorithm might require under the least favorable conditions.

Notes

The concepts of algorithm performance in this section are approached with a focus solely on time complexity. This helps students concentrate on understanding how execution time scales with input size before introducing additional factors like space complexity.

Example 3.2 demonstrates that as the dataset size increases, the time required for the search also increases at a constant rate of 3 seconds per item. For instance, searching through 10 books takes about 30 seconds, while searching through 10,000 books takes 30,000 seconds. This reveals a key limitation of the linear search algorithm: for large datasets, the search time becomes impractically long.

EXAMPLE 3.3

Global telephone operators may manage databases with hundreds of millions of telephone numbers. Let's assume one such operator manages 100 million numbers. Further, let's assume its computer systems have a 50-nanosecond latency, meaning it takes the processor about 50 nanoseconds to access memory. A call comes in from a client, and the customer service representative wants to search for this client's account. The execution time for a linear search would be:

$$\textit{Execution Time} = 50 \times 100{,}000{,}000 = 5{,}000{,}000{,}000 \text{ ns (5 seconds)}$$

A 5-second execution time for a single search in a call center could have significant repercussions for the operator's business. In a call center, where efficiency is crucial, this delay would slow down interactions, leading to longer wait times and potentially frustrating customers.

But the impact extends beyond customer service. Telecom operators rely on real-time systems to route calls, authenticate users, and manage billing. A 5-second delay in verifying or routing each call could disrupt these processes, resulting in dropped calls, misrouted calls, or costly billing errors. Moreover, as the operator's client base grows, so will the delays and related issues, emphasizing the need for more efficient search solutions to maintain smooth, reliable service.

BINARY SEARCH

What if we could use a strategy that reduces the size of the list as the search progresses? The binary search algorithm does exactly that. Instead of starting at the beginning of the list, binary search begins at the middle index. If the target is less than the value of the data item at the middle index, the algorithm knows the target must be in the first half of the list, so it disregards the second half. It then focuses on the middle index of the remaining portion of the list, checking the value there. Based on this comparison, it determines which half of the list to continue searching and which to disregard. This process repeats until the target is found or the list is fully divided, essentially halving the dataset with every iteration.

An important condition for the binary search algorithm to work is that the list must be sorted; otherwise, this strategy would not be effective. While a linear search algorithm works equally well on both sorted and unsorted lists, the binary search algorithm requires a sorted list to function properly. Without sorting, binary search would not be able to accurately determine which half of the list to focus on, making the algorithm ineffective.

A BINARY SEARCH ALGORITHM FINDS A TARGET VALUE IN A SORTED LIST BY REPEATEDLY DIVIDING THE SEARCH RANGE IN HALF.

EXAMPLE 3.4

Let's go back to our books on the bookshelf. There are 12 books on the shelf, and this time, we'll assume the books are sorted in alphabetical order. In a linear search, the worst-case scenario would require the algorithm to check all 12 books. Now, let's see what happens in the worst-case scenario for a binary search algorithm.

The 12 books on the shelf are sorted in alphabetical order.

In order to make the example more tangible, here is a list of all twelve book titles arranged in alphabetical order. In the worst-case scenario, the target book would be at the final possible location after all divisions, requiring multiple attempts to reach it. In this case, our target title is 'Zebra Crossing'. Here's how the binary search would proceed:

```
book_titles = [
    "A Tale",
    "Blue Moon",
    "Crimson Sky",
    "Deep Waters",
    "Endless Night",
    "Frost Fire",
    "Golden Path",
    "Hidden World",
    "Iron Fist",
    "Jungle King",
    "Silver Light",
    "Zebra Crossing"
    ]
```

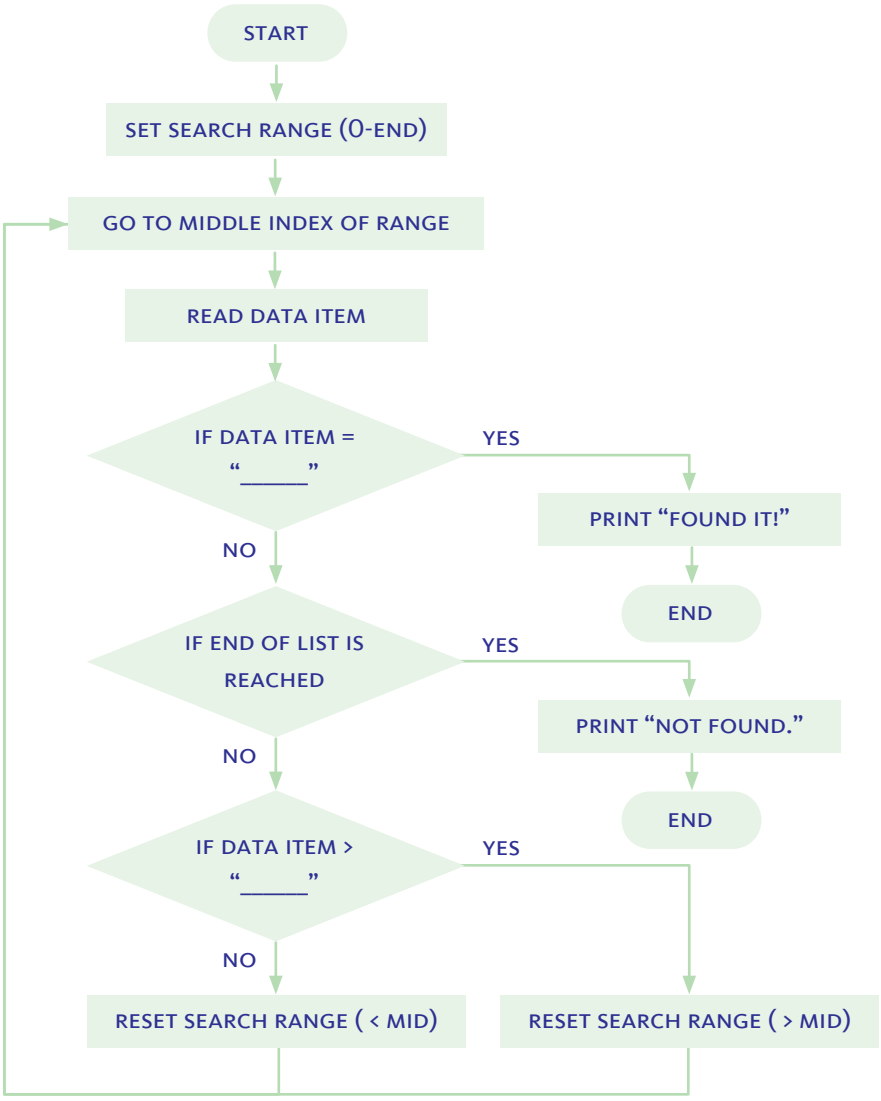
The list of 12 book titles in alphabetical order, which serves as the dataset for the binary search simulation.

The simulation illustrates the steps of a binary search to locate the book title 'Zebra Crossing' within the sorted list. Each attempt shows the search range, middle index, and the decision to disregard half of the list based on comparisons, continuing until the target is found at index 11.

Steps	Range (index)	Middle (index)	Comments
Attempt 1	0 - 11	5	Z > F, disregard 0 - 5
Attempt 2	6 - 11	8	Z > I, disregard 6 - 8
Attempt 3	8 - 11	10	Z > S, disregard 8 - 10
Attempt 4	11	11	Found it!

Had this search been conducted using a linear search algorithm, it would have taken 12 attempts. In comparison, the binary search algorithm's worst-case scenario required only 4 attempts, demonstrating the performance advantage of binary search over linear search when working with sorted lists.

Now that the basic concept of a binary search algorithm has been introduced, we can represent it in a flowchart that illustrates its essential steps: 1) setting the search range, 2) identifying the middle index of the search range, 3) evaluating the data item at the middle index to determine if the target has been found, and 4) if the target is not found, defining the next range of indices to continue the search.



This flowchart illustrates the steps of a binary search algorithm. Starting with an initial search range, the algorithm identifies the middle index, evaluates the data item at that index, and determines if it matches the target. If the target is not found, the algorithm narrows the search range by selecting the appropriate half and repeats the process until the target is located or the search range is exhausted.

Before writing the algorithm in pseudocode, we should explore its fundamental aspects. The first is setting the search range and identifying the search index, and the second is understanding the main loop of the algorithm and the conditions that repeat within it.

The Search Range and Index

The search range in a binary search algorithm is always confined between its lower end, the **start_index**, and its upper end, the **end_index**. When the algorithm begins, these indices are set to the first and last positions of the list: **start_index (0)** and **end_index (last index of the dataset)**.

Once the search range has been set, the next step is to calculate the midpoint of the range to identify the index at which to search, the **search_index**. This midpoint is determined by halving the sum of the **start_index** and **end_index** of the search range.

search_index = midpoint = (start_index + end_index) / 2

With each iteration of the loop, the search range is redefined by adjusting either the upper end, **end_index**, or the lower end, **start_index**, depending on which half of the current range the target is determined to be in.

- If the target is in the lower half of the current range, **end_index** is adjusted to one position before the middle, calculated as **search_index - 1**.
- If the target is in the upper half of the current range, **start_index** is adjusted to one position after the middle, calculated as **search_index + 1**.

This approach continually narrows down the search range, recalculating the midpoint with each iteration that resets the range.

Notes

The midpoint must be an integer. If the calculation results in a fractional number, round down to the nearest integer.

THE LOOP AND CONDITIONS

The loop in the binary search algorithm ensures that a specific set of steps is repeated for each search index and that certain conditions are evaluated:

- Has the target been found at the search index: Is **data_item = target**?
- Is the target in the lower half of the search range: Is **data_item > target**?
- Is the target in the upper half of the search range: Is **data_item < target**? In this algorithm, if **data_item** is neither equal to nor greater than the target, then it must be less than the target. This condition is therefore implied rather than explicitly stated.
- Has the whole list been searched? This is embedded within the **REPEAT_UNTIL** loop, which ensures that the loop continues until a specific condition is met—in this case, when the **search_index** has reached the end of the list.

```
START BinarySearch
OPEN list
SET start_index (0)
SET end_index (last index of list)

REPEAT_UNTIL search_index reaches end of list
    SET midpoint = (start_index + end_index)/2
    SET search_index (midpoint)
    READ data_item AT: search_index

    IF data_item = target THEN
        PRINT "Found it!"
        END BinarySearch
    ELSE IF data_item > target THEN
        SET end_index (search_index - 1)
    ELSE
        SET start_index (search_index + 1)

PRINT "Not found"
END BinarySearch
```

Pseudocode for a binary search algorithm that repeatedly narrows down the search range by adjusting the start and end indices until the target is found or the list is fully searched.

PERFORMANCE

Previously, in Example 3.4, we compared the performance of linear and binary search algorithms by counting the attempts required to find a target. In this worst-case scenario of searching through 12 book titles, the binary search algorithm located its target in significantly fewer attempts than the linear search—4 attempts versus 12.

We can also compare the two algorithms based on their execution time. For a linear search algorithm, the execution time is the product of the time required for a single attempt and the total number of attempts needed to identify the target. In contrast, while the execution time for a binary search algorithm also depends on the time required for each attempt, the number of possible attempts is halved with each step. To represent this reduction mathematically, we use the log-base 2 operation, which captures the exponential decrease in search range for each iteration.

If t is the time it takes to execute one attempt, and n is the number of items in the dataset, then the execution time for a binary search is:

$$Execution\ Time = t \times \log_2(n)$$

EXAMPLE 3.5

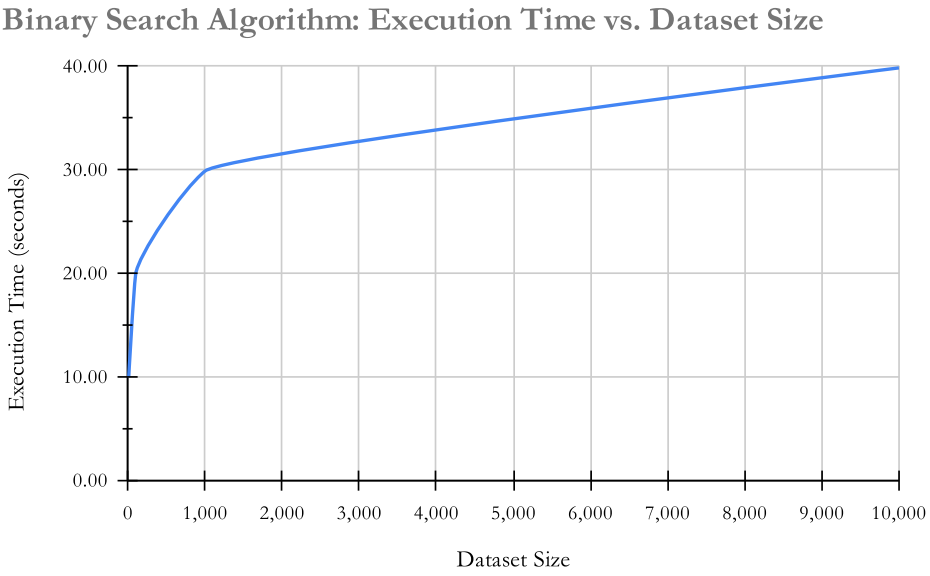
Referring back to the books on the shelf in Example 3.2, we will now use a binary search algorithm instead of a linear search. We'll compare the execution times for datasets of the same sizes to highlight the efficiency of binary search. The assumption that it takes 3 seconds to check each book will still hold.

Notes

Students are not expected to understand the logarithmic function in depth at this stage. However, they should grasp that the log-base 2 function is used to mathematically represent the exponential decrease in the search range with each iteration of the binary search algorithm.

Size of Dataset	Execution Time (Binary Search)	Execution Time (Linear Search)
10 books	*3 x $\log_2(10)$ = 9.97 seconds*	*30 seconds*
100 books	*3 x $\log_2(100)$ = 19.93 seconds*	*300 seconds (5 minutes)*
1,000 books	*3 x $\log_2(1{,}000)$ = 29.90 seconds*	*3,000 seconds (50 minutes)*
10,000 books	*3 x $\log_2(10{,}000)$ = 39.86 seconds*	*30,000 seconds (8 hours 20 minutes)*

The table highlights a drastic difference in performance between linear search and binary search algorithms. For a dataset of 10,000 books, the execution time for a linear search grows linearly, reaching 8 hours and 20 minutes, while the execution time for a binary search grows logarithmically, reaching only 39.86 seconds. This substantial difference arises because binary search halves the search range with each attempt, making it far more efficient for larger datasets.



Execution time of a binary search algorithm across different dataset sizes, illustrating logarithmic growth and the algorithm's efficiency as dataset size increases.

EXAMPLE 3.6

If the global telephone operator in Example 3.3 used a binary search algorithm instead of a linear search to locate a telephone number in a database of 100 million numbers, with a computer system latency of 50 nanoseconds per attempt, the execution time for the binary search would be:

$$Execution\ Time = 50 \times \log_2(100{,}000{,}000) = 1{,}330\ nanoseconds$$

Compared to the 5 seconds required for a linear search to accomplish this task, the 1,330 nanoseconds (1.33 microseconds) for a binary search is a significant improvement in performance.

CONCLUSION

In this lesson, we explored two fundamental search algorithms—linear search and binary search—and compared their performance across varying dataset sizes. While linear search is simple and works on unsorted data, its execution time grows linearly, making it inefficient for large datasets. In contrast, binary search, which requires sorted data, drastically reduces the search range with each attempt, resulting in logarithmic growth in execution time. Understanding these differences highlights the importance of choosing the right algorithm based on dataset size and performance requirements, a key concept in computer science.

SUMMARY

Searching is a fundamental operation needed to locate specific items in a dataset before performing further actions like modification, extraction, or processing.

Linear Search:

- Works by sequentially checking each item in the dataset from start to finish.
- Can be used on both sorted and unsorted datasets.
- Execution time grows linearly with the size of the dataset, making it inefficient for large datasets.

Binary Search:

- Requires the dataset to be sorted.
- Starts by checking the middle item of the search range and eliminates half of the range with each attempt.
- Execution time grows logarithmically with the size of the dataset, making it much more efficient than linear search for large datasets.

Comparison of Performance:

- Linear search checks every item, which results in a high execution time as the dataset grows.
- Binary search dramatically reduces the number of attempts by halving the search range, resulting in faster execution times for large, sorted datasets.
- Understanding the performance differences helps in choosing the right algorithm based on dataset size and structure.

Activity 3.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
T	A linear search algorithm can be used on both sorted and unsorted datasets.
F	A binary search algorithm can be used on both sorted and unsorted datasets.
F	A binary search algorithm begins by checking the first item in the dataset.
F	In a linear search algorithm, the search range is halved with each iteration.
T	Execution time for a linear search increases directly with the size of the dataset.
T	Execution time for a binary search increases logarithmically with the size of the dataset.
F	In the worst-case scenario, binary search requires more attempts than linear search.
T	In the best-case scenario, binary search and linear search are equally efficient.
T	In the worst-case scenario, binary search is more efficient than linear search, regardless of dataset size.
F	The choice of search algorithm does not impact the overall performance of a program.

Activity 3.2

Answer the questions below.

1. What is the primary characteristic of a linear search algorithm?
   a) It requires a sorted dataset.
   b) It searches each item sequentially.
   c) It divides the dataset in half each time.
   d) It starts at the middle of the dataset.

2. Which of the following is a limitation of linear search for large datasets?
   a) It requires a sorted list.
   b) It can be very slow because it checks each item sequentially.
   c) It cannot be used with unique values.
   d) It does not work on datasets with numbers.

3. What is the best-case scenario for a linear search algorithm?
   a) The target is in the middle of the dataset.
   b) The target is at the end of the dataset.
   c) The dataset is already sorted.
   d) The target is the first item in the dataset.

4. Which type of data structure can linear search be applied to?
   a) Only sorted arrays
   b) Only linked lists
   c) Any list, sorted or unsorted
   d) Only binary trees

5. What happens in a linear search if the target item is not in the dataset?
   a) The search stops at the midpoint of the dataset.
   b) The search checks every item and then stops.
   c) The search requires the dataset to be reordered.
   d) The search skips some items to save time.

Activity 3.3

Answer the questions below.

1. What is the main requirement for using a binary search algorithm on a dataset?
   a) The dataset must contain unique values.
   b) The dataset must be sorted.
   c) The dataset must be large.
   d) The dataset must be stored in a linked list.

2. What does binary search do with each iteration?
   a) Moves one step closer to the target from the beginning of the list.
   b) Divides the search range in half.
   c) Checks each item sequentially.
   d) Moves in reverse through the list.

3. In a binary search, if the target is less than the middle item, what should the algorithm do next?
   a) Focus on the lower half of the search range.
   b) Focus on the upper half of the search range.
   c) Ignore the entire list.
   d) Conclude that the target is not present.

4. What is the best-case scenario for a binary search algorithm?
   a) The dataset is very large.
   b) The dataset is unsorted.
   c) The target is the first item in the dataset.
   d) The target is the middle item on the first check.

5. What is the growth rate of a binary search algorithm?
   a) Linear
   b) Exponential
   c) Logarithmic
   d) Quadratic

Activity 3.4

Answer the questions below.

1. Which search algorithm is generally faster for large, sorted datasets?
   a) Binary search
   b) Linear search
   c) Both are equally fast
   d) It depends on the dataset size

2. What is a key difference in requirements between linear and binary search?
   a) Linear search requires sorted data, while binary search does not.
   b) Binary search requires sorted data, while linear search does not.
   c) Both require sorted data.
   d) Neither requires sorted data.

3. When would a linear search be preferred over a binary search?
   a) When there are fewer than 10 items.
   b) When the dataset is sorted and very large.
   c) When binary search fails to find the target.
   d) When the dataset is unsorted.

4. Why is binary search more efficient than linear search on large, sorted datasets?
   a) It skips every other item in the list.
   b) It checks each item twice.
   c) It divides the search range in half with each attempt.
   d) It starts at the end of the list.

5. If the target is the first item in a dataset, how does the performance of binary search compare to linear search?
   a) Binary search will be slower.
   b) Linear search will be slower.
   c) Both will perform the same.
   d) The result depends on dataset size.

Activity 3.5

Refer to the sorted list. Compare the number of attempts required for linear search and binary search to find specific numbers in the list.

List_A = [3, 7, 12, 19, 25, 31, 44, 52, 67, 72, 88, 93, 105, 117, 126, 139, 145, 159, 167, 178, 190]

1. Find the number 3.
   - Linear Search Attempts: 1
   - Binary Search Attempts: 5

2. Find the number 88.
   - Linear Search Attempts: 11
   - Binary Search Attempts: 1

3. Find the number 190.
   - Linear Search Attempts: 21
   - Binary Search Attempts: 5

4. Find the number 105.
   - Linear Search Attempts: 13
   - Binary Search Attempts: 3

5. What is the number of worst-case scenario attempts?
   - Linear Search Attempts: 21
   - Binary Search Attempts: 5

6. Which index in the list provides a best-case scenario for:
   - Linear Search: index (0) first
   - Binary Search: index (10) middle

Activity 3.6

For each of the three scenarios provided below, determine the number of attempts that a linear and a binary search algorithm would require to find the target item in the worst-case scenario. Then, calculate the execution time for each algorithm's search.

1. A dataset contains 1,024 items, and each attempt takes 2 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search on this dataset.

   Linear Search: Attempts: n = 1,024 (all items are checked)

   Execution time = t x n = 2 x 1,024 = 2,048 ms (about 2 seconds)

   Binary Search: Attempts: log2(n) = log2(1,024) = 10

   Execution time = t x log2(n) = 2 x 10 = 20 ms

2. A dataset of 20,000 items is searched, with each attempt taking 5 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search.

   Linear Search: Attempts: n = 20,000 (all items are checked)

   Execution time = t x n = 5 x 20,000 = 100,000 ms (100 seconds)

   Binary Search: Attempts: log2(n) = log2(20,000) = 14.29 = 15 (round up)

   Execution time = t x log2(n) = 5 x 15 = 75 ms

3. A large dataset of 1,000,000 items is searched, and each attempt takes 10 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search.

   Linear Search: Attempts: n = 1,000,000 (all items are checked)

   Execution time = t x n = 10 x 1,000,000 = 10,000,000 ms (10,000 seconds)

   Binary Search: Attempts: log2(n) = log2(1,000,000) = 19.93 = 20 (round up)

   Execution time = t x log2(n) = 10 x 20 = 200 ms

Activity 3.7

Use the linear search algorithm provided in pseudocode to answer the following questions.

```
START LinearSearch
OPEN list
SET search_index (0)

REPEAT_UNTIL search_index reaches end of list
    READ data_item AT: search_index

    IF data_item = target THEN
        PRINT "Found it!"
        END LinearSearch
    ELSE
        SET search_index (search_index + 1)

PRINT "Not found"
END LinearSearch
```

1. What is the purpose of the instruction: SET search_index (0) ?

Sets the starting position of the search at the first item in the list.

2. Which instruction increments the search index by 1 position?

SET search_index (search_index + 1)

3. What is the purpose of the instruction: REPEAT_UNTIL ?

It creates a loop to repeatedly check items in the list.

4. Which two conditions end the REPEAT_UNTIL loop?

search_index reaches end of list

data_item = target

Activity 3.8

Use the binary search algorithm provided in pseudocode to answer the following questions.

```
START BinarySearch
OPEN list
SET start_index (0)
SET end_index (last index of list)

REPEAT_UNTIL search_index reaches end of list
    SET midpoint = (start_index + end_index)/2
    SET search_index (midpoint)
    READ data_item AT: search_index

    IF data_item = target THEN
        PRINT "Found it!"
        END BinarySearch
    ELSE IF data_item > target THEN
        SET end_index (search_index - 1)
    ELSE
        SET start_index (search_index + 1)

PRINT "Not found"
END BinarySearch
```

1. Which two parameters define the search range?

The two parameters are the start_index and the end_index.

2. What is the purpose of the midpoint variable?

The midpoint variable calculates the middle index of the current search range.

3. What is the purpose of the instruction: SET end_index (search_index - 1) ?

It focuses the search on the lower half of the current search range by resetting the current end index to one position below the current midpoint.

4. Why is the instruction IF data_item < target THEN not included?

If data_item is neither equal to nor greater than the target, then it must be less than the target by exclusion.

Activity 3.9

Having gained an in-depth understanding of the mechanics, performance, and logic behind linear and binary search algorithms, it's time to explore them further. This activity allows students to examine linear and binary search algorithms in Python programs. Follow the links to download the three programs provided for this activity.

1. Linear Search Program: This program performs a linear search on a user-defined, sorted range. It checks each item in the list sequentially until it finds the target or reaches the end. The program outputs the number of attempts taken, demonstrating the step-by-step process of linear search.

2. Binary Search Program: This program performs a binary search on a user-defined, sorted range. It begins by checking the midpoint of the range and repeatedly halves the search space based on comparisons. The program reports the number of attempts, showing the efficiency of binary search on sorted data.

3. Combined Search Program: This program includes both linear and binary search algorithms, operating on the same user-defined, sorted range. It allows users to compare the performance of linear and binary search directly by running both methods on the same target and range. This comparison highlights the efficiency differences between the two search methods.

Notes

Notice the similarity between the pseudocode and the Python programs provided for linear and binary search. This similarity highlights how pseudocode serves as an essential bridge between algorithmic logic and actual coding. Pseudocode provides students with a clear, language-agnostic structure for algorithms, helping them transition smoothly from planning to implementation in Python or other programming languages. Encourage students to use pseudocode as a guide to understand, develop, and refine their programming skills.

LESSON 4: SORTING LINEAR DATA

In earlier lessons, we worked with data in many forms: we tracked temperature changes over time, compared students' exam scores with their study hours, and even helped organize a warehouse of electronics and groceries. We learned how to label, clean, and organize data, and how to search through it efficiently using linear and binary search.

But what if you wanted to find the highest temperature, rank students by their exam scores, or organize your inventory from oldest to newest? Before you can do any of that, you need to sort your data.

SORTING IS THE ARRANGEMENT OF DATA IN A SPECIFIC ORDER BASED ON A CERTAIN CRITERION.

Sorting means arranging data in a specific order based on a certain criterion. For example:

- Books can be sorted by their titles.
- Apple crates in a warehouse can be sorted by their arrival date.
- Students can be sorted by their exam scores.

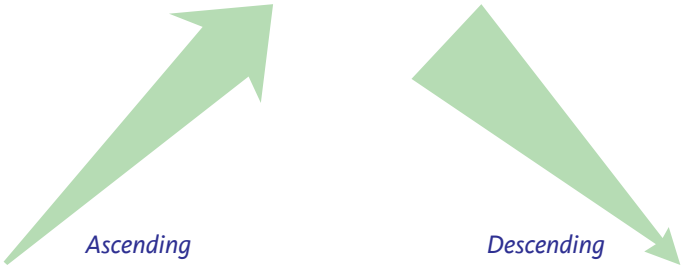
Sorting is a powerful approach to organizing data within any data structure. It typically makes the data more accessible and allows for easier extraction of information.

In this lesson, we will learn what sorting is, why it is useful, and explore two simple but powerful sorting strategies—Insertion Sort and Bubble Sort—and how they work.

ORDER & CRITERIA

Sorting is defined as the arrangement of data in a specific order based on a certain criterion. This definition combines two key ideas: “a specific order” and “a certain criterion.”

Order refers to the direction in which the data is arranged. It answers the question: Which direction will the sorted data follow? The two most common orders are ascending, which means increasing from least to greatest, and descending, which means decreasing from greatest to least.



The criterion, on the other hand, refers to the basis used to determine how the data will be sorted. It answers the question: What are we sorting by? For example, are we sorting by exam scores, temperatures, prices, student names, book titles, or time?

In general, sorting criteria fall into two main categories: numeric data, which consists of numbers, and alphanumeric data, which typically includes letters, words, and characters.

Sorting numeric data is based on numerical value and may be arranged in ascending (e.g. 1, 2, 3...) or descending (e.g. 10, 9, 8...) order. Examples include exam scores, temperatures, prices, and time.

Alphanumeric data is sorted based on character sequences and may also be ordered in ascending (e.g. A, B, C...) or descending (e.g. Z, Y, X...) order. Examples of alphanumeric data include student names, book titles, file names, or ID codes.

Whether a dataset is sorted in a specific order or by a particular criterion depends on the nature of the data and the purpose of sorting.

EXAMPLE 4.1

The school's database contains data on all enrolled students. The table below shows a sample segment of that database with 10 entries. Even though these are just a few data items, it is still difficult and tedious for a person to extract information from it manually.

Name	Family	Absences	Average	Transport
Leila	Carter	1	85	Bus
Tarek	Najem	0	91	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Ethan	Müller	0	94	Parents
Dana	Lankart	2	83	Parents
Isabel	Garcia	1	74	Parents
Roland	Bauer	13	45	Walk
Felix	Jensen	3	77	Bus
Ben	Zimmer	0	89	Bus

Let's say we wanted to find a particular family name, "Carter." We would need to check each family name starting from the top — a linear search approach. As we've previously discussed, linear search is not the most efficient algorithm but is necessary when the data is unsorted. However, we can sort this table by family name in ascending alphabetical order.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Leila	Carter	1	85	Bus
Ahmed	Dandash	8	64	Bus
Isabel	Garcia	1	74	Parents
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Ethan	Müller	0	94	Parents
Tarek	Najem	0	91	Walk
Sofia	Petrovic	10	51	Parents
Ben	Zimmer	0	89	Bus

With a sorted list, finding a name becomes easier. It also allows us to apply more efficient searching algorithms, such as the binary search algorithm.

The same dataset can be sorted using different orders and rules. The choice of order and rule depends on the nature of the data and the purpose of sorting.

EXAMPLE 4.2

Let's say, for example, that this time we want to extract information about absences and attendance. If our purpose is to identify the students who were absent the greatest number of times, and the data under "Absences" contains numeric values, then we would sort the dataset by Absences in descending order of numeric value. With the data now sorted, identifying the students with the highest number of absences becomes much easier.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Leila	Carter	1	85	Bus
Isabel	Garcia	1	74	Parents
Tarek	Najem	0	91	Walk
Ethan	Müller	0	94	Parents
Ben	Zimmer	0	89	Bus

Not only does sorting make finding information easier—either by making it visually evident or by enabling the use of efficient search algorithms—but it also helps us spot trends and patterns in data, particularly in smaller datasets like this one.

One of the key benefits of sorting is that it makes patterns more visible. When data is arranged in a meaningful order—such as alphabetically or numerically—it becomes easier to scan, compare, and interpret. For example, sorting students by the number of absences in descending order quickly highlights those with the highest absenteeism, drawing attention to potential attendance issues.

Similarly, sorting exam scores from highest to lowest reveals the top performers in a class. But it can also help uncover broader trends.

EXAMPLE 4.3

If we examine the table—specifically the fields Absences and Average—we notice a pattern: students with the most absences tend to have the lowest averages, while those with the fewest absences tend to have the highest.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Leila	Carter	1	85	Bus
Isabel	Garcia	1	74	Parents
Tarek	Najem	0	91	Walk
Ethan	Müller	0	94	Parents
Ben	Zimmer	0	89	Bus

It is important to note that, although this observation does not confirm a relationship between attendance and academic performance, it raises a question that could be explored through further analysis.

As we've seen, sorting is a valuable strategy for organizing data in a way that makes it easier to search, interpret, and extract information. Whether we're searching for a specific name or analyzing attendance trends, sorting helps reveal patterns and supports more efficient decision-making. While sorting alone does not confirm relationships within data, it can highlight areas worth investigating.

The same dataset can be sorted in different ways depending on what we're looking for and why. Sorting by alphabetical order helps us locate items more quickly, while sorting numerically allows us to rank or compare values. As we continue exploring sorting strategies, we'll begin to see how even simple algorithms like Insertion Sort and Bubble Sort can support meaningful data analysis when applied thoughtfully.

SORTING ALGORITHMS

Sorting algorithms determine the step-by-step process of arranging data in a specific order. All sorting algorithms must be able to compare data items and change their positions so that the data becomes organized. While the goal is the same, different algorithms follow different strategies—each varying in which data items are compared, and how and when those items are moved to new positions.

In this section, we will explore two simple but effective sorting algorithms—Insertion Sort and Bubble Sort.

THE INSERTION SORT ALGORITHM

The insertion sort algorithm follows a strategy of compare, shift, and insert. Starting at the beginning of the list, it temporarily removes one data item at a time and compares it with the items that precede it. Once the correct position is found, any intervening out-of-order data items are shifted up, and the removed data item is inserted into the empty position closer to the beginning.

THE INSERTION SORT ALGORITHM FOLLOWS A SEQUENTIAL STRATEGY OF COMPARE, SHIFT, AND INSERT.

In a sense, out-of-order items are shifted toward the end, and data items are inserted one by one into their correct positions.

If the goal is to sort in ascending order, data items with lesser values are inserted in increasing order near the beginning of the list, while greater out-of-order values are shifted upward toward the end. If the goal is to sort in descending order, the opposite occurs: greater values are inserted in decreasing order near the beginning, while lesser out-of-order values are shifted upward.

EXAMPLE 4.4

In this example, we will sort a list of five numbers in ascending order. Starting at index 1, each data item is compared to the values before it. Greater out-of-order values are shifted upward, and the lesser value is inserted into its correct position near the beginning of the list. These steps are repeated in iterative loops until the last item in the list has been selected, compared, and inserted into its correct position.

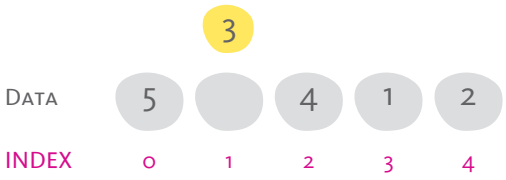
"5, 3, 4, 1, 2"

To better visualize the list, each data item is shown in its respective position, labeled by its index.

The First Iteration

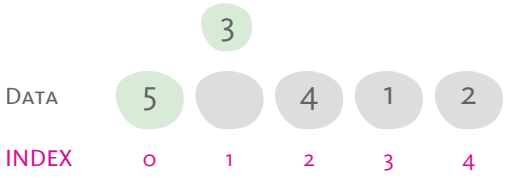
Step 1 - Hold

Data item at Index 1 is temporarily held.



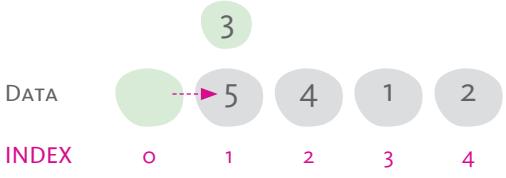
Step 2 - Compare

Yes, "5" is greater than "3".



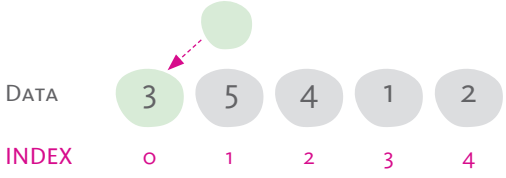
Step 3 - Shift

"5" is shifted to Index 1 instead of "3".



Step 4 - Insert

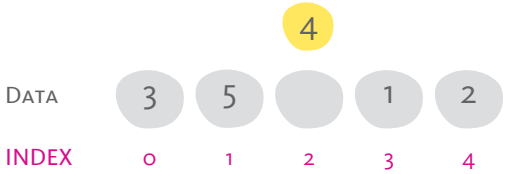
"3" is inserted at Index 0 instead of "5".



The Second Iteration

Step 1 - Hold

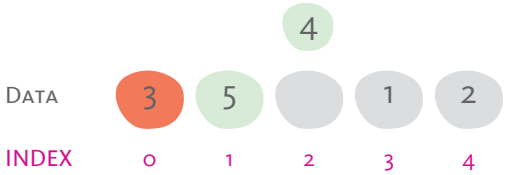
Data item at Index 2 is temporarily held.



Step 2 - Compare

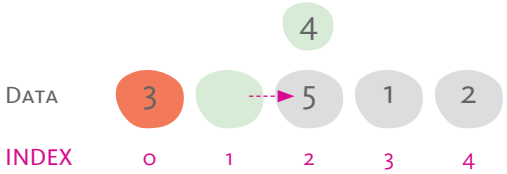
Yes, "5" is greater than "4".

No, "3" is not greater than "4".



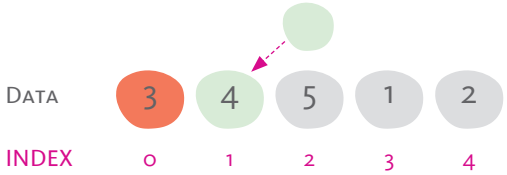
Step 3 - Shift

Only "5" is shifted to Index 2 instead of "4".



Step 4 - Insert

"4" is inserted at Index 1 instead of "5".



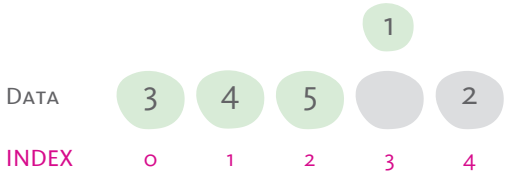
The Third Iteration

Step 1 - Hold

Data item at Index 3 is temporarily held.

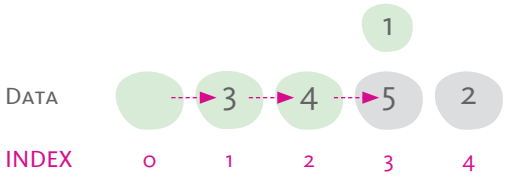
Step 2 - Compare

Yes, "5", "4", and "3" are greater than "1".



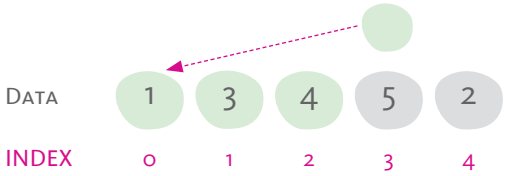
Step 3 - Shift

"5", "4", and "3" are shifted one index up.



Step 4 - Insert

"1" is inserted at Index 0 instead of "3".



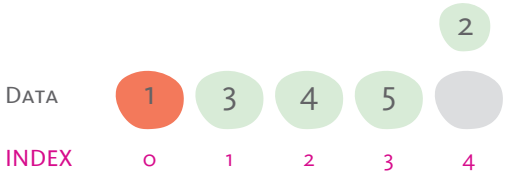
The Fourth Iteration

Step 1 - Hold

Data item at Index 4 is temporarily held.

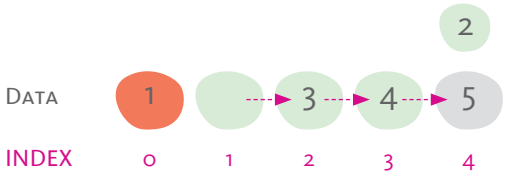
Step 2 - Compare

Yes, "5", "4", and "3" are greater than "2".
No, "1" is not greater than "2".



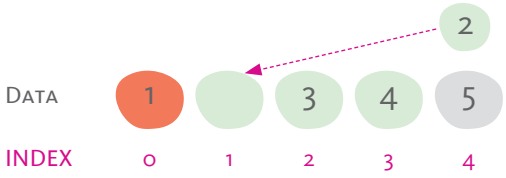
Step 3 - Shift

"5", "4", and "3" are shifted one index up.



Step 4 - Insert

"2" is inserted at Index 1 instead of "3".



Sorting Complete

The last index is reached.

All data is sorted in ascending order.

THE INSERTION SORT PROCESS

Now that we have demonstrated the mechanics of the algorithm, we need to establish the flow of steps that define the operations and conditions that make the insertion sort algorithm work.

VARIABLES & LABELS

Understanding how sorting works begins with recognizing how we name and refer to both data and positions within a dataset. In programming, we use variables as labels that help us hold and manipulate information.

> A VARIABLE IS A LABEL WHOSE VALUE OR CONTENT CAN CHANGE.

In the insertion sort process, we use several key variables to make the algorithm readable and workable. These variables may represent either the data values themselves or their positions (called indices) within the list:

- **hold_index** refers to the index whose data is temporarily held for comparison.
- **hold_data** refers to the data at the hold_index.
- **compare_index** refers to the index that precedes the hold_index and against which the held data is compared.
- **compare_data** refers to the data at the compare_index.

It is important to remember the difference between data and index:

- Indices are fixed positions in the list—they do not change.
- Data can move from one index to another during sorting.

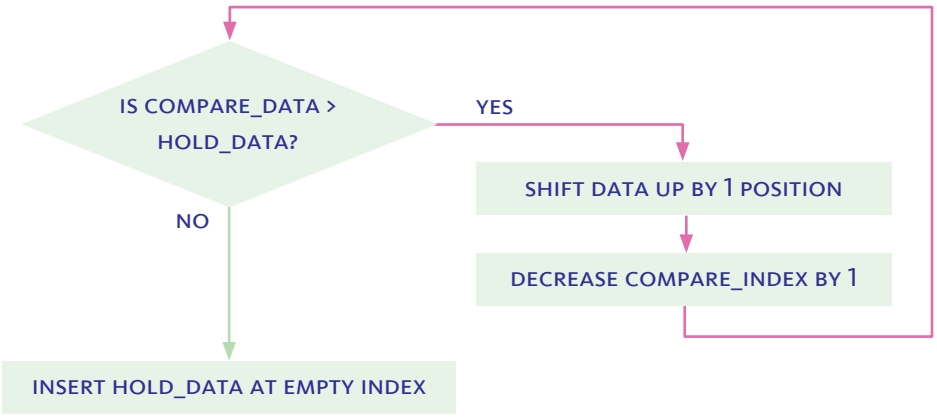
Flow & Control

Every algorithm has a flow or sequence of steps that are executed in order to achieve the required outcome—in this case, sorting a list of numbers in ascending order. This flow is controlled by certain conditions that determine the sequence in which instructions are carried out.

When a group of instructions needs to be repeated, they are collectively referred to as a loop. The insertion sort algorithm can generally be described as having two repeating sets of steps, which we will refer to as the outer loop and the inner loop.

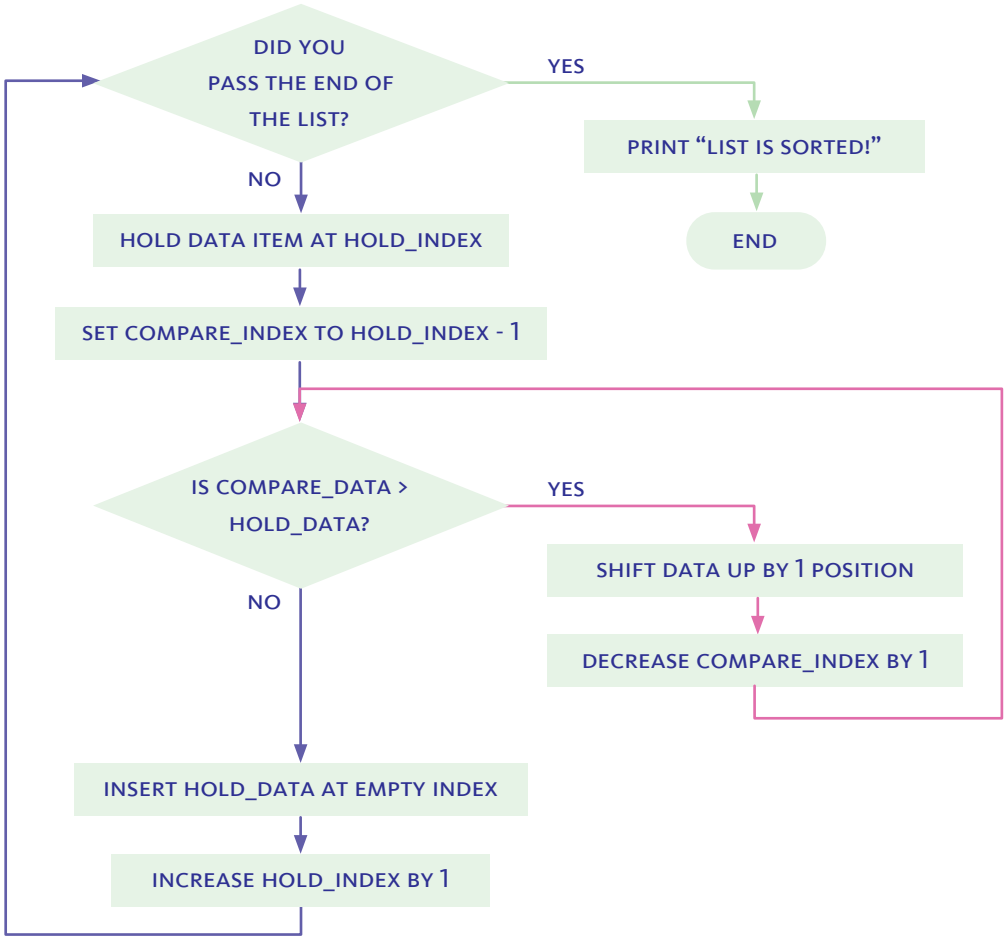
The **inner loop** performs a series of steps that compare the held data item with the items before it in the list. If the compared data is greater than the held data, it is shifted forward by one position. This process repeats as long as that condition is true.

Once the compared data is less than or equal to the held data, the loop ends, and the algorithm inserts the held data into the correct empty position toward the beginning of the list.



The inner loop compares and shifts data, moving one position at a time toward the beginning of the list. With each step, it shifts larger data items up by one index. The loop breaks when the compared data is less than or equal to the held data, at which point the held data is inserted into the empty index created by the shifting.

The **outer loop** is tasked with moving the inner loop sequentially through each index in the list. It starts at index 1, and at each step, it passes control to the inner loop to perform the comparisons, shifts, and insertion. When the inner loop completes its task, control is handed back to the outer loop, which moves the process forward by one index. The outer loop breaks when its condition is satisfied—that is, when it reaches the end of the list.



This flowchart highlights the two main components of the insertion sort process. The outer loop (blue) moves sequentially through each data item in the list using the hold_index. The inner loop (pink) repeatedly compares the held data with preceding items and shifts larger values upward until the correct position is found for insertion.

INITIALIZING

Initialization is the first part of the algorithm where we get everything ready before sorting begins. In this phase, we open the list we want to sort and set the starting point by choosing the first index we'll work with.

This flowchart shows the step-by-step process of the insertion sort algorithm. It begins with initialization, then uses an outer loop to move through the list and an inner loop to compare and shift data until each item is inserted into its correct position.

THE INSERTION SORT PROGRAM

Having understood the mechanics of the insertion sort algorithm and the flow of its process, it is now time to write the instructions for a program that implements this algorithm.

The program is divided into four blocks:

1. **Initialization** — this is where we begin by opening the list and declaring the variables needed for the sorting process. Before a variable can be used, it must first be declared and given an initial value. In this case, we declare hold_index and assign it the value 1, so that sorting starts with the second item in the list. We also declare end_index and assign it the value of the last index in the list, which sets the boundary for when the sorting process should stop.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
       SHIFT compare_data FROM compare_index TO compare_index + 1
       RESET compare_index (compare_index - 1)
       RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the initialization block of the insertion sort algorithm. This section sets up the list and defines key variables—hold_index and end_index—to prepare the algorithm before the sorting process begins.

2. **Outer loop** — which moves one step at a time through the list, selecting the next data item to be sorted.

   This loop is controlled by a **REPEAT_UNTIL** instruction, which allows the loop to continue until it reaches the end of the list—i.e., when the hold_index is greater than the end_index.

   Inside this block, we initialize the hold_data to be the data item at hold_index, the compare_index to be one position before the hold_index, and the compare_data to be the value at the compare_index.

   The block concludes with inserting the held data into the appropriate position, as determined by the inner loop, and then incrementing hold_index by 1 to prepare for the next outer loop cycle.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
       SHIFT compare_data FROM compare_index TO compare_index + 1
       RESET compare_index (compare_index - 1)
       RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the outer loop of the algorithm. This loop controls the progression through the list, selecting one data item at a time using the hold_index. For each pass, it prepares the held data and sets up comparisons before calling the inner loop. The loop continues until the hold_index has passed the end_index, meaning the entire list has been sorted.

3. **Inner loop** — which compares the held data with the values before it, shifts out-of-order data up the list, and determines where the held data should be inserted.

   This loop uses an **AS_LONG_AS** instruction, which means that as long as the value of hold_data is greater than the value of compare_data, the algorithm continues shifting the compare_data up by one position and resetting compare_index one step back.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
       SHIFT compare_data FROM compare_index TO compare_index + 1
       RESET compare_index (compare_index - 1)
       RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the inner loop, which repeatedly compares the held data with earlier items in the list. As long as the held data is smaller, larger values are shifted upward one position. The loop continues until the correct position is found for inserting the held data.

Notes

Program blocks can be arranged:

- Sequentially – one after another in order.
- Nested – with one block contained inside another.

4. **Termination** — After all data items have been compared, shifted, and inserted into their correct positions, the program reaches its termination block.

   This part is not part of any loop. It simply prints a message to indicate that the sorting is complete and then ends the program. It serves as a clear signal that the algorithm has finished running.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
       SHIFT compare_data FROM compare_index TO compare_index + 1
       RESET compare_index (compare_index - 1)
       RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the termination block of the program. This block runs after the sorting process is complete. It prints a message to confirm that the list has been successfully sorted and then ends the program.

Notes

Indentation in a program shows which instructions belong to a block. Lines indented under a loop or condition are part of that block. Further indentation indicates nested blocks.

The Bubble Sort Algorithm

The bubble sort algorithm follows a strategy of pair, compare, and swap. Starting at the beginning of the list, it repeatedly compares pairs of adjacent data items. If the two items are out of order, they are swapped so that the values begin to move toward their correct positions.

THE BUBBLE SORT ALGORITHM FOLLOWS A SEQUENTIAL STRATEGY OF PAIR, COMPARE, AND SWAP.

If the goal is to sort in ascending order, greater values are bubbled toward the end of the list with each full pass, while smaller values gradually settle into place near the beginning. This process is repeated through multiple passes until the entire list is sorted.

If the goal is to sort in descending order, each pass pushes the smallest unsorted value toward the end, while the greater values settle into place near the beginning.

Example 4.5

Let's go back to the same list we used in Example 4.4, and this time use the bubble sort algorithm to sort its data in ascending order.

"5, 3, 4, 1, 2"

The bubble sort algorithm makes several passes through the list, each time sequentially comparing pairs of data items starting from the beginning. Like the insertion sort algorithm, bubble sort uses two nested loops. The outer loop controls the number of passes through the list, with each pass considered an iteration of the loop. The inner loop handles the individual pairing, comparing, and swapping of data items, and it iterates through each pair in the list.

The First Iteration

Step 1 - Pair & Compare
Index 0 and Index 1 are paired.
Yes, "5" is greater than "3"

Step 2 - Swap
"5" and "3" are swapped.

Step 1 - Pair & Compare
Index 1 and Index 2 are paired.
Yes, "5" is greater than "4"

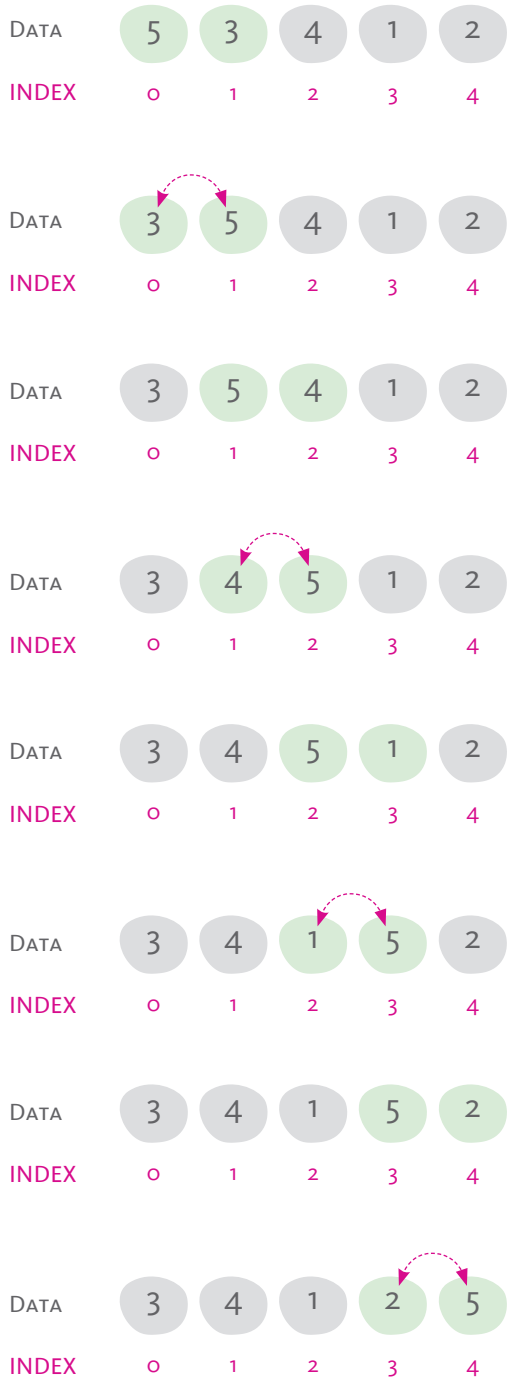
Step 2 - Swap
"5" and "4" are swapped.

Step 1 - Pair & Compare
Index 2 and Index 3 are paired.
Yes, "5" is greater than "1"

Step 2 - Swap
"5" and "1" are swapped.

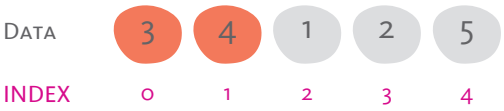
Step 1 - Pair & Compare
Index 3 and Index 4 are paired.
Yes, "5" is greater than "2"

Step 2 - Swap
"5" and "2" are swapped.



The Second Iteration

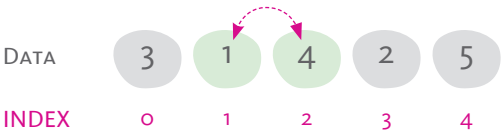
Step 1 - Pair & Compare
Index 0 and Index 1 are paired.
No, "3" is not greater than "4"



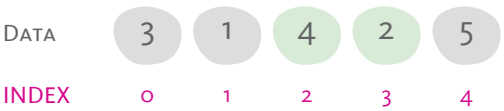
Step 2 - Swap
"3" and "4" are not swapped.

Step 1 - Pair & Compare
Index 1 and Index 2 are paired.
Yes, "4" is greater than "1"

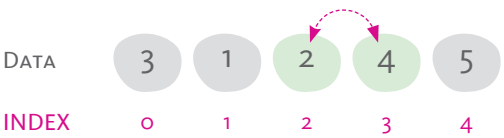
Step 2 - Swap
"4" and "1" are swapped.



Step 1 - Pair & Compare
Index 2 and Index 3 are paired.
Yes, "4" is greater than "2"



Step 2 - Swap
"4" and "2" are swapped.

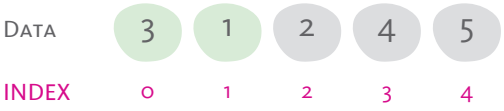


Step 1 - Pair & Compare
Index 3 and Index 4 are paired.
Yes, "4" is not greater than "5"

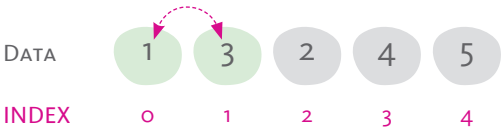
Step 2 - Swap
"4" and "5" are not swapped.

The Third Iteration

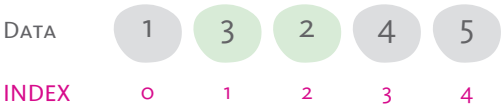
Step 1 - Pair & Compare
Index 0 and Index 1 are paired.
Yes, "3" is greater than "1"



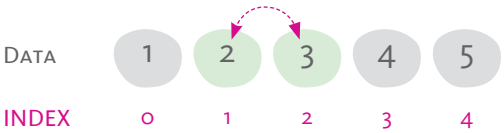
Step 2 - Swap
"3" and "1" are swapped.



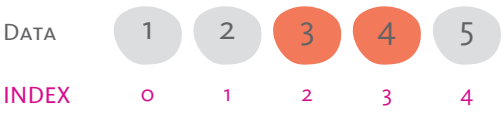
Step 1 - Pair & Compare
Index 1 and Index 2 are paired.
Yes, "3" is greater than "2"



Step 2 - Swap
"3" and "2" are swapped.



Step 1 - Pair & Compare
Index 2 and Index 3 are paired.
No, "3" is not greater than "4"



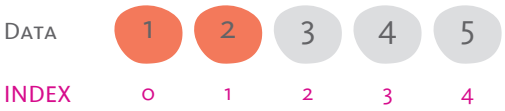
Step 2 - Swap
"3" and "4" are not swapped.

Step 1 - Pair & Compare
Index 3 and Index 4 are paired.
No, "4" is not greater than "5"

Step 2 - Swap
"4" and "5" are not swapped.

The Fourth Iteration

Step 1 - Pair & Compare
Index 0 and Index 1 are paired.
No, "1" is not greater than "2"

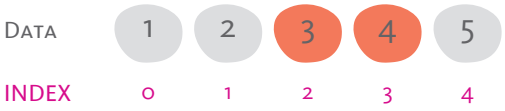


Step 2 - Swap
"1" and "2" are not swapped.

Step 1 - Pair & Compare
Index 1 and Index 2 are paired.
No, "2" is not greater than "3"

Step 2 - Swap
"2" and "3" are not swapped.

Step 1 - Pair & Compare
Index 2 and Index 3 are paired.
No, "3" is not greater than "4"



Step 2 - Swap
"3" and "4" are not swapped.

Step 1 - Pair & Compare
Index 3 and Index 4 are paired.
No, "4" is not greater than "5"

Step 2 - Swap
"4" and "5" are not swapped.

The Bubble Sort Process

Having understood the mechanics and demonstrated the algorithm step by step, we can now take a closer look at the underlying operations and conditions that make the bubble sort process work. By identifying these key components, we can begin to organize the logic into clear instructions and structured loops that reflect how the algorithm actually functions.

Data & Indices

As in any algorithm, we must start by defining our variables. These variables must be declared before they are used, and each is assigned an initial value that may change as the algorithm runs.

- **hold_index** refers to the index whose data is temporarily held for comparison.
- **hold_data** refers to the data at the hold_index.
- **compare_index** refers to the index that precedes the hold_index, and against which the held data is compared.
- **compare_data** refers to the data at the compare_index.
- **end_index** refers to the last index in the list. In this case, it remains fixed throughout the sorting process.

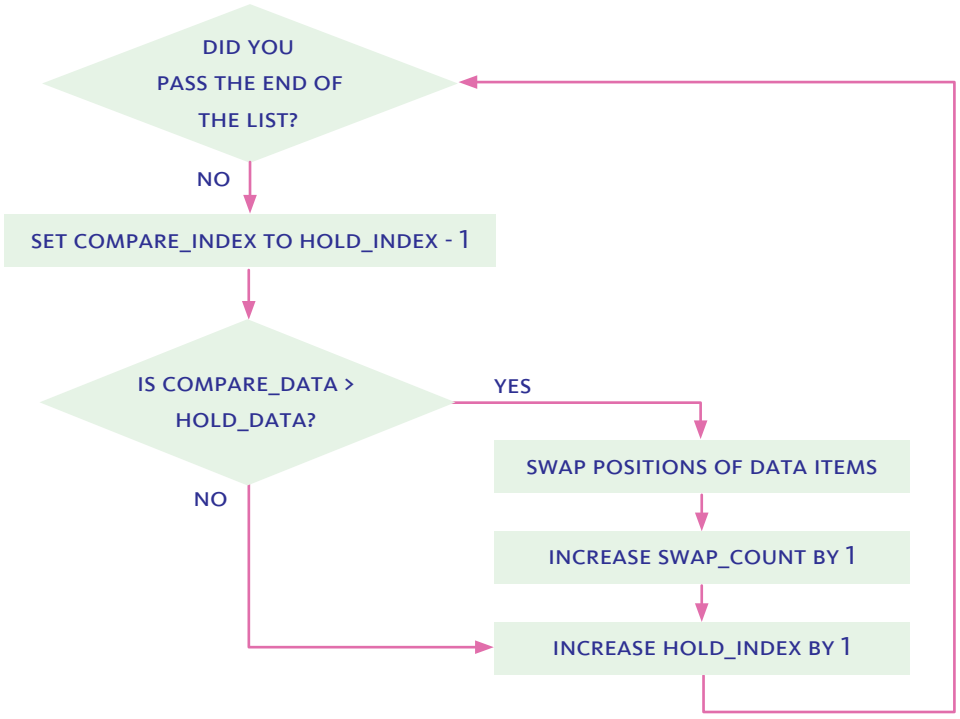
In addition to these variables—previously encountered in the insertion sort algorithm—the bubble sort algorithm requires an additional variable:

- **swap_count** is a variable that keeps track of the number of times a swap occurs during a single pass through the list.

Loops & Conditions

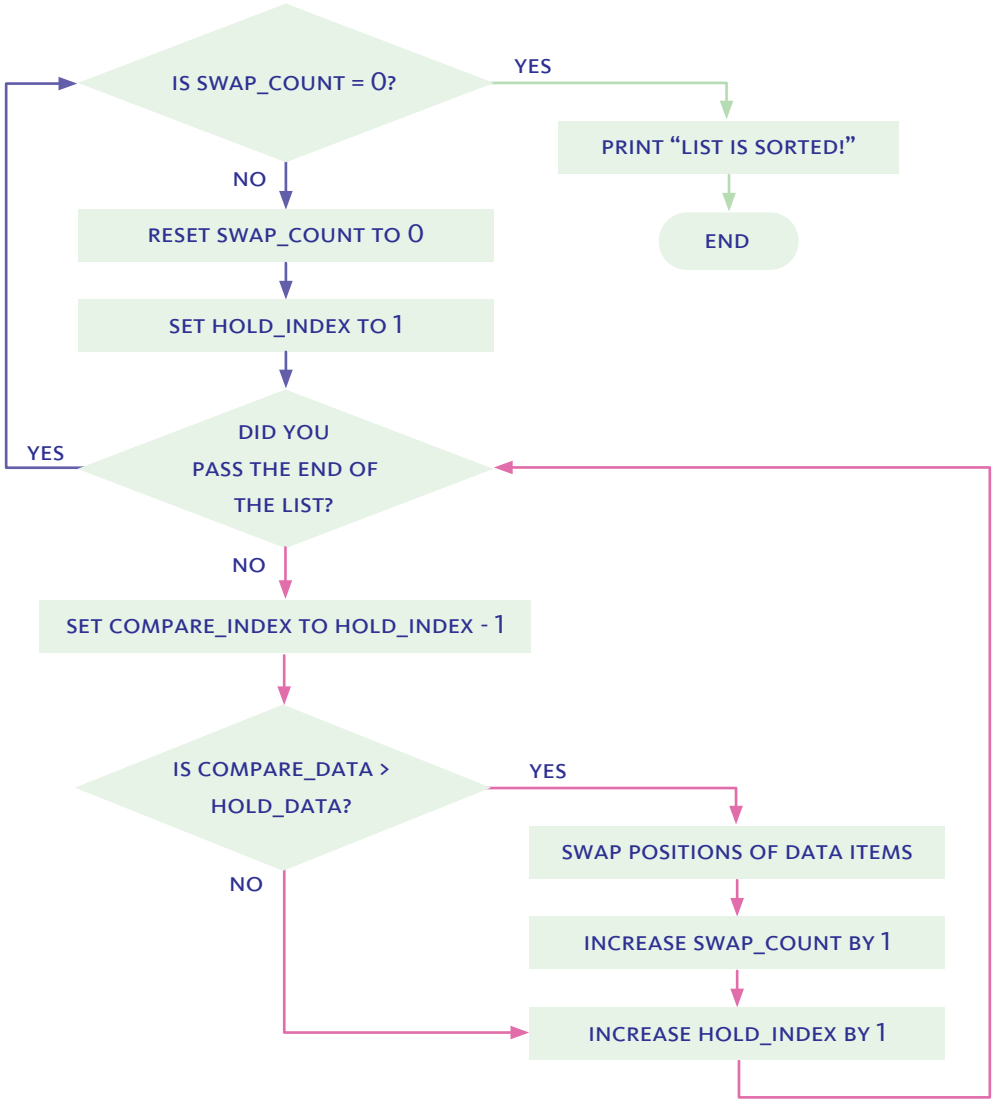
The bubble sort algorithm uses two loops: an inner loop and an outer loop. The inner loop performs a single pass through the list, pairing, comparing, and swapping adjacent data items. The outer loop repeats these passes until no swaps are needed, meaning the list is fully sorted.

The **inner loop** performs a series of steps that compare the held data item with the items before it in the list. If the compared data is greater than the held data, it is shifted forward by one position. This process repeats as long as that condition is true. Once the compared data is less than or equal to the held data, the condition is no longer satisfied. At that point, the loop ends, and the algorithm inserts the held data into the correct empty position toward the beginning of the list.



This flowchart illustrates the inner loop of the bubble sort process. It begins by setting the compare_index to one position before the hold_index, forming a pair of adjacent data items. The algorithm compares their values and swaps them if needed. Each time a swap occurs, the swap_count is increased. The hold_index then moves forward, and the loop continues until the end of the list is reached, completing a full pass.

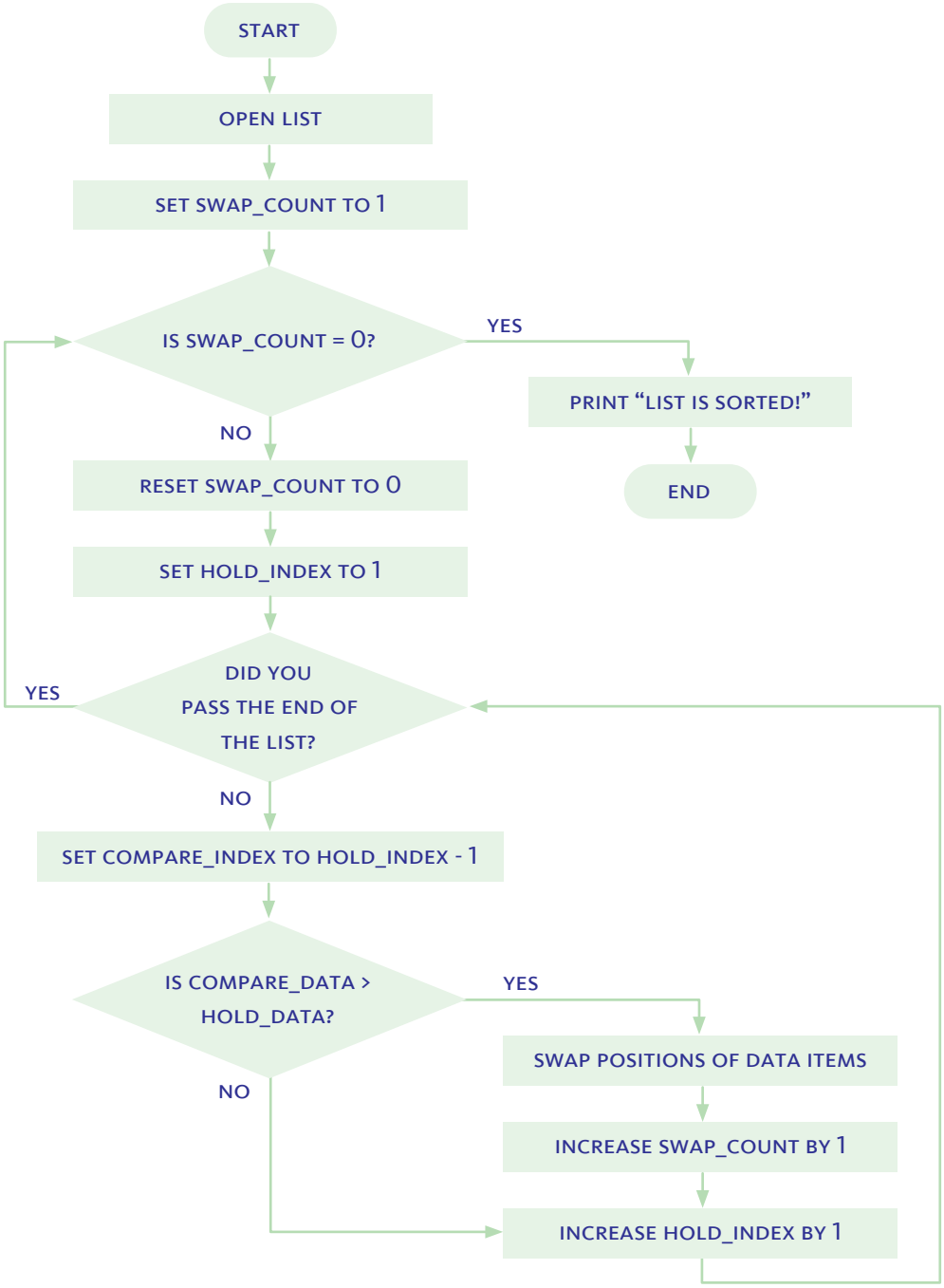
The **outer loop** is tasked with repeating the passes until the list is fully sorted. After each pass, it checks swap_count. If swap_count is 0, no swaps occurred, and the list is sorted. If not, the list is not completely sorted. The loop prepares for another pass by resetting swap_count to 0, moving hold_index back to the start, and running the pass again. This continues until a pass completes with no swaps.



The outer loop (blue) repeats passes through the list until no swaps occur. The inner loop (pink) performs a single pass, comparing and swapping adjacent data items as needed.

INITIALIZING

First, the list is opened, and the variable swap_count is set to 1. Since the loop stops when swap_count equals 0—meaning no swaps were made—we cannot start with it set to 0, or the algorithm would end before it begins. Setting it to 1 ensures that the sorting process is able to start.



The Bubble Sort Program

Now that we understand how the bubble sort algorithm works and how its steps flow, we can begin writing the instructions for a program that implements it.

The program has four blocks:

1. **Initialization** — This is where we begin by opening the list and declaring the variables needed for the sorting process. Before a variable can be used, it must first be declared and given an initial value. In this case, we declare end_index and assign it the value of the last index in the list, setting the boundary for the sorting range. We also declare swap_count and assign it an initial value of 1 to ensure that the algorithm enters the outer loop for its first pass.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
    RESET swap_count (0)
    SET hold_index (1)

    REPEAT_UNTIL hold_index > end_index
       SET hold_data (data_item AT hold_index)
       SET compare_index (hold_index - 1)
       SET compare_data (data_item AT compare_index)

       IF compare_data > hold_data THEN
              SWAP_POSITIONS (compare_data, hold_data)
              RESET swap_count (swap_count + 1)

       RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink brackets and green text indicate the initialization block of the bubble sort algorithm. These variables are essential to prepare the algorithm before the sorting process starts.

2. **Outer loop** — The outer loop controls the overall sorting process by repeating passes through the list until it is fully sorted.

   It begins with a **REPEAT_UNTIL** instruction that continues looping until swap_count is 0, indicating no swaps occurred during the last pass. At the start of each pass, the algorithm resets swap_count to 0 and sets hold_index to 1, preparing to step through the list. This outer loop ensures that the sorting continues as long as necessary to bring all elements into order.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
    RESET swap_count (0)
    SET hold_index (1)

    REPEAT_UNTIL hold_index > end_index
       SET hold_data (data_item AT hold_index)
       SET compare_index (hold_index - 1)
       SET compare_data (data_item AT compare_index)

       IF compare_data > hold_data THEN
              SWAP_POSITIONS (compare_data, hold_data)
              RESET swap_count (swap_count + 1)

       RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the outer loop block of the bubble sort algorithm. This section controls how many passes the algorithm makes through the list, repeating the process until no swaps occur—signaling that the list is sorted.

3. **Inner loop** — is responsible for performing a single pass through the list, comparing and swapping adjacent items if needed.

   It begins with a **REPEAT_UNTIL** instruction that runs as long as hold_index is less than or equal to end_index. At each step, the algorithm sets hold_data to the value at hold_index, identifies compare_index as the position just before it, and retrieves compare_data from that position. If compare_data is greater than hold_data, the two values are swapped, and swap_count is increased by one. The hold_index is then incremented, and the loop continues through the rest of the list.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
    RESET swap_count (0)
    SET hold_index (1)

    REPEAT_UNTIL hold_index > end_index
       SET hold_data (data_item AT hold_index)
       SET compare_index (hold_index - 1)
       SET compare_data (data_item AT compare_index)

       IF compare_data > hold_data THEN
              SWAP_POSITIONS (compare_data, hold_data)
              RESET swap_count (swap_count + 1)

       RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the inner loop block of the bubble sort algorithm. This section performs comparisons and swaps between adjacent items as it moves through the list from left to right during a single pass.

4. **Termination** — Once the sorting process is complete—meaning a full pass has been made with no swaps—the algorithm exits the outer loop. At this point, it prints the message "List is sorted!" to confirm that the list has been successfully ordered. The program then ends.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
    RESET swap_count (0)
    SET hold_index (1)

    REPEAT_UNTIL hold_index > end_index
        SET hold_data (data_item AT hold_index)
        SET compare_index (hold_index - 1)
        SET compare_data (data_item AT compare_index)

        IF compare_data > hold_data THEN
               SWAP_POSITIONS (compare_data, hold_data)
               RESET swap_count (swap_count + 1)

        RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the termination block of the bubble sort algorithm. This section confirms that the list is sorted and marks the end of the program.

Notes

AS_LONG_AS and REPEAT_UNTIL resemble natural language, making them ideal for beginners. They help students focus on how conditions control the flow of a program and highlight the key distinction between pre-check and post-check logic:

- AS_LONG_AS runs while a condition remains true.
- REPEAT_UNTIL runs until a condition becomes true.

Performance

When more than one algorithm can solve the same problem, performance becomes an important way to compare them. Two measures of performance are **step count** and **execution time**.

> Step count and execution time are two measures of performance.

Step count refers to the number of steps required by the algorithm to complete the task—which, in this case, is sorting. This measure helps us understand the structure and logical efficiency of an algorithm. Fewer steps means the algorithm performs a smaller number of operations to complete its task.

Execution time refers to how long it takes for the program to run and sort a particular dataset. This measure tells us about performance in practice. Shorter execution time means the program finishes faster—but that also depends on how long each type of step takes.

Both measures are useful—and together, they give us a fuller picture of performance.

Worst-Case Scenario

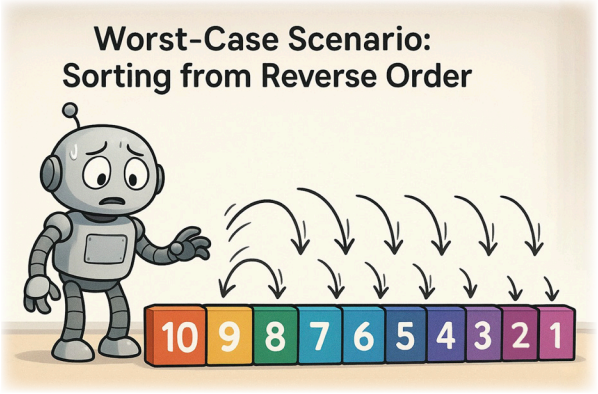
Algorithms and programs can perform differently depending on the dataset they are given. Since datasets can vary considerably, we typically use the worst-case scenario as a standard way to evaluate performance.

In the case of sorting, the worst-case scenario occurs when the list is arranged in the exact opposite order of the desired outcome. So, if we are trying to sort a list in ascending order, the worst case would be a list that starts in descending order. In this situation, the algorithm must shift or swap nearly every item to put the list in the correct order.

We use the worst case because:

1. It shows us the maximum number of steps an algorithm might take—this helps us understand its performance limits.
2. It prepares us for real-world situations where the data might be in the least helpful or most disorganized state.
3. It allows us to fairly compare algorithms by seeing how they handle the same challenging scenario.

In other words, if an algorithm performs well in the worst case, we can be confident that it will perform even better when the data is easier to work with.



INSERTION SORT

Let's explore the performance of insertion sort under the worst-case scenario: a list that is completely reversed.

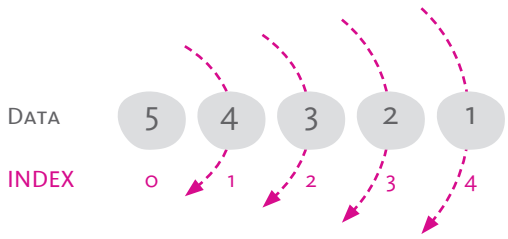
We will assume a list of 5 numbers that need to be sorted in ascending order. Since we want the worst-case, the list will start in descending order, meaning every number is in the opposite position of where it should be.

DATA	5	4	3	2	1
INDEX	0	1	2	3	4

Step Count

When we examine the steps, we'll group them into two categories: **comparisons** and **moves**. The difference is that comparisons can be done with data in place, while moves involve the physical repositioning of data—either through a hold, shift, or insert.

The algorithm starts at index 1 and moves through the list one index at a time until the end. This process is controlled by the outer loop. For a list with 5 data items, the outer loop will run 4 times, and during each iteration, it will hold one item for sorting.



Each curved arrow represents one iteration of the outer loop in the insertion sort algorithm. At each index, the algorithm holds the current value and prepares to compare it with the values in all preceding indices. This process repeats from index 1 to the end of the list.

Now let's look at the inner loop. At each index, the inner loop compares the held data with the data at all preceding indices. Since this is a worst-case scenario, every comparison results in a shift.

- At index 1, there is 1 comparison and 1 shift.
- At index 2, there are 2 comparisons and 2 shifts.
- At index 3, there are 3 comparisons and 3 shifts.
- At index 4, there are 4 comparisons and 4 shifts.

Since this is a worst-case scenario, every comparison results in a shift. That means the number of shifts is equal to the number of comparisons, which again equals the number of preceding indices.

Finally, we are left with the insertions. These occur once during each iteration of the outer loop, giving us 4 insertions in total.

Index	Hold	Compare	Shift	Insert
1	1	1	1	1
2	1	2	2	1
3	1	3	3	1
4	1	4	4	1
Total	**4**	**10**	**10**	**4**

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the insertion sort algorithm requires a total step count of 4 (holds) + 10 (comparisons) + 10 (shifts) + 4 (insertions) = 28 steps.

Execution Time

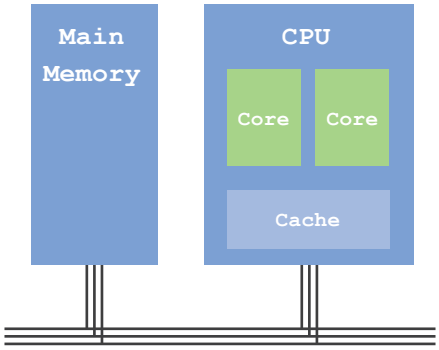
Execution time—ideally measured by actually running the program on a computer—can also be estimated using basic assumptions and an understanding of the steps the algorithm performs.

To keep our analysis realistic yet manageable, we categorize steps into two types:

1. Processor-only steps (like comparisons), which involve only the CPU cores and their internal registers.
2. Memory-related moves (like holds, shifts, and insertions), which involve moving data between memory locations, such as between cache and main memory (RAM).

In a real computer, memory access time can vary depending on whether data is stored in fast-access cache or slower main memory. However, since our

dataset is small, it's reasonable to assume that all memory-related steps occur within the processor's cache. Cache access is faster than accessing main memory, but still slower than internal CPU operations.



This figure is a simplified representation of the processor and memory components in a computer's architecture. The CPU contains multiple cores for executing instructions and a cache for fast, nearby access to frequently used data. The main memory (RAM) holds larger amounts of data for longer periods while programs run, but accessing it is slower than accessing the cache.

For the sake of estimation:

- We will assign 1 nanosecond (ns) to processor-only steps.
- We will assign 5 nanoseconds (ns) to memory-related steps, assuming cache-level latency.

This allows us to make performance comparisons based on an estimated execution time:

Notes

The time estimates used in these examples are intentionally simplified to support conceptual understanding. While actual timings depend on hardware architecture and system conditions, the goal is to help students recognize that different types of operations vary in cost—and that performance can be meaningfully estimated using reasonable assumptions.

- Each comparison = 1 ns
- Each hold, shift, or insert = 5 ns

By multiplying the number of each type of step by its estimated time cost, we can calculate the total execution time of the algorithm in nanoseconds.

This approach gives us a practical way to reason about performance, even without running the program on a real computer.

Index	Hold	Compare	Shift	Insert
1	1 (x 5)	1 (x 1)	1 (x 5)	1 (x 5)
2	1 (x 5)	2 (x 1)	2 (x 5)	1 (x 5)
3	1 (x 5)	3 (x 1)	3 (x 5)	1 (x 5)
4	1 (x 5)	4 (x 1)	4 (x 5)	1 (x 5)
Total	**20 ns**	**10 ns**	**50 ns**	**20 ns**

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the insertion sort program has an estimated execution time of 20 ns (holds) + 10 ns (comparisons) + 50 ns (shifts) + 20 ns (insertions) = 100 ns.

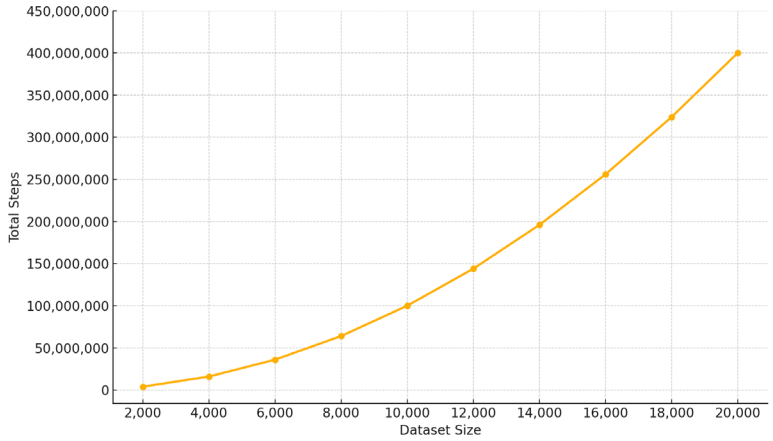
SCALABILITY

Now that we've explored how insertion sort performs on a small dataset, we can begin to explore what happens as the size of the dataset increases—a concept known as scalability.

As the dataset grows, both the step count and the execution time increase. This is especially true under the worst-case scenario, where the list is in completely reversed order.

As we scale up the size of the dataset, we can clearly see the impact on step count. With every new item added, the algorithm must compare it with all preceding items and shift them if necessary. This means that each additional item increases the total number of steps more than the one before it.

If we run simulations on datasets of increasing size—starting from 2,000 and going up to 20,000 items—we can demonstrate that, under the worst-case scenario, the step count grows rapidly. In fact, the amount of increase itself becomes greater with each step up in dataset size. This shows that the algorithm does not scale well and performs poorly as the size of the dataset increases.



Dataset Size	Step Count
2,000	4,001,998
4,000	16,003,998
6,000	36,005,998
8,000	64,007,998
10,000	100,009,998
12,000	144,011,998
14,000	196,013,998
16,000	256,015,998
18,000	324,017,998
20,000	400,019,998

This table and graph show how the total number of steps required by the insertion sort algorithm increases as the dataset size grows from 2,000 to 20,000 items under the worst-case scenario. The steep upward curve illustrates how step count accelerates with each additional item, highlighting the algorithm's poor scalability for large datasets.

Notes

This is an example of quadratic growth. The total step count increases approximately with the square of the dataset size, which is why performance drops rapidly as the dataset grows.

Now let's examine how step count reflects on execution time. One might assume that execution time increases in the same way step count increases as the dataset grows—and while that's generally true, execution time behaves slightly differently. That's because it depends not only on the number of steps or operations, but also on how long each operation takes to execute.

The time required to execute an operation depends on where the data is stored. The closer the data is to the processor, the faster it can be accessed. Small datasets usually fit entirely in the CPU's cache, where access is very fast. But with large datasets, the processor may need to access main memory (RAM), which is much slower.

This introduces memory latency—the time it takes to fetch or store data from memory—which becomes a noticeable part of total execution time for large datasets.

EXECUTION TIME DEPENDS ON BOTH STEP COUNT AND MEMORY LATENCY.

Going back to our simulation, let's determine the memory latency for each operation. As the dataset increases in size, more memory is needed to hold it. This means the data is likely to be held further from the processor, moving from fast-access cache to slower main memory (RAM).

Because of this, we assign different latency values depending on the dataset size:

- For small datasets, the data is likely held in the CPU's cache, where access is very fast.
- For medium-sized datasets, the data may spill into L3 cache, which is slower.
- For larger datasets, the data is held in main memory (RAM), where access time is significantly longer.

Comparisons always occur within the CPU's registers and are very fast (typically around 1 nanosecond or less).

Memory-related moves—including holds, shifts, and insertions—require access to memory beyond the CPU and are more sensitive to dataset size and memory latency.

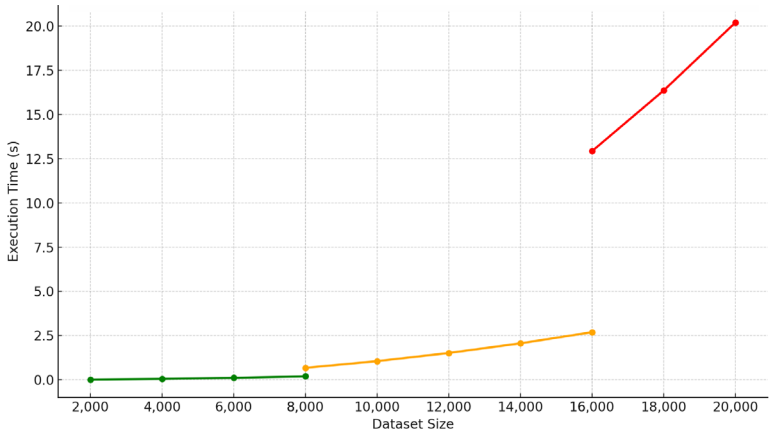
Dataset Size	Memory Location	Comparison (CPU)	Hold / Shift / Insert (Memory)
2,000–8,000	L1/L2 Cache	1 ns	5 ns
8,001–16,000	L3 Cache	1 ns	20 ns
16,001–20,000	Main Memory	1 ns	100 ns

According to these estimates, we can calculate execution times for each dataset size. As the dataset grows, it gradually begins spilling into higher memory tiers. Smaller datasets are held close to the processor in the L1 and L2 caches, but as the size exceeds the capacity of these caches, the data begins to spill into the L3 cache—which, in our simplified simulation, occurs at around 8,000 items. For even larger datasets that exceed the capacity of the L3 cache (around 16,000 items in the simulation), the data must be held in main memory (RAM).

Notes

In real systems, memory access doesn't shift abruptly between tiers. Instead, as the dataset grows, portions of it gradually move from fast-access cache to slower memory. Our simulation simplifies this behavior by assigning a dominant memory tier based on dataset size. This helps students understand the concept of memory latency without needing to model the complexities of real-world caching systems.

As a result, the algorithm slows down not only due to the increasing number of steps, but also due to memory latency—the added time required to access data stored in slower memory tiers.



Dataset Size	Execution Time
2,000	12,013,990
4,000	48,027,990
6,000	108,041,990
8,000	192,055,990
10,000	1,050,294,960
12,000	1,512,353,960
14,000	2,058,412,960
16,000	2,688,471,960
18,000	16,364,690,800
20,000	20,202,989,800

This table and graph show how execution time increases as dataset size grows from 2,000 to 20,000 items using the insertion sort algorithm under the worst-case scenario. Execution time is shown in nanoseconds in the table and converted to seconds in the graph. The curve is divided into three color-coded segments based on the type of memory used—cache (green), L3 cache (orange), and main memory (red). The sharp rise in execution time highlights the combined effect of step count growth and increased memory latency, revealing the algorithm's poor scalability with larger datasets.

BUBBLE SORT

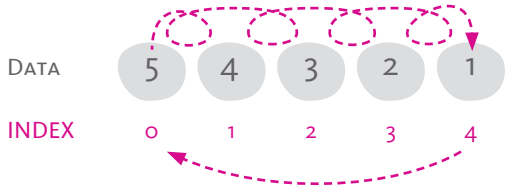
Let's explore the performance of the bubble sort algorithm in the same way we did for insertion sort. We'll start with a small list of 5 numbers that need to be sorted in ascending order. Since we're interested in the worst-case scenario, the list will begin in descending order—meaning that every number is in the opposite position of where it should be.

DATA	5	4	3	2	1
INDEX	0	1	2	3	4

Step Count

In the same way we examined step count for the insertion sort algorithm, we'll group the steps into two categories: comparisons and moves. The difference is that instead of holds, shifts, or insertions, bubble sort uses swaps to move data.

The bubble sort algorithm repeatedly compares adjacent pairs of data items and swaps them if they are in the wrong order. This process is controlled by the outer loop, which runs a fixed number of times regardless of whether the list becomes sorted earlier. For a list with 5 items, the outer loop runs 4 times, and during each pass, the inner loop performs 4 comparisons.



This diagram shows the structure of a single pass in the bubble sort algorithm. Each curved arrow represents a comparison between adjacent values. In the worst-case scenario, every comparison results in a swap. While the number of comparisons remains the same in every pass, the number of swaps decreases as larger values are moved to the end of the list.

Each pass in the bubble sort algorithm makes 4 comparisons (for a 5-item list), regardless of whether the list becomes partially sorted. The pattern of comparing adjacent pairs remains the same across all passes. However, the number of swaps decreases with each pass because the largest remaining value is moved into its correct position at the end of each round.

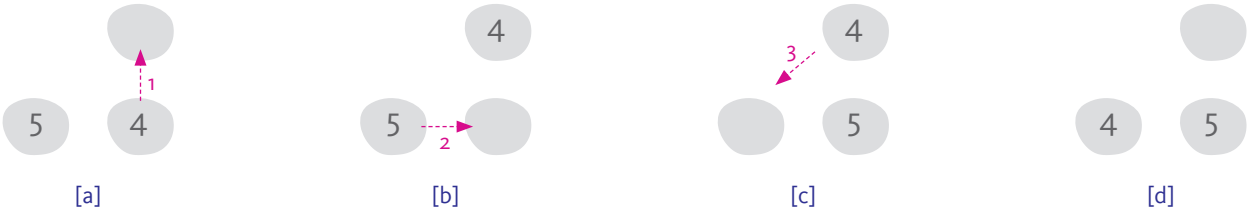
- In Pass 1, there are 4 comparisons and 4 swaps.
- In Pass 2, there are 4 comparisons and 3 swaps.
- In Pass 3, there are 4 comparisons and 2 swaps.
- In Pass 4, there are 4 comparisons and 1 swap.

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the bubble sort algorithm requires a total step count of 16 (comparisons) + 10 (swaps) = 26 steps.

Pass	Compare	Swaps
1	4	4
2	4	3
3	4	2
4	4	1
Total	**16**	**10**

However, it is important to examine the swap in detail as a type of move. A swap is not simply a single move of data from one place in memory to another; it actually requires three separate moves.

In real memory operations, one data item cannot simply overwrite a location already occupied by another. Instead, the existing value must first be moved to an empty temporary location, the target location is then overwritten with the new value, and finally, the temporarily held value is written into the second location. This sequence means that a single swap involves three distinct moves.



This figure shows the three moves involved in a swap operation during bubble sort. [a] The value 4 is moved to a temporary location. [b] The value 5 takes the place of 4. [c] The stored value 4 is moved from the temporary location into 5's original position. [d] The swap is complete, with 4 and 5 now in their new positions.

Accordingly, we must revise our step count to account for three moves per swap. This gives a total step count of 16 (comparisons) + 30 (moves) = 46 steps.

Pass	Compare	Swaps	Moves
1	4	4	12
2	4	3	9
3	4	2	6
4	4	1	3
Total	**16**	**10**	**30**

Execution Time

Having detailed the steps required for the bubble sort algorithm to sort a worst-case scenario list of 5 items, we can now discuss the time it would take to execute such a task. Using the same estimates of time as before—comparisons at 1 ns each and moves at 5 ns each—we can calculate the execution time.

- Comparisons: 16 x 1 ns = 16 ns
- Moves: 30 x 5 ns = 150 ns

This gives a total execution time of: 16 ns + 150 ns = 166 ns.

When we compare the results for a worst-case scenario list of 5 items, we see a clear difference between the two algorithms. Insertion sort required 28 steps and 100 ns to complete the task, while bubble sort required 46 steps and 166 ns.

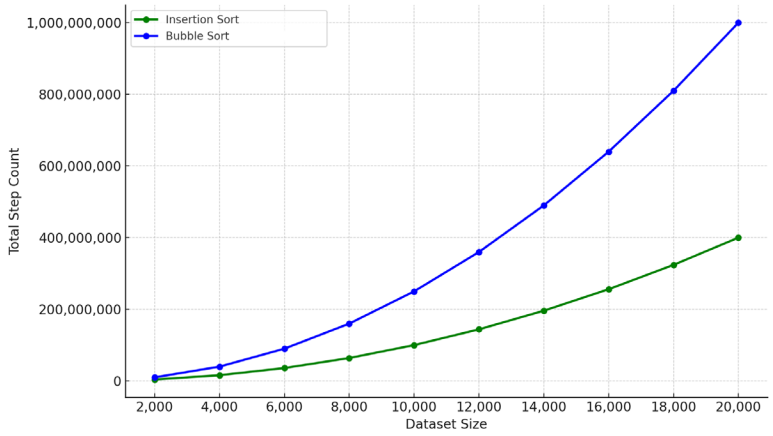
This means that bubble sort performed more operations and took longer to execute for the same dataset and conditions. Even at this small scale, the difference highlights bubble sort's lower efficiency compared to insertion sort in both step count and execution time.

SCALABILITY

While the difference in performance is already visible with a small dataset, it becomes even more pronounced as the dataset grows. Because bubble sort makes a fixed number of comparisons in every pass and performs swaps that require multiple moves, the total step count increases rapidly with larger lists.

As we saw with insertion sort, more steps mean more time—but for bubble sort, the higher number of moves per swap amplifies this effect. To understand the full impact, we need to examine how bubble sort scales with dataset size and how this affects execution time.

Let's return to our simulation using dataset sizes ranging from 2,000 to 20,000 data items. We will begin by comparing the insertion sort algorithm with the bubble sort algorithm, focusing on the scalability of their step counts.



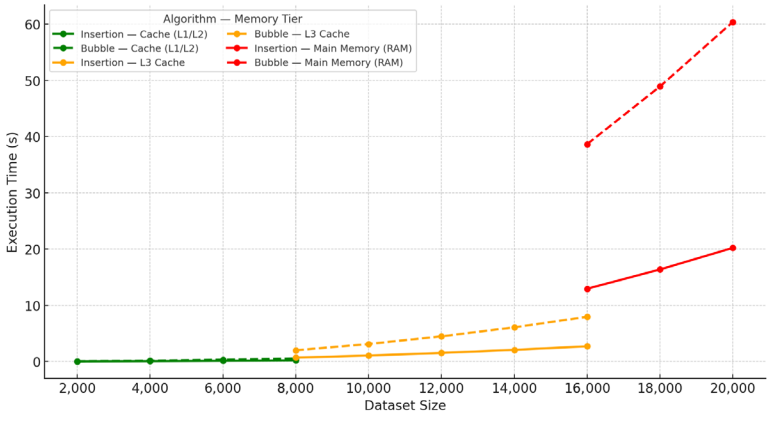
Dataset Size	Insertion Sort Step Count	Bubble Sort Step Count
2,000	4,001,998	9,993,001
4,000	16,003,998	39,986,001
6,000	36,005,998	89,979,001
8,000	64,007,998	159,972,001
10,000	100,009,998	249,965,001
12,000	144,011,998	359,958,001
14,000	196,013,998	489,951,001
16,000	256,015,998	639,944,001
18,000	324,017,998	809,937,001
20,000	400,019,998	999,930,001

This graph and table compare the total step counts for insertion sort and bubble sort under the worst-case scenario, for dataset sizes ranging from 2,000 to 20,000 items. Both algorithms exhibit quadratic growth, but bubble sort consistently requires about 2.5 times more steps. The gap between the two curves widens as dataset size increases, clearly illustrating bubble sort's poorer scalability.

Examining the graph and table, it becomes clear that bubble sort is far less efficient than insertion sort. The main reason is that bubble sort's structure forces it to perform a fixed number of comparisons in each pass, regardless of how much of the list is already sorted.

In a worst-case scenario, every comparison results in a swap, and each swap involves three separate moves in memory. This extra work adds up quickly, making bubble sort require about 2.5 times more steps than insertion sort for the same dataset size. As the dataset grows, this difference becomes increasingly significant, highlighting how the mechanics of an algorithm can have a major impact on its performance.

These differences become even more pronounced when we calculate the execution time required for each dataset. As the dataset grows, it often exceeds the capacity of faster memory tiers (such as the CPU's cache) and must be stored in slower tiers (such as L3 cache or main memory). This increase in memory latency further slows the execution of the sorting process, amplifying the performance gap between the two algorithms.



Dataset Size	Insertion Sort Execution Time	Bubble Sort Execution Time
2,000	12,013,990	33,981,001
4,000	48,027,990	135,962,001
6,000	108,041,990	305,943,001
8,000	192,055,990	543,924,001
10,000	1,050,294,960	3,099,680,001
12,000	1,512,353,960	4,463,616,001
14,000	2,058,412,960	6,075,552,001
16,000	2,688,471,960	7,935,488,001
18,000	16,364,690,800	48,921,264,001
20,000	20,202,989,800	60,396,960,001

This graph and table compare execution times for insertion sort and bubble sort under the worst-case scenario for dataset sizes from 2,000 to 20,000 items. Execution time is shown in nanoseconds in the table and converted to seconds in the graph. Times are segmented by memory tier—cache (green), L3 cache (orange), and main memory (red). Bubble sort's higher step count and costly swaps cause its execution time to grow much faster, especially as larger datasets move into slower memory.

Understanding algorithm performance goes beyond knowing whether an algorithm produces the correct result. How efficiently it solves a problem—measured in step count, execution time, and memory use—can make a significant difference, especially with large datasets or time-sensitive tasks.

As seen in our comparison of insertion sort and bubble sort, two algorithms with the same growth rate can perform very differently in practice due to their mechanics and the cost of individual operations. Choosing the right algorithm is therefore about matching the method to the problem, the dataset size, and the available computing resources.

SUMMARY

Sorting – the arrangement of data in a specific order based on a certain criterion.

- Order is the direction—ascending or descending.
- Criterion is what we sort by, such as exam scores, prices, names, or titles.
- Sorting helps organize data, making it easier to search, interpret, and analyze.

Algorithm & Program –

- An algorithm is a series of steps designed to perform a particular task.
- A program is the set of written instructions a computer can execute to run an algorithm.
- A variable is a label whose value or content can change. Variables in programming need to be declared, meaning their label is specified and an initial value is assigned.
- Flow is the sequence in which the steps of a program or algorithm are carried out.
- Control refers to the conditions that determine this sequence, such as when certain steps are executed, repeated, or skipped.
- Loops are a common control structure used to repeat a set of instructions, while conditional statements decide which instructions run based on specific criteria.

- A **block** is a segment of code that performs a particular task.A program is typically organized into three main blocks: initialization, main body, and termination. Blocks can be **sequential** (one after another) or **nested** (one inside another).
- Indentation is used to show which instructions belong to a loop block.

Insertion Sort – Builds a sorted section one item at a time.

- Outer Loop – Moves through each index in the list (starting at the second item) and hands control to the inner loop.
- Inner Loop – Compares the held item with preceding items, shifts items as needed, and inserts the held item into its correct position based on the sorting criterion.
- Strategy – Compare, shift, insert.

Bubble Sort – Repeatedly passes through the list, swapping adjacent items if out of order.

- Outer Loop – Repeats the inner loop for a fixed number of passes to ensure the list becomes fully sorted.
- Inner Loop – Compares adjacent pairs and swaps them if they are out of order, gradually moving items into their correct positions based on the sorting criterion.
- Strategy – Pair, compare, swap.

Performance –

- Step count and execution time are two key measures of performance.
- Worst-Case Scenario – The most challenging dataset arrangement for an algorithm (e.g., completely reversed list for ascending sort). Used for fair comparisons.
- Scalability refers to how an algorithm's performance changes as the dataset size increases.
- Memory latency is the delay between requesting data from memory and the moment it becomes available for use.
- Insertion sort is a more efficient algorithm than bubble sort, requiring fewer steps and less execution time. The difference widens as datasets grow, especially when memory latency becomes significant.

Activity 4.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
T	Sorting is the arrangement of data in a specific order based on a certain criterion.
F	In programming, a variable is a label whose value cannot change.
F	The order in sorting refers to the basis or criterion used to determine how data is arranged.
T	Flow refers to the sequence of steps in a program, while control refers to the conditions that decide which steps are executed, repeated, or skipped.
T	A program is typically organized into initialization, main body, and termination blocks.
T	In insertion sort, the inner loop compares the held item with preceding items, shifts items as needed, and inserts it in the correct position.
F	In bubble sort, each swap requires one move in memory.
T	Step count and execution time are two measures used to compare algorithm performance.
F	A worst-case scenario is the easiest dataset arrangement for an algorithm to sort.
T	As dataset size increases, the need for slower memory tiers can raise memory latency and increase execution time.

Activity 4.2

Answer the questions below.

1. What does "order" refer to in sorting?
   a) The basis for sorting
   b) The sequence in which algorithms are written
   c) The direction of arrangement
   d) The type of data being sorted

2. Which of the following is an example of a numeric sorting criterion?
   a) Student names
   b) Exam scores
   c) Book titles
   d) File names

3. Which statement about sorting criteria is correct?
   a) They can only be numeric
   b) They can be numeric or alphanumeric
   c) They must be alphabetical
   d) They must be in descending order

4. Which of the following is an example of alphanumeric data?
   a) Temperatures
   b) Prices
   c) ID codes
   d) Time

5. Why is sorting used?
   a) To make programs run faster
   b) To organize data for easier search and analysis
   c) To reduce memory usage
   d) To prevent data duplication

Activity 4.3

Answer the questions below.

1. What is an algorithm?
   a) A program written in a specific language
   b) A series of steps designed to perform a task
   c) A list of variables
   d) A method for compiling code

2. What is a program?
   a) A set of instructions a computer can execute
   b) A flowchart showing an algorithm
   c) A memory location in a computer
   d) A sorting method

3. What does declaring a variable involve?
   a) Stating its label and assigning an initial value
   b) Sorting its contents
   c) Creating a loop for it
   d) Printing it to the screen

4. Which of the following controls the order in which program steps run?
   a) Sorting
   b) Variables
   c) Flow & control structures
   d) Program termination

5. Which is NOT a typical program block?
   a) Initialization
   b) Main body
   c) Termination
   d) Compilation

Activity 4.4

Answer the questions below.

1. What is the basic strategy of insertion sort?
   a) Pair, compare, swap
   b) Compare, shift, insert
   c) Merge, split, compare
   d) Select, swap, compare

2. What does the outer loop in insertion sort do?
   a) Compares adjacent pairs
   b) Holds an item and inserts it in its correct position
   c) Moves through each index and hands control to the inner loop
   d) Swaps items until sorted

3. What does the inner loop in insertion sort do?
   a) Pairs adjacent items for comparison
   b) Shifts items and inserts the held item in the correct position
   c) Selects the smallest value in the list
   d) Divides the list into halves

4. Which step is NOT part of insertion sort?
   a) Compare
   b) Shift
   c) Insert
   d) Swap

5. In the worst-case scenario, how is the dataset arranged for insertion sort?
   a) Already in the intended order
   b) In reverse of the intended order
   c) In random order
   d) Grouped by type

Activity 4.5

Answer the questions below.

1. What is the basic strategy of bubble sort?
   a) Compare, shift, insert
   b) Merge, compare, shift
   c) Pair, compare, swap
   d) Select, swap, compare

2. What does the outer loop in bubble sort do?
   a) Moves through the list starting at the second item
   b) Compares each item with all previous items
   c) Repeats the inner loop until a pass completes with no swaps
   d) Inserts items into the sorted section

3. What does the inner loop in bubble sort do?
   a) Shifts items forward
   b) Swaps adjacent items if out of order
   c) Holds an item for insertion
   d) Divides the list in half

4. How many moves are required for one swap in bubble sort?
   a) 1
   b) 2
   c) 3
   d) 4

5. In bubble sort, after each pass, which item is moved into its correct position?
   a) The largest item
   b) The smallest item
   c) The next item in the intended order
   d) The item with the highest index

Activity 4.6

Answer the questions below.

1. Which two measures are common for evaluating algorithm performance?
   a) Order and criterion
   b) Memory and variables
   c) Flow and control
   d) Step count and execution time

2. What is a worst-case scenario?
   a) A program without loops
   b) The least challenging arrangement of data
   c) The most challenging arrangement of data
   d) A dataset with missing values

3. What does scalability describe?
   a) How well an algorithm works on small datasets
   b) How performance changes as dataset size increases
   c) How memory is allocated in a program
   d) How variables are declared

4. What is memory latency?
   a) The speed of the processor
   b) The delay between requesting data from memory and receiving it
   c) The time taken to write a program
   d) The order in which data is stored

5. Why is insertion sort generally more efficient than bubble sort?
   a) It requires more swaps
   b) It works only on small datasets
   c) It requires fewer steps and less execution time
   d) It avoids using loops

Activity 4.7

In this activity, you will use a spreadsheet application to sort three different lists of numbers. You will then extract specific information from each list before sorting and after sorting, and reflect on the time and effort it took.

Download the spreadsheet here: bit.ly/Activity4_7

a. List 1 – 50 Random 6-Digit Numbers (100,000–999,999)

The smallest number: 102372

The largest number: 955156

The duplicate number: 312944

Find the number specified in the spreadsheet.

b. List 2 – 100 Random 6-Digit Numbers (100,000–999,999)

The smallest number: 102372

The largest number: 955156

The duplicate number: 704924

Find the number specified in the spreadsheet.

Did the larger dataset size make it easier or harder to find the data without sorting? Explain:

It harder to find values without sorting because there is more data to scan.

Sorting makes locating values much faster.

c. List 3 – 100 Random 6-Digit Numbers (222,222–333,333)

The smallest number: 222354

The largest number: 331588

The duplicate number: 319738

Find the number specified in the spreadsheet.

Did the larger dataset size make it easier or harder to find the data without sorting? Explain:

Numbers look more alike, making numbers harder to spot.

Sorting makes locating values much faster.

Activity 4.8

In this activity, you will work with data presented in table form, each representing a real-world scenario. Sort the data and extract the information needed to answer the questions. Use sorting and functions such as COUNT and AVERAGE to investigate and answer.

Download the spreadsheet here: bit.ly/Activity4_8

a. You are given an excerpt from a school database for Grades 7 and 8 with the following fields: Grade, Section, Student ID, and GPA.

1. How many students are in each grade?

Grade 7 — 32 ; Grade 8 — 35

2. If the school wants to recognize the top 3 students in each grade, who are they? List their student IDs.

Grade 7 — ID-2885, ID-2696, ID-7516

Grade 8 — ID-7309, ID-1489, ID-2598

3. If the school is offering support to students with a GPA below 2.00, how many students in each grade would qualify?

Grade 7 — 8 students

Grade 8 — 7 students

4. Looking at the number of students in each section, which section in each grade has the fewest students and which has the most?

Grade 7 — fewest A (9), most B (12)

Grade 8 — fewest B (10), most C (13)

5. If the school wants to recognize the teacher whose section has the highest average GPA, which section would win? Show the average GPA for each section to support your answer.

Grade 7 — A: 2.65, B: 2.60, C: 2.76 (highest)

Grade 8 — A: 2.59, B: 3.35 (highest), C: 2.88

b. You are given an excerpt from a logistics company's delivery database showing: Order Number, Route, Day of Delivery, and Time En Route (minutes).

1. Which order was delivered the fastest, and which was the slowest?

Fastest: 700182 (29 min) ; Slowest: 700125 (62 min)

2. For each delivery day, which order was delivered the fastest?

Monday: 700115 (35 min) ; Wednesday: 700139 (35 min) ; Friday: 700182 (29 min)

3. Deliveries that require more than 50 minutes are considered slow. Compare delivery days and routes by the number of slow deliveries. What stands out?

By day — Monday: 15, Wednesday: 6, Friday: 5

By route — Route 1: 7, Route 2: 4, Route 3: 15

Monday and Route 3 have the most slow deliveries.

4. Which route has the lowest average delivery time? Show the average time for each route to support your answer.

Route 2 — 44.10 min (best) ; Route 1 — 45.63 min ; Route 3 — 47.83 min

5. For each delivery day, which route has the lowest average delivery time? Recommend the best route to take on each day and support your answer with the average times.

Monday: Route 2 — 38.10 min

Wednesday: Route 1 — 38.90 min

Friday: Route 3 — 38.30 min

Notes

Use the tables in this lesson to explain that databases store information in tables, with fields (column headings) and entries (rows of data). Point out that sorting, counting, and averaging are basic operations used in database queries, laying the groundwork for more formal database concepts in later lessons.

Activity 4.9

Use the Insertion Sort algorithm to arrange the two given lists in ascending order. For each step, record the number of comparisons, shifts, and inserts you perform, and write the updated list. Keep a running total of the comparisons, shifts, inserts, and total operations at the bottom of the table.

a. [C, D, A, B]

Index	Hold	Compare	Shift	Insert	List
1	1	1	0	0	C, D, A, B
2	1	2	2	1	A, C, D, B
3	1	3	2	1	A, B, C, D
Total	3	6	4	2	15

b. [1, 3, 0, 2, 4, 5]

Index	Hold	Compare	Shift	Insert	List
1	1	1	0	0	1, 3, 0, 2, 4, 5
2	1	2	2	1	0, 1, 3, 2, 4, 5
3	1	2	1	1	0, 1, 2, 3, 4, 5
4	1	1	0	0	0, 1, 2, 3, 4, 5
5	1	1	0	0	0, 1, 2, 3, 4, 5
Total	5	7	3	2	17

Activity 4.10

Use the Bubble Sort algorithm to arrange the two given lists in ascending order. For each pass through the list, record the number of comparisons and swaps you perform, and write the updated list. Keep a running total of the comparisons, swaps, and total operations at the bottom of the table.

a. [B, C, D, A]

Pass	Compare	Swap	List
1	3	1	B, C, A, D
2	3	1	B, A, C, D
3	3	1	A, B, C, D
Total	9	3	12

b. [2, 4, 0, 3, 5, 1]

Pass	Compare	Swap	List
1	5	3	2, 0, 3, 4, 1, 5
2	5	2	0, 2, 3, 1, 4, 5
3	5	1	0, 2, 1, 3, 4, 5
4	5	1	0, 1, 2, 3, 4, 5
5	5	0	0, 1, 2, 3, 4, 5
Total	25	7	32

Activity 4.11

Review the pseudocode program for Insertion Sort. Then answer the comprehension questions provided.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
       SHIFT compare_data FROM compare_index TO compare_index + 1
       RESET compare_index (compare_index - 1)
       RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

1. Which two variables are declared in the initialization block?

hold_index, end_index

2. What does this instruction do: RESET compare_index (compare_index - 1)?

This instruction moves the comparison one position to the left.

3. What does this instruction do: REPEAT_UNTIL hold_index > end_index?

Ensures the program ends when it reaches the end of the list.

4. Which instruction moves the sorting to the next index in the list?

RESET hold_index (hold_index + 1)

5. Which instruction would you change to sort the list in descending order?

Change the instruction to: AS_LONG_AS hold_data < compare_data

Activity 4.12

Review the pseudocode program for Bubble Sort. Then answer the comprehension questions provided.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
    RESET swap_count (0)
    SET hold_index (1)

    REPEAT_UNTIL hold_index > end_index
       SET hold_data (data_item AT hold_index)
       SET compare_index (hold_index - 1)
       SET compare_data (data_item AT compare_index)

       IF compare_data > hold_data THEN
              SWAP_POSITIONS (compare_data, hold_data)
              RESET swap_count (swap_count + 1)

       RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

1. Which two variables are declared in the initialization block?

end_index, swap_count

2. What does this instruction do: REPEAT_UNTIL swap_count (0)

Repeats passes through the list until no swaps occur.

3. Which instruction moves the sorting to the next index in the list?

RESET hold_index (hold_index + 1)

4. What happens in RESET swap_count (swap_count + 1)?

It increases the swap counter by one each time a swap is made.

5. Which instruction would you change to sort the list in descending order?

Change the condition to IF compare_data < hold_data THEN.

Activity 4.13

Having learned the mechanics, performance, and logic behind insertion and bubble sort algorithms, it's time to explore them further. This activity allows students to examine these algorithms in action using provided Python programs. Follow the links to download the three programs for this activity or use the short link: bit.ly/Activity4_13.

1. Insertion Sort Program: Creates a list from all integers in a user-defined range (maximum 20 items). The list can be generated in worst-case (descending) or random order. The program shows the list after each completed insert (labeled by index) and reports totals for comparisons, shifts, and inserts.

2. Bubble Sort Program: Builds the same kind of list (worst-case or random). The program shows the list after each completed pass (labeled by pass number) and reports totals for comparisons and swaps.

3. Combined Sorting Program: This program includes both insertion and bubble sort. It generates a dataset from all integers in a user-defined range and runs both algorithms on the same list.
   - If the list size is 10,000 items or fewer, the program runs both algorithms and reports operation counts and execution time.
   - If the list size is larger than 10,000, the program offers a Theory Mode that displays the worst-case (reverse-sorted) operation counts without running the algorithms.

Notes

Notice the similarity between the pseudocode and the Python programs provided for insertion and bubble sort. This shows how pseudocode serves as a bridge between algorithmic logic and actual coding. By following the same variable names and structure, students can clearly see how the logic of each step is translated into Python. Encourage students to use pseudocode as a guide to understand sorting processes, compare algorithm performance, and develop their programming skills.